



**Il Metodo COSMIC di Misurazione
della Dimensione Funzionale**

Versione 3.0

Manuale di Misurazione

(Guida Implementativa COSMIC a ISO/IEC 19761:2003)

Settembre 2007

Ringraziamenti

Autori del COSMIC Core Team, Versione 2.0¹ (ordine alfabetico)

Alain Abran, École de technologie supérieure – Université du Québec

Jean-Marc Desharnais, Software Engineering Laboratory in Applied Metrics – SELAM

Serge Oigny, Bell Canada

Denis St-Pierre, DSA Consulting Inc.

Charles Symons, Software Measurement Services Ltd.

Revisori della versione 2.0, 1998/1999 (ordine alfabetico)		
Moritsugu Araki, JECS Systems Research, Giappone	Thomas Fetcke Germania	Patrice Nolin, Hydro Québec Canada
Fred Bootsma, Nortel Canada	Eric Foltin, University of Magdeburg Germania	Marie O'Neill, Software Management Methods, Irlanda
Denis Bourdeau, Bell Canada Canada	Anna Franco, CRSSM Canada	Jolijn Onvlee Olanda*
Pierre Bourque, École de Technologie supérieure, Canada	Paul Goodman, Software Measurement Services, Regno Unito	Laura Primera, UQAM Canada
Gunter Guerhen, Bürhen & Partner Germania	Nihal Kececi, University of Maryland USA	Paul Radford, Charismatek Australia
Sylvain Clermont, Hydro Québec Canada	Robyn Lawrie Australia	Eberhard Rudolph Germania
David Déry, CGI Canada	Ghislain Lévesque, UQAM Canada	Grant Rule, Software Measurement Services, Regno Unito*
Gilles Desoblins Francia	Roberto Meli, Data Processing Organization, Italia	Richard Stutzke, Science Applications Int'l Corporation, USA
Martin D'Souza, Total Metrics Australia	Pam Morris, Total Metrics Australia*	Ilionar Sylva, UQAM Canada
Reiner Dumke, University of Magdeburg, Germania	Risto Nevalainen, Software Technology Transfer, Finlandia*	Vinh T. Ho, UQAM Vietnam
Peter Fagg Regno Unito	Jin Ng, Hmaster Australia	

* Membri fondatori del COSMIC Core Team, insieme agli autori del metodo COSMIC-FFP.

¹ La versione 2.0 è stata la prima versione pubblicamente disponibile del metodo COSMIC-FFP, come denominato in origine.

Revisori della versione 3.0, 2006/2007 (ordine alfabetico)		
Alain Abran, École de Technologie Supérieure, Université du Québec Canada	Jean-Marc Desharnais, Software Engineering Lab in Applied Metrics – SELAM, Canada	Arlan Lesterhuis* Sogeti Olanda
Bernard Londeix Telmaco Regno Unito	Roberto Meli Data Processing Organization Italia	Pam Morris Total Metrics Australia
Serge Oligny Bell Canada Canada	Marie O'Neill Software Management Methods Irlanda	Tony Rollo Software Measurement Services Regno Unito
Grant Rule Software Measurement Services Regno Unito	Luca Santillo Agile Metrics Italia	Charles Symons* Regno Unito
Hannu Toivonen Nokia Siemens Networks Finland	Frank Vogelesang Sogeti Olanda	

* Curatori della versione 3.0 del metodo COSMIC.

Traduttori della versione 3.0 in italiano, Aprile 2010 (ordine alfabetico)		
Luigi Buglione Engineering.IT	Gianfranco Lanza CSI Piemonte	Guido Moretto InfoCamere
Sandra Paoletti InfoCamere	Marino Quirico Tecnet Dati	Luca Santillo* Agile Metrics
Habib Sedehi Università di Roma I "La Sapienza"		

* Curatore della traduzione in italiano della versione 3.0 del metodo COSMIC.

NdT Nel mese di maggio 2009 il COSMIC ha emesso la versione 3.0.1 del manuale di misurazione (includente tre bollettini di aggiornamento emessi dopo la pubblicazione della versione 3.0). Il gruppo di traduttori in italiano, a fronte della pubblicazione della presente traduzione della versione 3.0, emetterà la versione aggiornata 3.0.1 in italiano entro il prossimo semestre.

Per commenti sulla traduzione in italiano, scrivere a: Luca Santillo (lucasantillo@cosmicon.com).

Copyright 2007. Tutti i diritti riservati. Il Common Software Measurement International Consortium (COSMIC). Si concede il permesso di copiare tutto o parte di questo materiale purché le copie non siano fatte o distribuite per scopi commerciali, siano citati il titolo della pubblicazione, il numero di versione e la data, e sia data informativa che la copia avviene per concessione del Common Software Measurement International Consortium (COSMIC). Copie di altro genere richiedono un permesso specifico.

Una versione di pubblico dominio del Manuale di Misurazione COSMIC e altri documenti tecnici, incluse le traduzioni in altre lingue, sono disponibili presso il sito web www.cosmicon.com.

Controllo di Versione

La seguente tabella riporta la storia delle versioni di questo documento.

DATA	REVISORE/I	Modifiche / Aggiunte
31/03/99	Serge Oligny	Prima bozza, distribuita ai revisori per commenti.
31/07/99	V. Ringraziamenti	Versione revisionata con i commenti dei revisori.
6/10/99	V. Ringraziamenti	Versione revisionata con i commenti provenienti dal workshop IWSM „99 ² .
29/10/99	COSMIC Core Team	Versione revisionata con i commenti finali prima della pubblicazione della versione 2.0 per “field trial”.
1/05/01	COSMIC Core Team	Versione revisionata per conformità a ISO/IEC 14143-1:1998 + chiarimenti sulle regole di misurazione per la versione 2.1.
31/01/03	COSMIC Measurement Practices Committee	Versione revisionata per conformità a ISO/IEC FDIS 19761:2002 + ulteriori chiarimenti sulle regole di misurazione per la versione 2.2.
1/09/07	COSMIC Measurement Practices Committee	Versione revisionata con ulteriori chiarimenti e aggiunte alle regole di misurazione per la versione 3.0, in particolare sulla fase di Strategia della Misurazione. Nome del metodo modificato da „metodo COSMIC-FFP” a „metodo COSMIC”. Nell’aggiornamento dalla versione 2.2. alla versione 3.0, alcune parti del Manuale di Misurazione 2.2 sono state spostate in altri documenti (v. Prefazione e App. D).

² Atti dell’International Workshop on Software Measurement IWSM „99, Lac Supérieur, Québec, Canada, 8-10 Settembre 1999.

Prefazione

Scopo del metodo COSMIC (Common Software Measurement International Consortium) è fornire un metodo standardizzato di misurazione della dimensione funzionale del software nei domini funzionali comunemente denotati come software “applicativo aziendale” (“business application”, o “gestionale” – “Management Information System”, MIS) e software “real-time”.

Il metodo COSMIC è stato accettato dal Joint Technical Committee (JTC) 1, Sub-Committee (SC) 7 dell'ISO/IEC nel Dicembre 2002 quale Standard Internazionale ISO/IEC 19761 “Software Engineering – COSMIC-FFP – A functional size measurement method” (in seguito indicato con “ISO/IEC 19761”).

Per chiarezza, lo standard ISO/IEC 19761 contiene le definizioni e le regole normative fondamentali del metodo. Scopo del presente Manuale di Misurazione è non solo fornire queste regole e definizioni, ma anche ulteriori spiegazioni e molti più esempi per aiutare i misuratori a comprendere a fondo e ad applicare il metodo. Tuttavia, all'aumentare delle esperienze con il metodo, si è ritenuto significativo fare delle aggiunte alle regole e agli esempi e in alcuni casi perfezionare le definizioni dei relativi concetti. Il COSMIC prevede di sottoporre tali aggiunte e perfezionamenti all'ISO per includerli nello standard ISO/IEC 19761, in occasione della revisione prevista nel 2007/2008.

Il Manuale di Misurazione è uno dei quattro documenti COSMIC che definiscono la versione 3.0 del metodo. Gli altri tre documenti sono:

- “Documentation Overview and Glossary of Terms”. (Il Glossario definisce tutti i termini comuni all'intera documentazione COSMIC. Questo documento descrive anche altra documentazione importante, come i casi di studio e le linee guida per domini particolari.)
- “Method Overview”.
- “Advanced and Related Topics”. (Questo documento tratta in maggior dettaglio la questione della confrontabilità tra diverse misurazioni della dimensione e comprende inoltre capitoli sul dimensionamento approssimato rapido o anticipato e sulla convertibilità di misurazioni, temi già trattati nella precedente versione 2.2 del Manuale di Misurazione.)

Ai lettori nuovi del tema della “misurazione della dimensione funzionale” (Functional Size Measurement, FSM), o che hanno familiarità con altre metodologie, si consiglia vivamente di leggere il documento “Method Overview” prima del Manuale di Misurazione.

Principali cambiamenti della versione 3.0 del metodo COSMIC

Il cambio di versione del metodo COSMIC da 2.2 a 3.0 indica che la nuova versione rappresenta un significativo progresso rispetto alla precedente. I principali cambiamenti apportati nel produrre questa versione 3.0 del metodo COSMIC a partire dalla precedente versione definita nel Manuale di Misurazione versione 2.2 sono di seguito elencati.

- Per rendere la documentazione del metodo COSMIC più “facile da usare”, la versione 3.0 del metodo è ora definita nei quattro documenti distinti sopra elencati.
- Sono stati incorporati i contenuti di due “Method Update Bulletins (MUB)”, pubblicati successivamente alla versione 2.2 del Manuale di Misurazione. Si tratta del MUB 1 “Proposed Improvements to the Definition and Characteristics of a software „Layer”” e del MUB 2 “Proposed Improvement to the Definition of an „object of interest””.
- È stata definita a parte una fase di “Strategia di Misurazione”, come prima fase del processo di misurazione. La fase della strategia è arricchita inoltre da indicazioni sul concetto di “livello di granularità” dei Requisiti Utente Funzionali (Functional User Requirements, FUR) del software oggetto di misurazione, per assicurare la confrontabilità delle misurazioni tra porzioni distinte di software.

- Dall'esperienza è emerso che i concetti di "punto di vista della misurazione dell'utente finale" e "punto di vista della misurazione dello sviluppatore" (introdotti nella versione 2.2) possono essere sostituiti dal concetto più semplice di "utente funzionale". Quest'ultimo si può genericamente definire come il mittente o il destinatario designato dei dati nei FUR del software oggetto di misurazione. Tutte le misurazioni di dimensione di una porzione di software sono quindi svolte sulla base delle funzionalità fornite agli utenti funzionali del software, così come identificati nei relativi FUR.
- Il nome dell'unità di misura del metodo è stato cambiato da "unità di dimensione funzionale COSMIC" (COSMIC functional size unit, Cfsu) a "Function Point COSMIC" (COSMIC Function Point, CFP). Tale modifica è stata apportata per agevolare la lettura e la pronuncia e per conformità con altri metodi di tipo "Function Point". Come ulteriore semplificazione, il nome del metodo è stato cambiato da "(metodo) COSMIC-FFP" a "(metodo) COSMIC".
- Il glossario è stato aggiornato e migliorato per motivi di leggibilità, ed è stato spostato nel nuovo documento "Documentation Overview and Glossary of Terms".
- È stato rimosso materiale riguardante in particolare alcuni concetti di analisi dei dati. Questo materiale è ora incluso nel documento "Guideline for Sizing Business Application Software", trattandosi di un tema specifico di tale ambito e non riferibile al metodo COSMIC in generale.
- Sono state apportate varie migliorie e aggiunte di tipo editoriale, per aumentare la coerenza della terminologia e migliorare la comprensibilità. Tra queste, al riguardo, è stata apportata una distinzione più netta tra "principi" e "regole" del metodo, adottando una convenzione tratta dal mondo della contabilità, secondo la quale "le regole esistono per applicare le definizioni e i principi". Sia i principi che le regole dovrebbero essere considerati obbligatori. Pertanto la maggior parte degli esempi è stata spostata dai principi e dalle regole al corpo del testo che li segue.

Tutti i cambiamenti apportati sono riportati nell'Appendice D.

I lettori familiari con la versione 2.2 del Manuale di Misurazione noteranno il maggior numero di cambiamenti nella nuova fase a se stante della "Strategia di Misurazione".

Conseguenze dei cambiamenti principali sulle misurazioni passate

I principi originali del metodo COSMIC sono rimasti invariati fin dalla pubblicazione della prima bozza del Manuale di Misurazione nel 1999, nonostante i perfezionamenti e le aggiunte successive richieste per produrre lo Standard Internazionale e apportate con le versioni 2.1, 2.2 e 3.0 del manuale.

Le dimensioni funzionali misurate secondo i principi e le regole di questa versione 3.0 del Manuale di Misurazione possono differire da quanto misurato con le precedenti versioni solo perché le nuove regole sono volutamente più precise. Ne consegue che i misuratori godono di minore discrezionalità nell'interpretare le regole rispetto alle versioni precedenti. Le modifiche nell'ambito della Strategia di Misurazione e il cambio di nome dell'unità di misura conducono a differenze banali nelle regole di presentazione dei risultati di misurazione rispetto alle versioni precedenti.

Ragione dei principali cambiamenti da versione 2.2 a versione 3.0 del manuale di misurazione

Prima di tutto è il caso di sottolineare che l'unità di misura COSMIC ("Cfsu", ora "CFP"), è rimasta invariata dalla sua introduzione con la prima pubblicazione del Manuale di Misurazione COSMIC-FFP nel 1999. Si tratta dell'equivalente COSMIC per es. del metro quale unità di misura standard della lunghezza. Tuttavia la dimensione funzionale di una porzione di software può essere misurata in molti modi, ed è talvolta problematico, per qualunque metodo di misurazione della dimensione funzionale (FSM), rispondere alla domanda: "quale dimensione dovremmo misurare?"

Il primo problema è che sappiamo che una qualsiasi porzione di software fornisce funzionalità a vari tipi di "utenti", laddove un utente è definito nella terminologia dello standard ISO/IEC 14143-1 ("Principles of FSM") essenzialmente come "qualsiasi cosa che interagisce con il software oggetto di misurazione". Ne consegue che la dimensione funzionale di una porzione di software dipende da chi o cosa è definito come suo utente/i.

Consideriamo ad esempio il software applicativo di un telefono cellulare. Gli “utenti” potrebbero essere: la persona che “schiaccia i bottoni”, le periferiche hardware (per es. il video, la tastiera, ecc.) che interagiscono con l'applicazione, oppure il sistema operativo che supporta l'applicazione o addirittura altre porzioni alla pari di software separate che interagiscono con l'applicazione oggetto di misurazione. Ognuno di questi tipi di utente necessita di funzionalità diverse (quindi la dimensione funzionale risulta diversa a seconda di chi o cosa è definito come utente). Come possiamo dunque stabilire, a fronte di una particolare dimensione funzionale data, chi o cosa erano gli utenti considerati, oltretutto quali funzionalità sono state effettivamente misurate?

È stato questo problema a portare inizialmente il COSMIC Measurement Practices Committee (MPC) a introdurre nella versione 2.2 del Manuale di Misurazione i concetti di Punti di Vista della Misurazione “dell'Utente Finale” e “dello Sviluppatore”. Tuttavia, è emerso dall'esperienza che tali definizioni, in particolare il Punto di Vista della Misurazione dello Sviluppatore, non si sono dimostrate sufficientemente di aiuto nell'identificare tutte le possibili esigenze di misurazione. L'MPC ha concluso che l'approccio corretto e più generale è quello di definire il concetto di “utente funzionale” e che la dimensione funzionale varia a seconda di chi o cosa è definito come utente funzionale. L'identificazione degli utenti funzionali dipende dallo scopo della misurazione e gli utenti funzionali dovrebbero risultare normalmente identificabili nei Requisiti Utente Funzionali (FUR) del software oggetto di misurazione. L'idea di definire specifici “Punti di Vista della Misurazione” non è quindi più necessaria.

Il secondo problema è che è noto che i FUR evolvono all'avanzare del progetto e, a seconda di come si misura la dimensione, può sembrare che questa cresca durante il progetto. La prima versione dei FUR per una nuova porzione di software può essere specificata a un livello “alto”. Con l'avanzare del progetto e l'elaborazione più dettagliata dei requisiti, i FUR vengono specificati con maggiore dettaglio, o a un livello “più approfondito”, e può sembrare che la loro dimensione aumenti. Distinguiamo questi differenti livelli di dettaglio come “livelli di granularità”.

Dunque, il problema da affrontare adesso è: come possiamo sapere se due differenti misurazioni sono state svolte allo stesso livello di granularità? Questa versione 3.0 del Manuale di Misurazione fornisce raccomandazioni su tale questione, particolarmente importante laddove le misurazioni siano svolte nelle prime fasi del progetto, quando i FUR sono ancora in evoluzione. La questione diventa cruciale quando le dimensioni sono usate per misure di performance da confrontare tra fonti diverse, come avviene per es. nelle attività di benchmarking.

È importante sottolineare che questi nuovi concetti di “utente funzionale” e di “livello di granularità”, ed i relativi processi di identificazione introdotti nella Strategia di Misurazione non sono necessariamente riferibili al solo metodo COSMIC. Essi sono validi per tutti i metodi FSM. Essendo il metodo COSMIC basato su principi consolidati di ingegneria del software ed essendo applicabile a una gamma di domini software più ampia rispetto ai metodi FSM di “prima generazione”, il problema di stabilire correttamente: “quale dimensione dovremmo misurare?” è stato individuato ed è stata trovata una soluzione.

Quando lo scopo della misurazione è correlato alle performance (per es. per stime, benchmarking, ecc.), la maggior parte dei misuratori con il metodo COSMIC non dovrà spendere tempo nell'identificare gli “utenti funzionali” o nello stabilire il “livello di granularità” rispetto al quale misurare, poiché questi risulteranno ovvi. Tuttavia, nel caso di misurazioni per le quali tali scelte non siano ovvie, i nuovi contenuti della “Fase di Strategia della Misurazione” nel presente Manuale di Misurazione e il capitolo sulla confrontabilità delle misurazioni della dimensione nel documento “Advanced and Related Topics” discutono gli aspetti da considerare in maggior dettaglio.

Il COSMIC Measurement Practices Committee

Settembre 2007

1	INTRODUZIONE	10
1.1	Applicabilità del metodo COSMIC	10
1.1.1	<i>Domini di applicabilità</i>	<i>10</i>
1.1.2	<i>Dominio di non applicabilità</i>	<i>10</i>
1.1.3	<i>Limiti sui fattori che contribuiscono alla dimensione funzionale</i>	<i>10</i>
1.1.4	<i>Limiti sulla misurazione di porzioni molto piccole di software</i>	<i>11</i>
1.2	I Requisiti Utente Funzionali	11
1.3	Il Modello del Contesto del Software COSMIC	13
1.4	Il Modello Generale del Software COSMIC	14
1.5	Il Processo di Misurazione COSMIC	15
2	LA FASE DI STRATEGIA DELLA MISURAZIONE	16
2.1	Definire lo scopo della misurazione	17
2.1.1	<i>Lo scopo della misurazione determina la particolare dimensione che sarà misurata</i>	<i>17</i>
2.1.2	<i>Esempi di scopi di misurazione</i>	<i>17</i>
2.2	Definire l'ambito della misurazione	18
2.2.1	<i>L'ambito della misurazione dipende dallo scopo</i>	<i>19</i>
2.2.2	<i>Esempi di ambito di misurazione e livelli di scomposizione</i>	<i>20</i>
2.2.3	<i>Strati</i>	<i>21</i>
2.2.4	<i>Componenti alla pari</i>	<i>23</i>
2.3	Identificare gli utenti funzionali	24
2.3.1	<i>La dimensione funzionale varia con l'utente funzionale</i>	<i>24</i>
2.3.2	<i>Utenti funzionali</i>	<i>24</i>
2.4	Identificare il livello di granularità	26
2.4.1	<i>La necessità di un livello di granularità standard</i>	<i>26</i>
2.4.2	<i>Chiarimento sul "livello di granularità"</i>	<i>27</i>
2.4.3	<i>Il livello di granularità standard</i>	<i>27</i>
2.4.4	<i>Un esempio di funzionalità a differenti livelli di granularità</i>	<i>28</i>
2.5	Osservazioni finali sulla fase di strategia della misurazione	31
3	LA FASE DI MAPPATURA	32
3.1	Applicare il Modello Generale del Software	33
3.2	Identificare i processi funzionali	34
3.2.1	<i>Definizioni</i>	<i>34</i>
3.2.2	<i>Approccio all'identificazione dei processi funzionali</i>	<i>35</i>
3.2.3	<i>Esempi nel dominio degli applicativi aziendali</i>	<i>36</i>
3.2.4	<i>Esempi nel dominio delle applicazioni real-time</i>	<i>37</i>
3.2.5	<i>Ulteriori considerazioni sui processi funzionali distinti</i>	<i>38</i>
3.3	Identificare gli oggetti di interesse e i gruppi di dati	38
3.3.1	<i>Definizioni e principi</i>	<i>38</i>
3.3.2	<i>Sulla materializzazione di un gruppo di dati</i>	<i>40</i>
3.3.3	<i>Sull'identificazione di oggetti di interesse e gruppi dati</i>	<i>40</i>
3.3.4	<i>Gruppi di dati che non sono candidati ad essere movimenti di dati</i>	<i>41</i>
3.3.5	<i>L'utente funzionale come oggetto d'interesse</i>	<i>41</i>
3.4	Identificare gli attributi dei dati (opzionale)	42
3.4.1	<i>Definizione</i>	<i>42</i>
3.4.2	<i>Sulla relazione tra attributi dei dati e gruppi di dati</i>	<i>42</i>

4	LA FASE DI MISURAZIONE	43
4.1	Identificare i movimenti di dati	43
4.1.1	<i>Definizione dei tipi di movimento di dati.....</i>	44
4.1.2	<i>Identificare gli Entry (E).....</i>	45
4.1.3	<i>Identificare gli Exit (X).....</i>	46
4.1.4	<i>Identificare i Read (R).....</i>	47
4.1.5	<i>Identificare i Write (W).....</i>	47
4.1.6	<i>Sulle manipolazioni dei dati associate ai movimenti di dati.....</i>	48
4.1.7	<i>Unicità dei movimenti di dati e possibili eccezioni.....</i>	49
4.1.8	<i>Quando un processo funzionale muove dati da o verso la memoria permanente.....</i>	51
4.1.9	<i>Quando un processo funzionale richiede dati da un utente funzionale.....</i>	54
4.1.10	<i>Comandi di controllo.....</i>	56
4.2	Applicare la funzione di misurazione.....	57
4.3	Aggregare i risultati della misurazione	57
4.3.1	<i>Regole generali di aggregazione.....</i>	57
4.3.2	<i>Ulteriori informazioni sull'aggregazione delle dimensioni funzionali.....</i>	58
4.4	Ulteriori informazioni sulla dimensione delle modifiche del software	59
4.4.1	<i>Modifica delle funzionalità.....</i>	60
4.4.2	<i>Dimensione del software funzionalmente modificato.....</i>	61
4.5	Estendere il metodo di misurazione COSMIC	61
4.5.1	<i>Introduzione.....</i>	61
4.5.2	<i>Estensione locale con algoritmi complessi.....</i>	61
4.5.3	<i>Estensione locale con sotto-unità di misura.....</i>	61
5	PRESENTAZIONE DELLE MISURAZIONI.....	63
5.1	Etichettatura	63
5.2	Archiviazione	64
	APPENDICE A – DOCUMENTARE UNA MISURAZIONE DI DIMENSIONE COSMIC	65
	APPENDICE B – SOMMARIO DEI PRINCIPI DEL METODO COSMIC	66
	APPENDICE C – SOMMARIO DELLE REGOLE DEL METODO COSMIC	69
	APPENDICE D – STORIA DEI RILASCI DEL METODO COSMIC	75
	APPENDICE E – PROCEDURA PER RICHIESTE DI MODIFICA E COMMENTI	79

INTRODUZIONE

1.1 Applicabilità del metodo COSMIC

1.1.1 Domini di applicabilità

Il metodo COSMIC per la misurazione della dimensione funzionale è progettato per essere applicabile alle funzionalità di software dei seguenti domini:

- Software applicativo aziendale, tipicamente richiesto in supporto alla gestione dei processi aziendali, come il software bancario, assicurativo, per la contabilità, il personale, gli acquisti, la distribuzione o la produzione. Questo tipo di software è spesso denotato come “a prevalenza di dati” (“data rich”), poiché la sua complessità è largamente dominata dalla necessità di gestire grandi quantità di dati riguardanti eventi nel mondo reale.
- Software real-time, avente il compito di seguire o governare eventi che accadono nel mondo reale. Esempi sono il software per centraline telefoniche e per commutazione di messaggi, il software integrato (“embedded”) in periferiche di controllo di macchinari come elettrodomestici, ascensori, motori di automobili e velivoli, il software per controllo di processo e acquisizione automatica di dati e il software incorporato nei sistemi operativi per computer.
- Ibridi di quanto sopra, come per es. nei sistemi di prenotazione in tempo reale per le linee aeree o gli alberghi.

1.1.2 Dominio di non applicabilità

Il metodo di misurazione COSMIC non è ancora stato progettato per prendere in considerazione le funzionalità di software “mathematically-intensive”, software cioè caratterizzato da algoritmi matematici complessi o da altre regole specializzate e complesse, come quelle presenti nei sistemi esperti, nel software di simulazione, nel software ad auto-apprendimento, nei sistemi di previsioni meteorologiche, ecc., o che elaborano variabili continue quali suoni (audio) o immagini (video), come quelle contenute per es. nel software per videogiochi, negli strumenti musicali, ecc.

Per tali tipi di software è possibile, tuttavia, definire estensioni locali del metodo di misurazione COSMIC. Il Manuale di Misurazione illustra in quali contesti si dovrebbero usare tali estensioni locali e ne fornisce degli esempi. Se usate, le estensioni locali devono essere presentate attenendosi alle convenzioni riportate nel capitolo riguardante la presentazione delle misurazioni del presente Manuale di Misurazione.

1.1.3 Limiti sui fattori che contribuiscono alla dimensione funzionale

Inoltre, nei domini in cui è applicabile, il metodo COSMIC non pretende di misurare tutti i possibili aspetti delle funzionalità che si potrebbe pensare contribuiscano alla “dimensione” del software. Per es. il metodo di misurazione non coglie né l’impatto della complessità (comunque sia definita), né l’impatto del numero di attributi dei dati per ciascun movimento di dati sulla dimensione funzionale del software (per una discussione di quest’ultimo punto, v. il capitolo sulla Fase di Mappatura). Come descritto nella sez. 1.1.2, se lo si desidera, tali aspetti della dimensione funzionale possono essere presi in considerazione attraverso un’estensione locale del metodo di misurazione COSMIC.

1.1.4 Limiti sulla misurazione di porzioni molto piccole di software

Tutti i metodi di misurazione della dimensione funzionale si basano sull'assunzione di un modello semplificato delle funzionalità del software che si presuppone essere "in media" ragionevole per il dominio di applicabilità designato. Occorre quindi cautela nel misurare, confrontare o utilizzare (per es. per scopi di stima) le dimensioni di porzioni molto piccole di software, e in particolar modo di modifiche molto piccole apportate a una porzione di software, dove l'assunzione "in media" può non essere più valida. Nel caso del metodo COSMIC, "molto piccolo" significa "pochi movimenti di dati".

1.2 I Requisiti Utente Funzionali

Il metodo di misurazione COSMIC comporta l'applicazione di un insieme di modelli, principi, regole e procedure ai **Requisiti Utente Funzionali (Functional User Requirements, FUR)** di una data porzione di software. Il risultato è un "valore (numerico) di una quantità" (come definito dall'ISO), che rappresenta la dimensione funzionale della porzione del software secondo il metodo COSMIC.

Le dimensioni funzionali misurate con il metodo COSMIC sono progettate per essere indipendenti da qualsiasi decisione implementativa racchiusa nei manufatti operativi del software oggetto di misurazione. Le "funzionalità" si riferiscono alle "elaborazioni di informazioni che il software deve svolgere per soddisfare le esigenze dell'utente".

Più precisamente, un'affermazione dei FUR descrive "cosa" il software deve fare per gli utenti funzionali che sono i mittenti e i destinatari designati dei dati da e verso il software. Un'affermazione dei FUR esclude qualsiasi requisito tecnico o qualitativo che affermi "come" il software deve svolgersi. (Per la definizione formale di FUR, v. sez. 2.2). Quando si misura una dimensione funzionale, si prendono in considerazione solo i FUR.

Estrarre i requisiti utente funzionali dai manufatti del software in pratica

Nel mondo reale dello sviluppo software è raro trovare dei manufatti per il software in cui tutti i FUR siano radicalmente distinti da altri tipi di requisiti e siano espressi in una forma adatta per una misurazione diretta senza alcuna necessità di interpretazione. Questo significa che abitualmente il misuratore dovrà estrarre i FUR così come sono forniti oppure presenti implicitamente negli effettivi manufatti del software, prima di mapparli sui concetti dei "modelli del software" del metodo COSMIC.

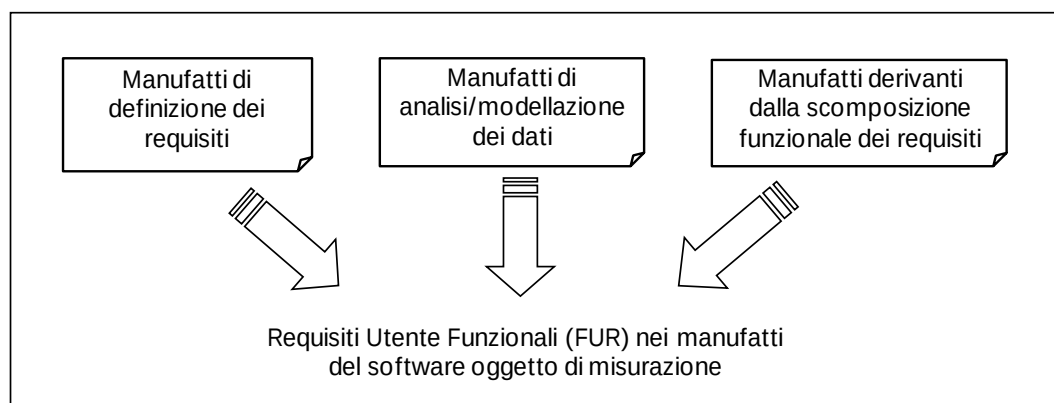


Figura 1.2.1 – Modello COSMIC dei requisiti utente funzionali pre-implementazione.

Come mostrato in Fig. 1.2.1, i FUR possono essere derivati dai manufatti di ingegneria del software che sono prodotti ancor prima del prodotto software. Pertanto la dimensione funzionale del software può essere misurata prima della sua implementazione in un sistema informatico.

È importante notare che i manufatti prodotti prima dell'implementazione del software, per es. durante la raccolta dei requisiti o l'analisi, possono descrivere il software a diversi "livelli di granularità", man mano che i requisiti evolvono – v. sez. 2.4 successiva.

NOTA. I requisiti utente funzionali possono essere prodotti prima di essere assegnati all'hardware o al software. Poiché il metodo COSMIC si pone l'obiettivo di dimensionare i FUR di una porzione di software, solamente i requisiti assegnati al software sono misurati. Tuttavia, in linea di principio il metodo COSMIC può essere applicato ai FUR prima della loro assegnazione al software o all'hardware, indipendentemente dall'eventuale decisione di allocazione. Per es. è facile dimensionare le funzionalità di una calcolatrice con il metodo COSMIC senza alcuna conoscenza di quale hardware o software (se presente) sia effettivamente utilizzato. Ad ogni modo, l'affermazione che il metodo COSMIC può essere utilizzato per dimensionare i FUR assegnati all'hardware necessita di ulteriori verifiche nella pratica prima di potersi considerare pienamente validata senza la necessità di ulteriori regole.

Derivare i requisiti utente funzionali dal software installato

In altre circostanze, delle porzioni esistenti di software possono richiedere di essere misurate in assenza di, o con solo pochi, manufatti architeturali o di progettazione a disposizione, e i FUR potrebbero non essere documentati (per es. nel caso di software "legacy"). In questi casi, è ancora possibile derivare i FUR dai manufatti installati nel sistema informatico, anche successivamente alla sua implementazione, come illustrato in Fig. 1.2.2.

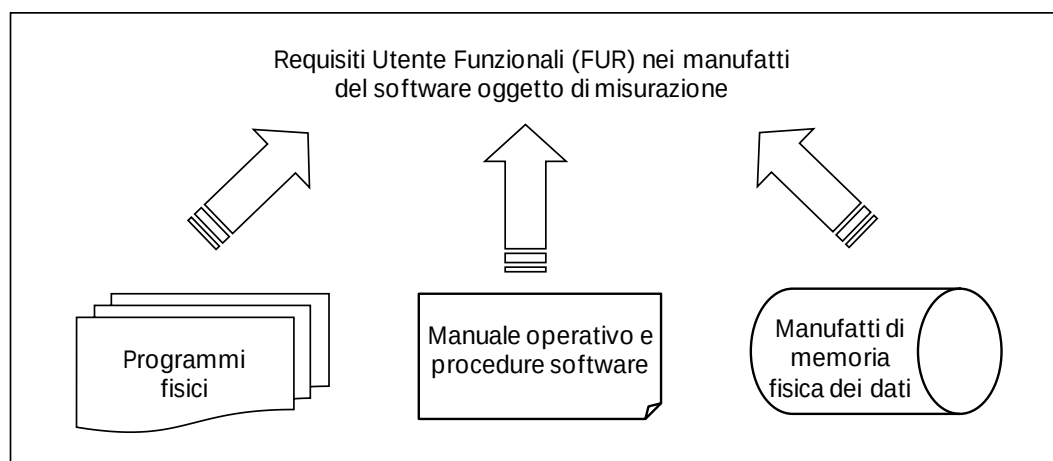


Figura 1.2.2 – Modello COSMIC dei requisiti utente funzionali post-implementazione.

Estrarre o derivare i requisiti utente funzionali dai manufatti del software

Il processo da utilizzare e quindi l'impegno necessario per estrarre i FUR dai diversi tipi di manufatti di ingegneria del software, oppure per dedurli dal software installato, e per esprimerli nella forma richiesta per misurarli con il metodo COSMIC, varieranno ovviamente da caso a caso; questi processi non possono essere trattati nel presente Manuale di Misurazione. Si assume che i requisiti utente funzionali del software oggetto di misurazione o esistono già o possono essere estratti o derivati dai relativi manufatti in funzione dello scopo della misurazione.

Il Manuale di Misurazione si limita quindi a descrivere e definire i concetti dei modelli del software COSMIC, cioè gli obiettivi dell'estrazione o deduzione³. Questi concetti sono incorporati in un insieme di principi esposti in due modelli del software COSMIC, il "Modello del Contesto del Software" e il "Modello Generale del Software".

³ Il documento "Guideline for Sizing Business Application Software" fornisce indicazioni per mappare vari metodi di analisi dei dati e di definizione dei requisiti usati nel dominio delle applicazioni aziendali sui concetti del metodo COSMIC.

Questi due modelli sono descritti nel documento “Method Overview”. Si rimandano i lettori interessati alla comprensione e illustrazione di tali modelli a questo documento. Per comodità, tali modelli sono riportati di seguito e sono ulteriormente dettagliati rispettivamente nei Cap. 2 e 3 del presente Manuale di Misurazione.

1.3 Il Modello del Contesto del Software COSMIC

Una porzione di software oggetto di misurazione secondo il metodo COSMIC deve essere ben definita (nell’ambito della misurazione) e tale definizione deve tener conto del suo contesto relativamente a qualsiasi altro software e/o hardware con cui essa interagisce. Il Modello del Contesto del Software introduce i principi e i concetti necessari per tale definizione.

N.B. I termini riportati in grassetto al loro primo utilizzo nelle sez. 1.3 e 1.4 sono utilizzati con un significato specifico del metodo COSMIC. Per le definizioni formali, v. il glossario nel documento “Documentation Overview and Glossary of Terms”. Tutti questi termini sono spiegati in maggior dettaglio nei successivi capitoli 2, 3 e 4.

PRINCIPI – Modello del Contesto del Software COSMIC
a) Il software è delimitato dall’hardware. b) Il software è tipicamente strutturato in strati. c) Uno strato può contenere una o più porzioni di software “alla pari” separate e ogni porzione di software può ulteriormente consistere di componenti alla pari separate. d) Ogni porzione di software oggetto di misurazione è individuata dal proprio ambito di misurazione, che deve essere confinato interamente in un singolo strato. e) L’ambito di una porzione di software oggetto di misurazione dipende dallo scopo della misurazione. f) Gli utenti funzionali di una porzione di software sono identificati dai requisiti utente funzionali della porzione di software oggetto di misurazione come i mittenti e/o destinatari designati dei dati. g) Una porzione di software interagisce con i propri utenti funzionali tramite movimenti di dati attraverso un confine e può muovere i dati da e verso la memoria permanente all’interno del confine. h) I FUR del software possono essere espressi a differenti livelli di granularità. i) Il livello di granularità al quale si dovrebbero effettuare normalmente le misurazioni è quello dei processi funzionali. j) Se non è possibile misurare al livello di granularità dei processi funzionali, i FUR del software dovrebbero allora essere misurati con un approccio di approssimazione e riscalati al livello di granularità dei processi funzionali⁴.

I concetti relativi al Modello del Contesto del Software sono elaborati nel Cap. 2 “Fase di Strategia della Misurazione” del presente Manuale di Misurazione.

⁴ I temi del “dimensionamento approssimato” (cioè della stima di una dimensione quando non sono disponibili tutti i dettagli per misurare una dimensione accurata) e di “scaling” tra differenti livelli di granularità sono affrontati nel documento “Advanced and Related Topics” del metodo COSMIC.

1.4 Il Modello Generale del Software COSMIC

Avendo interpretato i FUR del software oggetto di misurazione in termini del Modello del Contesto del Software, applichiamo ora il Modello Generale del Software ai FUR per identificare le componenti delle funzionalità che saranno misurate. Il Modello Generale del Software assume che per qualsiasi software che possa essere misurato con il metodo COSMIC valgano i seguenti principi generali. (Come riportato nel glossario, qualsiasi metodo di misurazione della dimensione funzionale si pone l'obiettivo di identificare i "tipi" e non le "occorrenze" dei dati o delle funzioni. Nel testo seguente si ometterà perciò il prefisso "tipo (di)" ogniqualvolta venga menzionato un concetto base del metodo COSMIC, salvo laddove sia indispensabile distinguere i "tipi" dalle "occorrenze".)

PRINCIPI – Modello Generale del Software COSMIC
a) Il software riceve dati di input dai propri utenti funzionali e produce un output e/o un altro risultato per gli utenti funzionali.
b) I requisiti utente funzionali di una porzione di software oggetto di misurazione possono essere mappati su processi funzionali unici.
c) Ogni processo funzionale consiste di sotto-processi .
d) Un sotto-processo può essere o un movimento di dati o una manipolazione di dati .
e) Ogni processo funzionale è innescato da un movimento di dati di Entry da parte di un utente funzionale che informa il processo funzionale che l'utente funzionale ha identificato un evento .
f) Un movimento di dati muove un singolo gruppo di dati .
g) Un gruppo di dati consiste di un insieme unico di attributi dei dati che descrivono un singolo oggetto d'interesse .
h) Esistono quattro tipi di movimenti di dati. Un Entry muove un gruppo di dati all'interno del software da parte di un utente funzionale. Un Exit muove un gruppo di dati fuori del software verso un utente funzionale. Un Write muove un gruppo di dati dal software verso la memoria permanente. Un Read muove un gruppo di dati dalla memoria permanente verso il software.
i) Un processo funzionale include almeno un movimento di dati di Entry e un movimento di dati o di Write o di Exit, oltretutto include un minimo di due movimenti di dati.
j) Come approssimazione per scopi di misurazione, i sotto-processi di manipolazione dei dati non sono misurati separatamente; la/e funzionalità di una qualsiasi manipolazione di dati si assumono come incluse nel movimento di dati a cui essa è associata.

I concetti inerenti il Modello Generale del Software sono elaborati nel Cap. 3 "Fase di Mappatura" del presente Manuale di Misurazione.

Una volta che i FUR oggetto di misurazione sono stati mappati sul Modello Generale del Software, questi possono essere misurati in accordo con il processo della Fase di Misurazione (Cap. 4). I risultati della misurazione dovrebbero poi essere presentati attenendosi alle convenzioni di "Presentazione delle Misurazioni" (Cap. 5).

1.5 Il Processo di Misurazione COSMIC

Il processo generale di misurazione del metodo COSMIC consiste di tre fasi:

- la Strategia di Misurazione, in cui si applica il Modello del Contesto del Software al software oggetto di misurazione (Cap. 2)
- la Fase di Mappatura, in cui si applica il Modello Generale del Software al software oggetto di misurazione (Cap. 3)
- la Fase di Misurazione, in cui si ottiene la misura effettiva (Cap. 4)

Il risultato dell'applicazione del processo di misurazione a una porzione di software è la misura della dimensione funzionale dei Requisiti Utente Funzionali di tale porzione di software espressa in "Function Point COSMIC" (COSMIC Function Point, CFP).

Le relazioni tra le tre fasi del metodo COSMIC sono illustrate in Fig. 1.5.1.

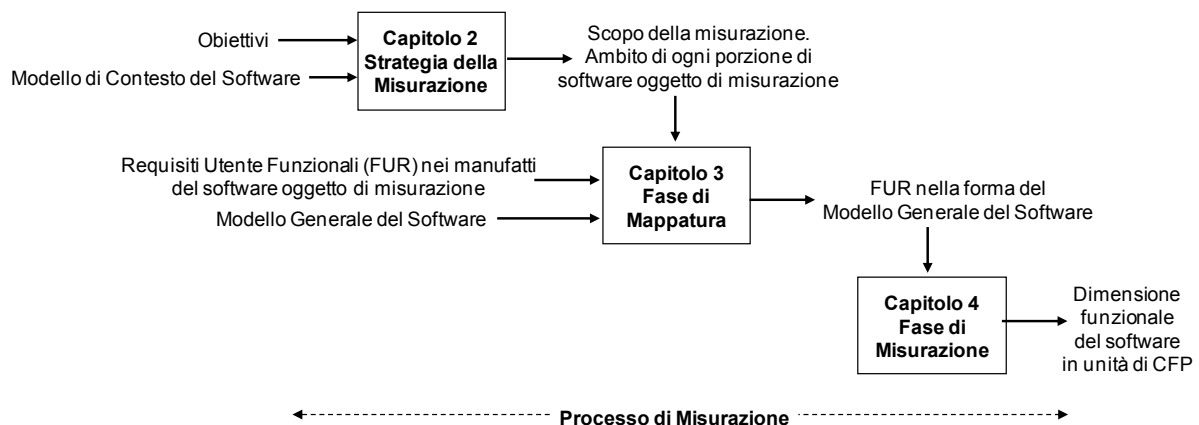


Figura 1.5.1 – Struttura del metodo COSMIC.

LA FASE DI STRATEGIA DELLA MISURAZIONE

Questo capitolo tratta i quattro parametri fondamentali della misurazione funzionale del software da considerare prima di iniziare effettivamente a misurare, ossia lo scopo e l'ambito della misurazione, l'identificazione degli utenti funzionali ed il livello di granularità che dovrebbe essere misurato. Determinare tali parametri aiuta ad affrontare le questioni relative a “quale dimensione si dovrebbe misurare?” o, per una misurazione esistente, “come si dovrebbe interpretare tale misurazione?”.

È molto importante registrare i dati che hanno origine da questa fase di Strategia della Misurazione (come elencato in sez. 5.2) al momento dell'archiviazione dei risultati di ogni misurazione. Errori nel definire e registrare tali parametri in modo coerente condurranno a misurazioni che non si potranno interpretare in modo affidabile e confrontare tra loro, o che non potranno essere usate quali input per processi come la stima dell'impegno di un progetto.

Le quattro sezioni di questo capitolo forniscono le definizioni formali, i principi e le regole con alcuni esempi per ciascuno dei parametri fondamentali, al fine di aiutare il misuratore attraverso il processo di determinazione della Strategia di Misurazione, come illustrato in Fig. 2.0 seguente.

Ciascuna sezione fornisce spiegazioni sull'importanza del parametro esaminato, utilizzando analogie per illustrare perché tale parametro è dato per scontato in altri campi della misurazione e debba quindi essere considerato anche nel campo della misurazione della dimensione funzionale del software.

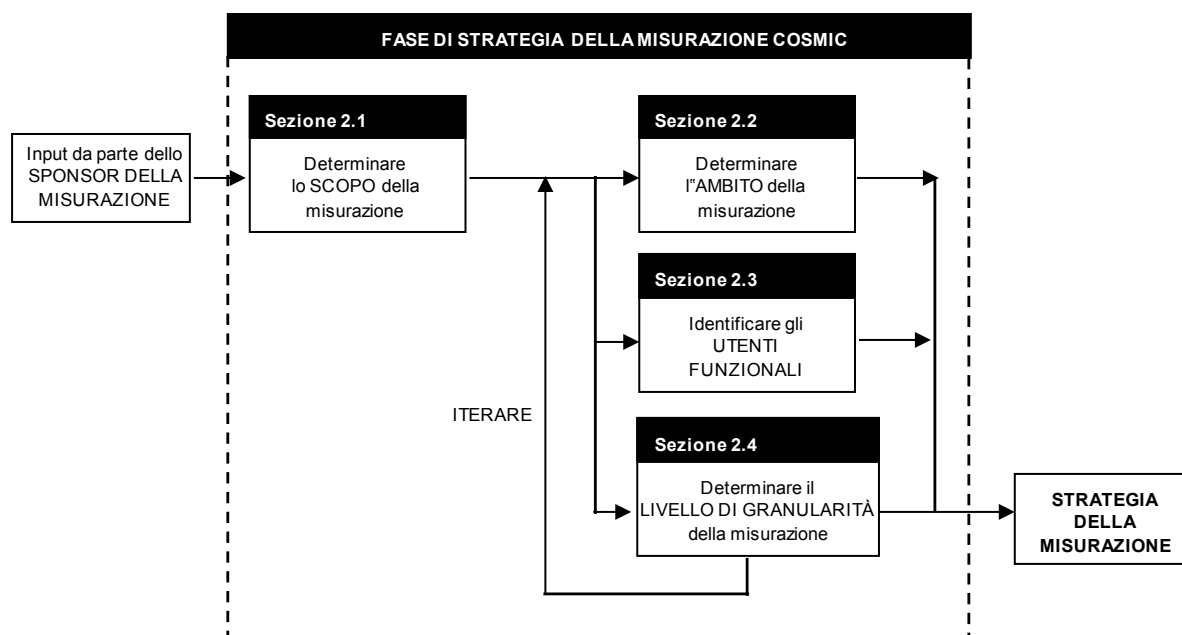


Figura 2.0 – Processo di determinazione della Strategia di Misurazione.

2.1 Definire lo scopo della misurazione

Il termine “scopo” è usato nella sua normale accezione.

DEFINIZIONE – Scopo della misurazione

Un'affermazione che definisce il motivo per cui si intraprende una misurazione e il fine per cui sarà usato il risultato della stessa.

2.1.1 *Lo scopo della misurazione determina la particolare dimensione che sarà misurata*

Esistono molte ragioni per misurare la dimensione funzionale del software, così come esistono molte ragioni per misurare per es. l'area della superficie di una casa. Quando lo scopo è stimare il costo dello sviluppo di un nuovo software, potrebbe essere necessario misurare la dimensione funzionale del software prima dello sviluppo, così come potrebbe essere necessario misurare le aree di superficie di una casa prima della costruzione. In un diverso contesto, per es. quando occorre comparare i costi effettivi con quelli stimati, potrebbe essere necessario misurare la dimensione funzionale del software dopo che sia stato messo in produzione, così come potrebbe essere utile misurare le aree di superficie di una casa dopo che questa sia stata costruita per verificare che le dimensioni effettive siano conformi alle planimetrie concordate. Il motivo per cui si effettua una misurazione ha un impatto, sebbene a volte sottile, su *quale* sia l'oggetto della misurazione, *senza influire* sull'unità di misura o sui principi di misurazione.

Nell'esempio della casa, la misurazione delle aree della sua superficie prima della costruzione si basa, ovviamente, sulle planimetrie dell'edificio. Le dimensioni richieste (lunghezza e larghezza) sono estratte dalle planimetrie tramite convenzioni di scala adeguate e le aree di superficie si calcolano secondo convenzioni ben consolidate.

Analogamente, la misurazione della dimensione funzionale del software prima dello sviluppo si basa sui requisiti utente funzionali del software, che sono derivati dalle sue “planimetrie”, ovverosia i manufatti del software prodotti prima dello sviluppo. I requisiti utente funzionali sono derivati da tali manufatti tramite convenzioni adeguate e le dimensioni richieste (il numero di movimenti di dati) sono identificate, così che si possa calcolare la dimensione.

Per proseguire ulteriormente con l'analogia della casa, la misurazione delle aree della sua superficie dopo che sia stata costruita comporta una procedura di misurazione alquanto differente. Ora, le dimensioni richieste (lunghezza e larghezza) sono estratte dall'edificio stesso tramite un mezzo differente (il metro a nastro). Tuttavia, sebbene l'oggetto fisico misurato sia differente (la casa piuttosto che le sue planimetrie), le dimensioni, l'unità di misura (inclusa qualsiasi convenzione di scala) e i principi di misurazione rimarranno gli stessi.

Analogamente, la misurazione della dimensione funzionale del software dopo che sia stato messo in produzione comporta una procedura di misurazione alquanto differente, laddove le dimensioni richieste siano estratte dai vari manufatti del software stesso. Sebbene la natura di tali manufatti sia differente, le dimensioni, l'unità di misura e i principi di misurazione rimangono gli stessi.

È compito del misuratore, in base allo scopo della misurazione, determinare se l'oggetto da misurare è la casa descritta a voce dal proprietario, la casa descritta tramite le planimetrie o la casa così come è stata costruita, e selezionare i manufatti più adeguati per la misurazione. Chiaramente, le tre dimensioni potrebbero essere differenti. Il medesimo ragionamento si applica alla misurazione del software.

2.1.2 *Esempi di scopi di misurazione*

I seguenti sono esempi di tipici scopi di misurazione.

- Misurare la dimensione dei FUR man mano che evolvono, come variabile in input per un processo di stima dell'impegno di sviluppo.
- Misurare la dimensione delle modifiche apportate ai FUR dopo che sono state inizialmente concordate, per gestire lo "slittamento dell'ambito" ("scope creep") del progetto.
- Misurare la dimensione dei FUR del software consegnato, come variabile in input per la misurazione delle performance dello sviluppatore.
- Misurare la dimensione dei FUR di tutto il software consegnato ed anche la dimensione dei FUR del software che è stato sviluppato, per ottenere una misura del riuso funzionale.
- Misurare la dimensione dei FUR del software esistente, come variabile in input per la misurazione delle performance dei responsabili della manutenzione e del supporto del software.
- Misurare la dimensione di alcune modifiche a (i FUR di) un sistema software esistente, come misura della dimensione del lavoro prodotto da un gruppo di progetto di manutenzione evolutiva.
- Misurare la dimensione della funzionalità di un software esistente fornito ad utenti funzionali umani.

Lo scopo aiuta il misuratore a determinare:

- l'ambito da misurare e quindi i manufatti che saranno necessari per la misurazione;
- gli utenti funzionali (come sarà illustrato in sez. 2.3, la dimensione funzionale varia in funzione di chi o cosa è definito come utente funzionale);
- il momento del ciclo di vita del progetto in cui avrà luogo la misurazione;
- l'accuratezza richiesta per la misurazione e quindi se si debba applicare il metodo di misurazione COSMIC o una sua versione di approssimazione derivata localmente (per es. nelle fasi iniziali del ciclo di vita di un progetto, prima che i FUR siano stati completamente elaborati). Entrambi gli ultimi due punti determineranno il livello di granularità al quale i FUR saranno misurati.

2.2 Definire l'ambito della misurazione

DEFINIZIONE – Ambito della misurazione

L'insieme di Requisiti Utente Funzionali che devono essere inclusi in una specifica esecuzione di misurazione della dimensione funzionale.

NOTA. Si dovrebbe fare una distinzione tra l'"ambito complessivo", per es. tutto il software che dovrebbe essere misurato in base allo scopo, e l'"ambito" di ogni singola porzione di software, nel contesto nell'ambito complessivo, la cui dimensione dovrebbe essere misurata separatamente. Nel presente Manuale di Misurazione, il termine "ambito" (o l'espressione "ambito della misurazione") si riferirà a una singola porzione di software la cui dimensione deve essere misurata separatamente.

I Requisiti Utente Funzionali sono definiti dall'ISO come segue:

DEFINIZIONE – Requisiti Utente Funzionali (Functional User Requirements, FUR)

Un sotto-insieme dei Requisiti Utente. Requisiti che descrivono cosa il software deve eseguire, in termini di attività e servizi.

NOTA. I Requisiti Utente Funzionale includono, ma non sono limitati a:

- trasferimento dati (per es. Inserimento dati cliente; Invio segnale di controllo);
- trasformazione dati (per es. Calcolo interessi bancari; Calcolo temperatura media);

- memorizzazione dati (per es. Archivia ordine cliente; Registra temperatura ambiente nel tempo);
- recupero dati (per es. Lista impiegati attuali; Recupero ultima posizione velivolo).

Esempi di Requisiti Utente che non rappresentano Requisiti Utente Funzionali includono, ma non sono limitati a:

- vincoli di qualità (per es. usabilità, affidabilità, efficienza e portabilità);
- vincoli organizzativi (per es. località operative, hardware di destinazione e conformità agli standard);
- vincoli ambientali (per es. interoperabilità, sicurezza, privacy e incolumità);
- vincoli implementativi (per es. linguaggio di sviluppo, data prevista di consegna).

REGOLE – Ambito

- a) L'ambito di una Misurazione della Dimensione Funzionale (Functional Size Measurement, FSM) deve derivare dallo scopo della misurazione.
- b) L'ambito di una qualsiasi misurazione non deve estendersi oltre uno strato del software oggetto di misurazione.

2.2.1 L'ambito della misurazione dipende dallo scopo

È importante definire l'ambito di una misurazione prima di avviare una particolare esecuzione di misurazione.

Proseguendo l'analogia della costruzione di una casa, può essere necessario, se lo scopo è stimare i costi, misurare la dimensione di diverse parti della casa separatamente, per es. le fondamenta, i muri e il tetto, poiché usano metodi di costruzione differenti. Lo stesso vale per la stima dei costi dello sviluppo del software.

Qualora un sistema software da sviluppare consista di porzioni che risiederanno in diversi strati dell'architettura del sistema, la dimensione del software in ogni strato dovrà in tal caso essere misurata separatamente, ossia ogni porzione avrà un separato ambito definito ai fini della misurazione della dimensione. Ciò discende dal principio d) del Modello di Contesto del Software (per maggiori dettagli sugli strati, v. sez. 2.2.3 nel seguito).

Allo stesso modo, se il software deve essere sviluppato come un gruppo di componenti alla pari in un singolo strato, usando per ognuna una tecnologia differente, sarà allora necessario definire un ambito di misurazione separato per ogni componente prima di misurarne le dimensioni. Ciò sarebbe necessario per es. se le differenti tecnologie da usare fossero associate a differenti valori di produttività di sviluppo (per maggiore dettaglio sulle componenti alla pari, v. sez. 2.2.4 nel seguito).

Lo scopo aiuterà a decidere anche quale software dovrebbe essere incluso o escluso dall'ambito della misurazione. Per es. se lo scopo è misurare la dimensione funzionale di una porzione di software consegnata da un particolare gruppo di progetto, sarà necessario in primo luogo definire i requisiti utente funzionali di tutte le varie componenti che devono essere consegnate dal gruppo. Questi potrebbero includere i FUR di una porzione di software utilizzata una sola volta per convertire i dati provenienti dal software che viene sostituito. Se tuttavia lo scopo venisse cambiato nel misurare la dimensione del nuovo software che risulta disponibile per l'uso operativo, questa risulterebbe essere più piccola, poiché i FUR del software usato per la conversione non sarebbero inclusi nell'ambito della dimensione misurata.

Riepilogando, lo scopo della misurazione deve essere sempre usato per determinare: a) quale software è incluso o escluso dall'ambito complessivo, e b) in che modo il software incluso può richiedere di essere diviso in porzioni separate, ciascuna delle quali con il proprio ambito, da misurare separatamente.

2.2.2 Esempi di ambito di misurazione e livelli di scomposizione

I seguenti sono tipi generici di ambito.

- Un patrimonio d'impresa.
- Un'affermazione contrattualmente concordata dei requisiti.
- Il risultato lavorativo *consegnato* da un Gruppo di Progetto (cioè incluse le parti ottenute sfruttando parametri software esistenti, pacchetti acquistati e codice riusabile, qualsiasi software usato per la conversione dei dati e successivamente scartato, servizi e software per il test sviluppati specificatamente per tale progetto).
- Il risultato lavorativo *sviluppato* da un Gruppo di Progetto (cioè incluso qualsiasi software usato per la conversione dei dati, ma successivamente scartato, servizi e software per il test sviluppati specificatamente per tale progetto, ma *escluse* tutte le nuove funzionalità ottenute modificando parametri e sfruttando codice riusabile o per mezzo di pacchetti acquistati).
- Tutto il software in uno strato.
- Un pacchetto (package) software.
- Un'applicazione.
- Una componente principale ("alla pari") di un'applicazione.
- Una classe di oggetti riusabile.
- Tutte le modifiche richieste per un nuovo rilascio di una porzione di un software esistente.

In pratica, un'affermazione di ambito deve essere esplicita piuttosto che generica, per es. "il risultato lavorativo sviluppato dal Gruppo di Progetto „A“", "l'Applicazione „B“" o "il Patrimonio dell'Impresa „C“". L'affermazione dell'ambito può anche dover specificare, per chiarezza, cosa è escluso.

Si noti che alcuni dei "tipi generici di ambito" sopra elencati corrispondono a diversi "livelli di scomposizione" del software, definiti come segue:

DEFINIZIONE – Livello di scomposizione
Ogni livello di suddivisione di una porzione di software che mostri le sue componenti, sotto-componenti, ecc.
NOTA. Da non confondere con "livello di granularità".

Un esempio di diversi livelli di scomposizione, corrispondenti a diversi "tipi di ambito generici", sarebbe quello di un "portafoglio applicativo" che consiste di "applicazioni" multiple, ciascuna delle quali può consistere di "componenti principali (alla pari)", ciascuna delle quali può consistere di "classi di oggetti riusabili".

Determinare l'ambito della misurazione può perciò rappresentare ben più della questione di stabilire semplicemente quali funzionalità dovrebbero essere incluse nella misurazione. La decisione può anche implicare considerazioni sul livello di scomposizione del software al quale la/le misurazioni si svolgeranno. Questa è una decisione importante nella Strategia della Misurazione, che dipende dallo scopo della misurazione, poiché misurazioni effettuate a diversi livelli di scomposizione potrebbero non essere facilmente confrontate tra loro. Ciò deriva dal fatto che, come si vedrà in sez. 4.3.1, la dimensione di qualsiasi porzione di software non può ottenersi semplicemente sommando le dimensioni delle sue componenti.

2.2.3 Strati

Poiché l'ambito di una porzione di software oggetto di misurazione deve essere limitato a un singolo strato software, il processo di definizione dell'ambito può richiedere che il misuratore stabilisca in primo luogo quali sono gli strati dell'architettura software. In questa sezione definiamo e discutiamo quindi gli "strati" e le "componenti alla pari", in quanto tali termini sono utilizzati nel metodo COSMIC.

I motivi per cui occorrono tali definizioni e regole sono i seguenti:

- Il misuratore può dover affrontare la misurazione di parti di software in un ambiente "legacy", evolutosi nel corso di molti anni senza esser mai stato progettato secondo un'architettura di fondo (ovvero, con un'architettura disorganizzata – in gergo, con una "spaghetti architecture"). Il misuratore potrebbe perciò aver bisogno di indicazioni su come distinguere gli strati nel senso della terminologia COSMIC.
- Le espressioni "strato", "architettura a strati" e "componente alla pari" non sono usate in modo omogeneo nell'industria software. Se il misuratore deve misurare parti di software descritte come facenti parte di un' "architettura a strati", è consigliabile verificare che gli "strati" di tale architettura siano definiti in modo compatibile con il metodo COSMIC. Per far ciò, il misuratore dovrebbe stabilire l'equivalenza tra oggetti architetturali specifici del paradigma dell' "architettura a strati" e il concetto di strati come definito in questo manuale.

Gli strati possono essere identificati attenendosi alla definizione e ai principi seguenti.

DEFINIZIONE – Strato
Uno strato è una partizione risultante dalla divisione funzionale di un'architettura software che insieme con l'hardware forma complessivamente un sistema informatico dove: • gli strati sono organizzati in una gerarchia, • esiste un solo strato in ogni livello della gerarchia, • esiste una dipendenza gerarchica di tipo "superiore/subordinato" tra i servizi funzionali forniti dal software in ciascuno dei due strati dell'architettura software che scambiano dati direttamente tra loro, • i software in due strati qualsiasi dell'architettura software che scambiano dati tra loro interpretano solo parte di tali dati in modo identico.

L'identificazione degli strati è un processo iterativo. Gli strati esatti saranno perfezionati man mano che procede il processo di mappatura. Una volta identificato, ogni candidato strato deve soddisfare i seguenti principi e regole:

PRINCIPI – Strato
a) Il software in uno strato scambia dati con il software in un altro strato attraverso i rispettivi processi funzionali. b) La "dipendenza gerarchica" tra strati è tale che il software in uno strato può usare i servizi funzionali del software in qualsiasi altro strato sottostante nella gerarchia. Laddove esistano tali relazioni, si denota lo strato software che usa l'altro strato come "superiore" e ogni strato che contiene software usato come suo "subordinato". Il software nello strato superiore poggia sui servizi del software contenuti in tali strati subordinati per funzionare correttamente; questi ultimi poggiano a loro volta sul software negli strati a loro subordinati per funzionare correttamente, e così via nella gerarchia. Al contrario, il software in uno strato subordinato, insieme con il software in qualsiasi strato subordinato da cui dipende, può funzionare senza necessità dei servizi del software in un qualsiasi strato superiore nella gerarchia. c) Il software in uno strato non usa necessariamente tutti i servizi funzionali forniti dal software in uno strato subordinato. d) I dati scambiati tra i software in due strati qualsiasi sono definiti ed interpretati in maniera differente dai rispettivi FUR delle due porzioni di software, ossia le due

porzioni di software riconoscono attributi dei dati differenti e/o sotto-gruppi differenti dei dati che scambiano. Tuttavia, devono anche esistere uno o più attributi o sotto-gruppi di dati definiti in comune per consentire al software nello strato ricevente di interpretare i dati che gli vengono passati dal software nello strato mittente, in funzione delle esigenze del software ricevente.

REGOLE – Strato

- Se il software è concepito usando un'architettura a strati stabilita, come si intende in questa sede, allora si dovrebbe usare tale architettura per identificare gli strati per scopi di misurazione.
- Nel dominio del software gestionale (MIS) o aziendale, lo strato "più alto", cioè lo strato che non è subordinato a nessun altro strato, è normalmente definito come lo strato "dell'applicazione". Il software (applicativo) contenuto in tale strato poggia fondamentalmente sui servizi dei software in tutti gli altri strati sottostanti per funzionare correttamente. Nel dominio del software real-time, il software nello "strato più alto" è normalmente definito come il "sistema", come per es. in "software di sistema di controllo del processo", "software di sistema di controllo di volo".
- Non si assuma che un software qualsiasi, evolutosi senza alcuna considerazione di progettazione architetturale o alcuna strutturazione, possa essere partizionato in strati secondo il modello COSMIC.

(NdT. Regola c) originale eliminata, refuso in manuale 3.0. La regola d) diviene regola c.)

Pacchetti software di servizi funzionali, quali sistemi di gestione di database (Database Management System), sistemi operativi o driver di periferiche ("device driver"), si dovrebbero normalmente considerare come allocati in strati distinti.

Una volta identificato, ogni strato può essere registrato quale "componente" separata nella matrice del Modello Generale del Software (App. A), con la corrispondente etichetta.

Esempi della struttura fisica di una tipica architettura software a strati (usando il termine "strato" come qui definito) che supporta software applicativo aziendale e una porzione di un software integrato ("embedded") real-time sono presentati rispettivamente in Fig. 2.2.3.1 e 2.2.3.2.

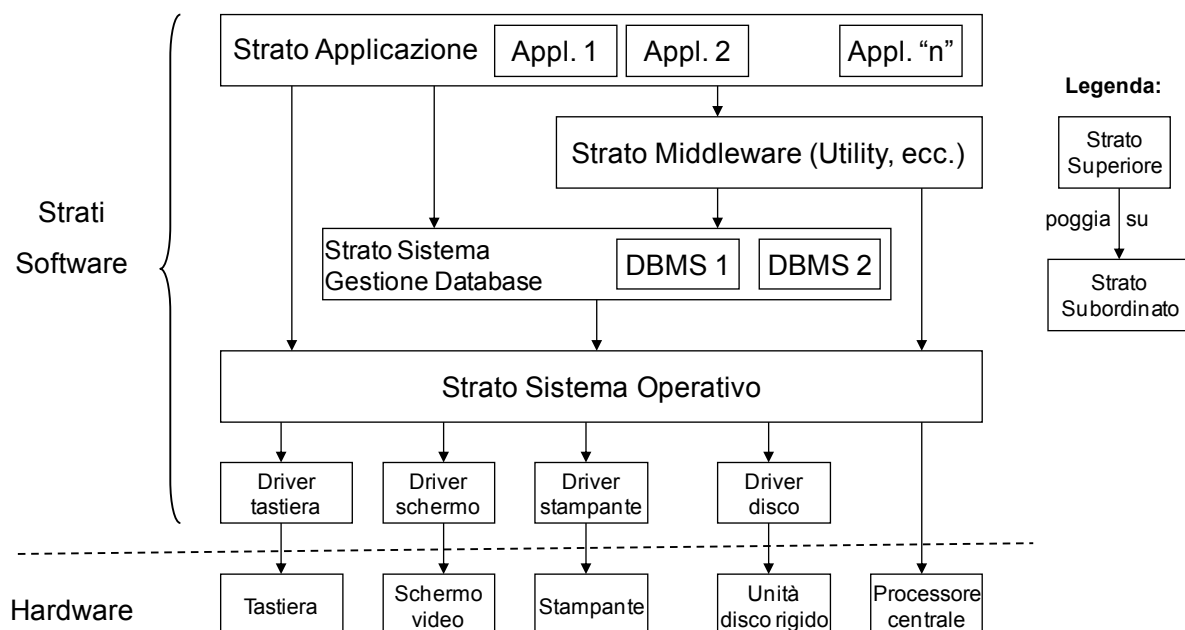


Figura 2.2.3.1 – Tipica architettura software a strati per un sistema informatico aziendale/MIS.

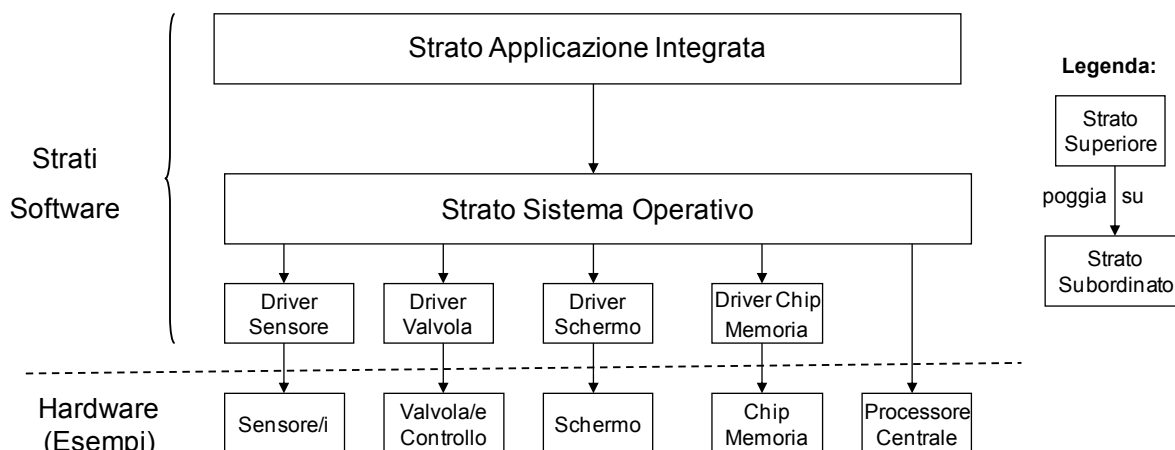


Figura 2.2.3.2 – Tipica architettura a strati per un sistema informatico con software integrato real-time.

2.2.4 Componenti alla pari

Le componenti alla pari possono essere identificate attenendosi alla definizione e ai principi seguenti.

DEFINIZIONE – Componente alla pari

Una componente alla pari è una partizione risultante dalla divisione funzionale dei FUR di una porzione di software contenuta in uno strato in un insieme mutualmente cooperante tale che ciascuna partizione esegue una specifica porzione dei FUR di tale porzione di software.

NOTA. L'espressione "porzione di software alla pari" è usata laddove si intenda ogni porzione di software indipendente (e alla pari).

Una volta identificata, ogni candidata componente alla pari deve soddisfare i seguenti principi:

PRINCIPI – Componente alla pari

- In un insieme di componenti alla pari di una porzione di software in uno strato non esiste dipendenza gerarchica tra le componenti alla pari come avviene tra strati. I FUR di tutte le componenti alla pari di una porzione di software in un qualsiasi strato sono allo stesso "livello" nella gerarchia degli strati.
- Tutte le componenti alla pari di una porzione di software devono cooperare mutualmente affinché la porzione di software possa funzionare correttamente.
- I FUR di componenti alla pari in un qualsiasi strato che si passano o condividono dati in comune, definiscono tali dati in modo identico, cioè riconoscono gli stessi attributi dei dati e sotto-raggruppamenti dei dati che si passano o condividono.

Una volta identificata, ogni componente alla pari può essere registrata quale componente distinta nella matrice del Modello Generale del Software (App. A), con la corrispondente etichetta.

Un esempio di un insieme di componenti alla pari è dato da un applicativo aziendale sviluppato con tre componenti principali, e precisamente una componente di "interfaccia utente" (o "front-end"), una componente di "regole di trattamento" e una componente di "servizi sui dati". Un esempio di due "porzioni di software alla pari" sarebbe quello di due applicazioni sviluppate separatamente, collocate nello stesso strato e che scambiano dati tra loro, o che condividono dati in comune.

2.3 Identificare gli utenti funzionali

2.3.1 La dimensione funzionale varia con l'utente funzionale

Iniziando con un'analogia, la misurazione delle aree dei pavimenti di un ufficio può essere svolta secondo quattro diverse convenzioni, come segue (N.B. L'ambito, il particolare ufficio, è il medesimo per tutte e quattro le convenzioni.)

- Il proprietario dell'edificio deve pagare le tasse sull'ufficio. Per il proprietario dell'edificio, l'area della superficie è data dalla "metratura quadra lorda", determinata in base alle dimensioni esterne ed includendo perciò tutti i corridoi pubblici, lo spazio occupato dalle pareti, ecc.
- Il responsabile del riscaldamento dell'edificio è interessato alla "metratura quadra netta", cioè alle aree interne, incluse le aree pubbliche e lo spazio occupato dagli ascensori, ma esclusi gli spessori dei muri.
- Il fornitore dei servizi di pulizia, assunto dal titolare dell'ufficio, è interessato alla "metratura quadra netta / netta più accessori", che esclude le aree pubbliche, ma include i passaggi usati dal titolare.
- Il responsabile dell'organizzazione dell'ufficio è interessato alla "metratura quadra netta / netta", cioè solo lo spazio utilizzabile per gli uffici.

La lezione che si trae da questa analogia è che la dimensione di un oggetto può variare in base alle funzionalità che sono di interesse per diversi tipi di utente dell'oggetto stesso. Nel caso del software, (tipi di) utente funzionale differenti possono richiedere e utilizzare funzionalità differenti e pertanto le dimensioni funzionali varieranno con la scelta degli utenti funzionali.

2.3.2 Utenti funzionali

Un "utente" è definito effettivamente⁵ come "ogni cosa che interagisce con il software oggetto di misurazione". Tale definizione è eccessivamente ampia per le necessità del metodo COSMIC. Per il metodo COSMIC, la scelta dell'utente (o degli utenti) è determinata dai requisiti "utente" funzionali che si devono misurare. Questo (tipo di) utente, noto come l'"utente funzionale", è definito come segue.

DEFINIZIONE – Utente funzionale
Un (tipo di) utente che è un mittente e/o un destinatario designato dei dati nei FUR di una porzione di software.

Come esempio del perché sia importante distinguere gli utenti funzionali da tutti i possibili utenti, consideriamo una porzione di un software applicativo che debba essere misurato. I suoi utenti funzionali, se è un applicativo aziendale, includerebbero normalmente esseri umani ed altre applicazioni alla pari con le quali l'applicazione si interfaccia. Se è un'applicazione real-time, gli utenti funzionali includerebbero normalmente periferiche hardware ingegnerizzate o altri software alla pari interfacciati. I requisiti utente funzionali (FUR) di tale software sono normalmente espressi in modo tale che gli utenti funzionali siano rispettivamente i mittenti dei dati e/o i destinatari designati dei dati da e verso il software.

Tuttavia, l'insieme totale degli "utenti", includente cioè "ogni oggetto che interagisce con il software", deve includere il sistema operativo. Ma i FUR di un'applicazione qualsiasi non includerebbero mai il sistema operativo come un utente. Qualsiasi vincolo che il sistema operativo può imporre a un'applicazione sarà comune a tutte le applicazioni, sarà normalmente gestito dal compilatore o dall'interprete del linguaggio di programmazione, ed è invisibile ai veri utenti funzionali dell'applicazione. Nella misurazione pratica della dimensione funzionale, un sistema operativo non sarebbe mai considerato come un utente funzionale di un'applicazione.⁶

⁵ V. il glossario per la definizione, tratta dallo standard ISO/IEC 14143/1:2007.

⁶ In realtà, naturalmente, è vero il contrario. Le applicazioni sono utenti funzionali di un sistema operativo. I FUR di un sistema operativo sono definiti considerando le applicazioni che esso deve supportare quali suoi utenti funzionali.

Ma non sempre si ha che gli utenti funzionali siano ovvi. Si prenda l'esempio del software applicativo di un telefono cellulare (già citato nell'Introduzione). Sebbene si sia eliminato il sistema operativo del telefono cellulare come possibile utente funzionale dell'applicazione, gli "utenti" potrebbero ancora essere o a) esseri umani che premono i tasti, o b) periferiche hardware (per es. lo schermo, i tasti, ecc.) e applicazioni alla pari che interagiscono direttamente con l'applicazione del telefono. L'utente umano per es. vedrà solo un sotto-insieme di tutte le funzionalità che devono essere fornite per permettere all'applicazione di funzionare. Perciò questi due tipi di utenti "vedranno" funzionalità differenti; la dimensione dei FUR per l'utente umano sarà più piccola di quella dei FUR che devono essere sviluppati per permettere all'applicazione del telefono nel suo complesso di funzionare.⁷

REGOLE – Utenti funzionali

- a) Gli utenti funzionali di una porzione di software oggetto di misurazione derivano dallo scopo della misurazione.
- b) Quando lo scopo della misurazione di una porzione di software è collegato all'impegno per sviluppare o modificare la porzione di software, allora gli utenti funzionali dovrebbero essere coloro ai quali devono essere fornite le funzionalità nuove o modificate.

Avendo identificato gli utenti funzionali, è poi conseguente identificare il confine, poiché esso si colloca semplicemente tra la porzione di software oggetto di misurazione e i suoi utenti funzionali. Si trascura qualsiasi altro hardware o software in tale spazio interposto.⁸

DEFINIZIONE – Confine

Il confine è definito come un'interfaccia concettuale tra il software oggetto di misurazione e i suoi utenti funzionali.

NOTA. Il confine di una porzione di software è la frontiera concettuale tra tale porzione e l'ambiente in cui essa opera, così come è percepita esternamente nell'ottica dei suoi utenti funzionali. Il confine permette al misuratore di distinguere, senza ambiguità, cosa è incluso nel software misurato da cosa è parte dell'ambiente operativo del software misurato.

N.B. Tale definizione è tratta dallo standard ISO/IEC 14143/1:2007, con l'aggiunta dell'aggettivo "funzionale" per qualificare l'"utente". Per evitare ambiguità, si noti che il confine non dovrebbe essere confuso con qualsiasi linea che potrebbe essere tracciata intorno a qualche software oggetto di misurazione per definire l'ambito della misurazione. Il confine non è usato per definire l'ambito di una misurazione.

Le seguenti regole potrebbero risultare utili per convalidare lo stato di un candidato confine:

REGOLE – Confine

- a) Identifica l'utente/i funzionali che interagiscono con il software misurato. Il confine si colloca tra gli utenti funzionali e tale software.
- b) Per definizione, esiste un confine tra ogni coppia identificata di strati, dove il software di uno strato è l'utente funzionale del software dell'altro, e quest'ultimo è l'oggetto della misurazione. Analogamente, esiste un confine tra due qualsiasi componenti alla pari del medesimo strato; in tal caso, ogni componente può essere un utente funzionale della componente sua pari.

⁷ Toivonen per es. ha comparato le funzionalità dei telefoni cellulari, accessibili agli utenti umani, nell'articolo "*Defining measures for memory efficiency of the software in mobile terminals*", International Workshop on Software Measurement, Magdeburgo, Germania, Ottobre 2002.

⁸ In effetti, se il misuratore avesse dovuto esaminare i FUR al fine di identificare i mittenti e i destinatari designati dei dati, il confine sarebbe stato già identificato.

Il confine permette di fare una chiara distinzione tra qualsiasi cosa che sia parte della porzione di software oggetto di misurazione (*cioè che è dal lato del confine del software*) e qualsiasi cosa che sia parte dell'ambiente degli utenti funzionali (*cioè che è dal lato del confine degli utenti funzionali*). La memoria permanente non è considerata come un utente del software e si colloca perciò dal lato del confine del software.

2.4 Identificare il livello di granularità

2.4.1 La necessità di un livello di granularità standard

Quando si avvia un progetto per progettare e costruire una nuova casa, le prime planimetrie tracciate da un architetto sono a un "livello alto", cioè mostrano le linee principali e uno scarso dettaglio. Man mano che il progetto avanza verso la fase di costruzione, sono necessarie delle planimetrie più dettagliate ("livello basso").

Lo stesso vale per il software. Nelle fasi iniziali di un progetto di sviluppo software, i Requisiti Utente Funzionali (FUR) sono definiti "a un livello alto", cioè a grandi linee o con scarso dettaglio. Man mano che il progetto avanza, i FUR sono perfezionati (per es. tramite versioni successive 1, 2, 3, ...), rivelando un maggior dettaglio "a un livello più basso". Tali diversi gradi di dettaglio dei FUR sono noti come "livelli di granularità" differenti. (V. anche sez. 2.4.3 per altri termini che possono essere confusi con il concetto qui definito di "livello di granularità").

DEFINIZIONE – Livello di granularità

Ogni livello di espansione della descrizione di una singola porzione di software (per es. un'affermazione relativa ai suoi requisiti, o una descrizione della struttura della porzione di software) tale che a ogni livello ulteriore di espansione, la descrizione della funzionalità della porzione di software è espressa a un livello di dettaglio maggiore e uniforme.

NOTA. I misuratori dovrebbero considerare il fatto che dovrebbero essere consapevoli che, quando i requisiti stanno evolvendo nelle fasi alte della vita di un progetto software, in qualsiasi momento porzioni differenti delle funzionalità software richieste saranno state tipicamente documentate a livelli di granularità differenti.

Le planimetrie per la costruzione di una casa sono tracciate secondo scale standard, ed è facile convertire le dimensioni misurate su un disegno a quelle di un altro disegno con una scala differente. Di contro, non esistono scale standard per i vari livelli di granularità a cui il software può essere espresso, così che può risultare difficile essere sicuri che due affermazioni nei FUR siano allo stesso livello di granularità. Senza un'intesa su alcuni livelli di granularità standard a cui misurare (o a cui le misurazioni dovrebbero essere riscalate) è impossibile sapere con certezza se due misurazioni di dimensione funzionale possono essere confrontate tra loro. E i misuratori possono dover sviluppare il proprio metodo per riscalare le misurazioni da un livello di granularità all'altro.

Per illustrare ulteriormente tali problematiche, consideriamo un'altra analogia. Un insieme di mappe stradali rivela i dettagli di una rete stradale nazionale a tre livelli di granularità.

- La mappa A riporta solo autostrade e strade principali.
- La mappa B riporta tutte le autostrade, strade principali e strade secondarie (come in un atlante per automobilisti).
- La mappa C riporta tutte le strade con i loro nomi (come in un insieme di mappe di zona locali).

Se non riconosciamo il fenomeno dei diversi livelli di granularità, sembrerebbe che queste tre mappe rivelino dimensioni differenti della rete stradale nazionale. Naturalmente, con le mappe stradali, chiunque comprende i diversi livelli di dettaglio riportati ed esistono scale standard per interpretare la dimensione della rete stradale rivelata a qualsiasi livello. Dietro le scale di queste diverse mappe si trova il concetto astratto di "livello di granularità".

Nel caso della misurazione del software, esiste un solo livello standard di granularità che è possibile definire in maniera non ambigua. Si tratta del livello di granularità al quale i singoli processi funzionali sono stati identificati e i loro movimenti di dati definiti. Le misurazioni dovrebbero essere effettuate a questo livello o riscalate a questo livello ogniquale sia possibile.⁹

2.4.2 Chiarimento sul “livello di granularità”

Prima di procedere oltre, è importante assicurarsi che non ci siano incomprensioni riguardo il significato di “livello di granularità” nel metodo COSMIC. Nel senso qui definito, estendere la *descrizione* di un software da un livello di granularità “più alto” ad uno “più basso” implica effettuare uno “zoom” e rivelarne un maggior dettaglio, *senza modificarne l’ambito*. Tale processo NON dovrebbe essere confuso con nulla di quanto segue.

- Effettuare uno “zoom” su un manufatto descrivente un software per mostrare diversi sotto-insiemi di funzionalità rilasciati a utenti differenti, delimitando quindi probabilmente la funzionalità oggetto di misurazione.
- Effettuare uno “zoom” su un manufatto descrivente un software, o sul software stesso, e allo stesso tempo scomporlo per mostrare la struttura delle sue componenti, sotto-componenti, ecc. (mostrando cioè diversi “livelli di scomposizione” – v. sez. 2.2.2). Effettuare uno “zoom” su livelli più bassi di scomposizione del software può portare (in base allo scopo della misurazione) a una suddivisione dell’ambito complessivo della misurazione.
- Far evolvere la descrizione di un software man mano che procede lungo il suo ciclo di sviluppo, per es. dai requisiti alla progettazione logica, alla progettazione fisica, ecc. A prescindere dalla fase nello sviluppo di un software, per scopi di misurazione ci si interessa solamente ai FUR.

Si intende perciò che il concetto di “livello di granularità” va interpretato come applicato solamente ai Requisiti Utente Funzionali del software. Altri modi di “effettuare zoom” o “suddividere” il software o le sue varie descrizioni possono anche essere molto importanti nell’utilizzare o nel confrontare tra loro le misurazioni della dimensione funzionale. Queste alternative saranno trattate nel documento “Advanced and Related Topics”, nel capitolo “Ensuring the Comparability of Size Measurements”.

2.4.3 Il livello di granularità standard

Il livello di granularità standard per la misurazione è il “livello dei processi funzionali” definito come segue.¹⁰

DEFINIZIONE – Livello di granularità dei processi funzionali
Un livello di granularità della descrizione di una porzione di software al quale gli utenti funzionali: • sono esseri umani o periferiche ingegnerizzate o porzioni di software singole (e non raggruppamenti qualsiasi di queste/i), E • rilevano occorrenze singole di eventi a cui la porzione di software deve rispondere (e non qualsiasi livello al quale siano definiti gruppi di eventi). NOTA 1. In pratica, la documentazione software che contiene i requisiti funzionali utente spesso descrive le funzionalità a diversi livelli di granularità, in particolare quando la documentazione è ancora in evoluzione. NOTA 2. “Gruppi di questi (utenti funzionali)” potrebbero essere per es. un “dipartimento” i cui membri gestiscono molti tipi di processi funzionali, o un “pannello di controllo” che ha diversi tipi di strumenti, o “sistemi centrali”.

⁹ Il tema della scalabilità delle misure da un livello di granularità all’altro era stato già introdotto nella versione 2.2 del Manuale di Misurazione nel capitolo 7 sul “Dimensionamento approssimato anticipato con il metodo COSMIC”. (Questo tema si trova ora nel documento “Advanced and Related Topics”).

¹⁰ Il motivo del nome “livello di granularità dei processi funzionali” è che questo è il livello al quale si identificano i processi funzionali e i loro movimenti di dati – v. sez. 3.2 per una discussione più dettagliata dei processi funzionali.

NOTA 3. “Gruppi di eventi” potrebbero essere indicati per es. in un’affermazione di FUR a un livello di granularità alto da un flusso di input verso un sistema software di contabilità denominato “transazioni di vendita”; o da un flusso di input verso un sistema software avionico denominato “comandi di pilotaggio”.

Con tale definizione, possiamo ora definire le seguenti regole e una raccomandazione.

REGOLE – Livello di granularità dei processi funzionali

- a) La misurazione della dimensione funzionale dovrebbe essere effettuata al livello di granularità dei processi funzionali.
- b) Laddove sia richiesta la misurazione della dimensione funzionale di alcuni FUR che non sono ancora evoluti al livello al quale tutti i processi funzionali siano stati identificati e tutti i dettagli sui loro movimenti di dati definiti, si dovrebbero effettuare delle misurazioni delle funzionalità fin lì definite, e si dovrebbero poi riscalarle al livello di granularità dei processi funzionali. (V. il documento “Advanced & Related Topics” per metodi di “dimensionamento approssimato”, cioè di stima di una dimensione funzionale nelle prime fasi del processo di determinazione dei FUR.)

In aggiunta a tali regole, il COSMIC *raccomanda*¹¹ che il livello di granularità dei processi funzionali dovrebbe costituire lo standard a cui le misurazioni della dimensione funzionale vengono richieste ed utilizzate dai fornitori di servizi di benchmarking e di tool software progettati per supportare o utilizzare misurazioni della dimensione funzionale, per es. per stimare l’impegno di progetto.

2.4.4 Un esempio di funzionalità a differenti livelli di granularità

L’esempio, tratto dal dominio del software applicativo aziendale, è quello di un sistema molto popolare per ordinare merci in Internet, che chiameremo “sistema Everest” (il documento “Advanced & Related Topics” contiene un esempio tratto dal dominio del software real-time).

La descrizione che segue è fortemente semplificata al fine di illustrare i livelli di granularità. Questa descrizione inoltre comprende solo le funzionalità disponibili agli utenti clienti del sistema Everest. Sono escluse perciò le funzionalità che devono essere presenti affinché il sistema possa completare la fornitura delle merci ad un cliente, come per es. funzionalità disponibili allo staff dell’Everest, ai fornitori dei prodotti, ai pubblicitari, ai fornitori dei servizi di pagamento, ecc.

L’ambito della misurazione è perciò definito come “le parti del sistema Everest accessibili ai clienti via Internet”. Si assume che lo scopo della misurazione sia determinare la dimensione funzionale dell’applicazione disponibile agli utenti cliente umani (in qualità di “utenti funzionali”).

Al livello più alto dell’intera applicazione (livello 1) un’affermazione dei Requisiti Utente Funzionali (FUR) del Sistema Ordini Everest sarebbe una semplice affermazione sintetica come la seguente.

“Il sistema Everest deve consentire ai clienti di cercare, selezionare, ordinare, pagare ed ottenere la consegna di un qualsiasi prodotto nella gamma dei prodotti dell’Everest, inclusi i prodotti disponibili da parte di altri fornitori.”

Effettuando uno “zoom” su questa affermazione di massimo livello, si trova che al successivo livello sottostante (livello 2) il sistema consiste di quattro sotto-sistemi, come illustrato in Fig. 2.4.

¹¹ Il motivo per cui l’utilizzo del livello di granularità dei processi funzionali è “raccomandato” piuttosto che dato come regola è che tale raccomandazione si applica non solo agli utilizzatori individuali del metodo COSMIC, ma anche alla loro rete di fornitori di servizi e strumenti che fanno uso di misurazioni di dimensione. Il COSMIC può solo fare raccomandazioni a questa comunità più vasta.

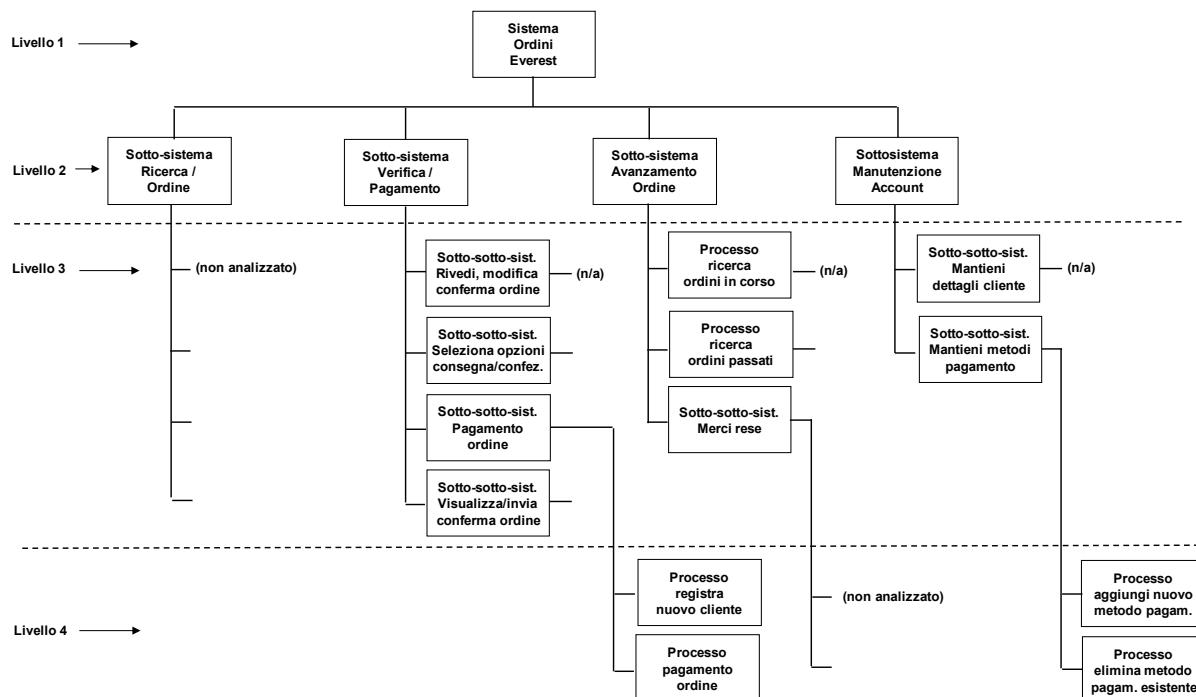


Figura 2.4 – Analisi parziale del Sistema Ordini Everest, con quattro livelli di analisi.

I quattro sotto-sistemi sono:

- Il sotto-sistema “Ricerca/Ordine”, che permette a un cliente di trovare un qualsiasi prodotto nel database del sistema Everest, così come il relativo prezzo e disponibilità, e di aggiungere qualsiasi prodotto selezionato in un “carrello” per l’acquisto. Questo sotto-sistema promuove anche le vendite suggerendo offerte speciali, proponendo le recensioni degli articoli selezionati e permettendo di effettuare delle ricerche generali, per es. sui termini di consegna, ecc. È un sotto-sistema molto complesso e non è analizzato in ulteriore dettaglio sotto il livello 2 per gli scopi di questo esempio.
- Il sotto-sistema “Verifica/Pagamento”, che permette a un cliente di impegnare l’ordine e pagare le merci poste nel “carrello”.
- Il sotto-sistema “Avanzamento Ordine”, che permette a un cliente di interrogare lo stato di avanzamento di un ordine esistente in fase di consegna, di gestire i propri ordini (per es. modificare l’indirizzo per il recapito delle merci) e di restituire eventuali merci non soddisfacenti.
- Il sotto-sistema “Manutenzione Account”, che permette a un cliente esistente di mantenere vari dettagli del suo account come l’indirizzo di casa, i mezzi di pagamento, ecc.

La Fig. 2.4 mostra anche alcuni dettagli evidenti quando si effettua un “zoom”, fino a due livelli più in basso nei sotto-sistemi Verifica/Pagamento, Avanzamento Ordine e Manutenzione Account. In questo processo di “zoom” è importante notare che:

- non si è modificato l’ambito dell’applicazione oggetto di misurazione, e
- tutti i livelli della descrizione del sistema software Everest mostrano le funzionalità disponibili ai clienti (quali utenti funzionali). Un cliente può “vedere” le funzionalità del sistema a tutti questi livelli di analisi.

La Fig. 2.4 ci rivela anche che quando effettuiamo uno “zoom” ai livelli più bassi di questa particolare analisi dei tre sotto-sistemi, troviamo distinti processi funzionali a livello 3 (per due interrogazioni del sotto-sistema Avanzamento Ordine) e a livello 4 negli altri due sotto-sistemi. Questo esempio dimostra, perciò, che quando alcune funzionalità sono analizzate *con un approccio top-down*, non si

può assumere che le funzionalità mostrate ad un particolare “livello” su un diagramma corrispondano sempre allo stesso “livello di granularità” nel senso in cui tale concetto è definito nel metodo COSMIC. (Tale definizione richiede che ad ogni singolo livello di granularità le funzionalità siano “ad un livello di dettaglio comparabile”).

Inoltre, altri analisti potrebbero anche descrivere il diagramma di flusso in modo differente, mostrando altri raggruppamenti di funzionalità ad ogni livello del diagramma. Non esiste un’unica maniera “corretta” di analizzare le funzionalità di un sistema di tale complessità.¹²

Date tali variazioni che si hanno inevitabilmente in pratica, un misuratore deve esaminare con attenzione i vari livelli di un diagramma di analisi per trovare i processi funzionali che devono essere misurati. Quando ciò non è possibile in pratica, per es. perché l’analisi non ha ancora raggiunto il livello a cui tutti i processi funzionali sono stati espressi, deve applicarsi la suddetta regola b). Per illustrare ciò, esaminiamo il caso del “sotto-sotto-sistema di manutenzione dei dettagli del cliente” (v. Fig. 2.4), nel ramo del sotto-sistema di Manutenzione Account.

Per un misuratore esperto, il termine “mantenere” suggerisce quasi invariabilmente un gruppo di eventi e quindi un gruppo di processi funzionali. Si può perciò assumere che questo sotto-sotto-sistema di “Manutenzione” debba comprendere tre processi funzionali, e precisamente una “interrogazione dei dettagli del cliente”, l’ “aggiornamento dei dettagli del cliente” e la “cancellazione del cliente”. (Il processo di “creazione dei dettagli del cliente” deve anche ovviamente esistere, ma ciò avviene in un altro ramo del sistema, quando un cliente ordina delle merci per la prima volta. Ciò è fuori dall’ambito di questo esempio semplificato).

Un misuratore esperto dovrebbe essere in grado di “ipotizzare” (indovinare e stimare, o “guestimate”) una dimensione di questo sotto-sotto-sistema in unità di CFP considerando il numero assunto di processi funzionali (tre, in questo caso) e moltiplicando tale numero per la dimensione media di un processo funzionale. Questa dimensione media sarebbe ottenuta per mezzo di calibrazione in altre parti di questo sistema o in altri sistemi confrontabili con questo. Esempi di questo processo di calibrazione sono riportati nel documento “Advanced & Related Topics” sul dimensionamento approssimato e contenente anche esempi di altri approcci al dimensionamento approssimato.

Chiaramente, tali metodi di approssimazione hanno i propri limiti. Applicando tale approccio al livello 1, l’affermazione dei FUR precedentemente riportata (*“Il sistema Everest deve consentire ai clienti di cercare, selezionare, ordinare, pagare ed ottenere la consegna di un qualsiasi prodotto nella gamma dei prodotti dell’Everest ...”*), potremmo identificare alcuni processi funzionali. Ma un’analisi di maggior dettaglio rivelerebbe che il numero reale di processi funzionali in questo sistema complesso deve essere superiore. Questo è il motivo per cui le dimensioni funzionali solitamente sembrano crescere al determinarsi di maggiori dettagli dei requisiti, anche senza alcuna variazione dell’ambito. Tali metodi di approssimazione devono perciò essere utilizzati con la massima attenzione a livelli di granularità elevati, dove è disponibile un dettaglio molto scarso.

Il documento “Advanced & Related Topics” fornisce un diverso tipo di esempio del dimensionamento a vari livelli di granularità, in questo caso per parti di software di un’architettura software di telecomunicazioni. Questo esempio è più complesso del caso “Everest”, poiché gli utenti funzionali della parte di architettura oggetto di misurazione sono tutte le altre porzioni di software della stessa architettura. Questo esempio ha perciò a che fare con differenti livelli di scomposizione del software oggetto di misurazione e dei suoi utenti funzionali.

¹² La Fig. 2.4 non può nemmeno costituire un esempio di *best practice*, ma è rappresentativa di come tipicamente tali diagrammi possano essere tracciati.

2.5 Osservazioni finali sulla fase di strategia della misurazione

Una attenta valutazione dei quattro elementi del processo della strategia di misurazione, prima di iniziare una misurazione, dovrebbe garantire che la dimensione risultante possa essere propriamente interpretata. I quattro elementi sono:

- a) stabilire lo scopo della misurazione;
- b) definire l'ambito complessivo della porzione di software oggetto di misurazione e l'ambito specifico delle porzioni da misurare separatamente, considerando gli strati e le componenti alla pari dell'architettura software;
- c) stabilire gli utenti funzionali della porzione di software oggetto di misurazione;
- d) stabilire il livello di granularità dei manufatti del software oggetto di misurazione e come, se necessario, risalire a dimensioni al livello di granularità standard dei processi funzionali.

Potrebbe essere necessaria una reiterazione tra i passi b), c) e d), laddove i requisiti siano in evoluzione e nuovi dettagli segnalino la necessità di perfezionare la definizione dell'ambito.

La stragrande maggioranza delle misurazioni di dimensione funzionale è svolta per uno scopo collegato in qualche modo all'impegno di sviluppo, per es. per misurare la performance dello sviluppatore o per stime di progetto. In tali situazioni, definire la strategia di misurazione dovrebbe essere piuttosto immediato. Lo scopo e l'ambito sono normalmente facili da definire, gli utenti funzionali sono gli utenti ai quali lo sviluppatore deve fornire le funzionalità e il livello di granularità a cui sono richieste le misurazioni è quello in cui gli utenti funzionali rilevano i singoli eventi.

Tuttavia, non tutte le misurazioni ricadono sotto questo profilo comune, motivo per cui la strategia di misurazione dovrebbe essere definita in ogni situazione.

È importante notare che tali elementi della strategia di misurazione non sono legati al metodo di misurazione della dimensione funzionale (Functional Size Measurement, FSM) COSMIC, ma dovrebbero essere comuni a tutti i metodi FSM. Altri metodi FSM usano altri termini, quali "processo elementare" o "transazione logica" in luogo di "processo funzionale", ma questi sono essenzialmente gli stessi concetti. È solo la più ampia applicabilità e flessibilità del metodo COSMIC che richiede che tali elementi siano considerati con maggiore attenzione rispetto agli altri metodi FSM.

LA FASE DI MAPPATURA

Questo capitolo presenta le regole ed il metodo del processo di mappatura. Il metodo generale per mappare il software sul Modello Generale del Software COSMIC è riepilogato in Fig. 3.0.

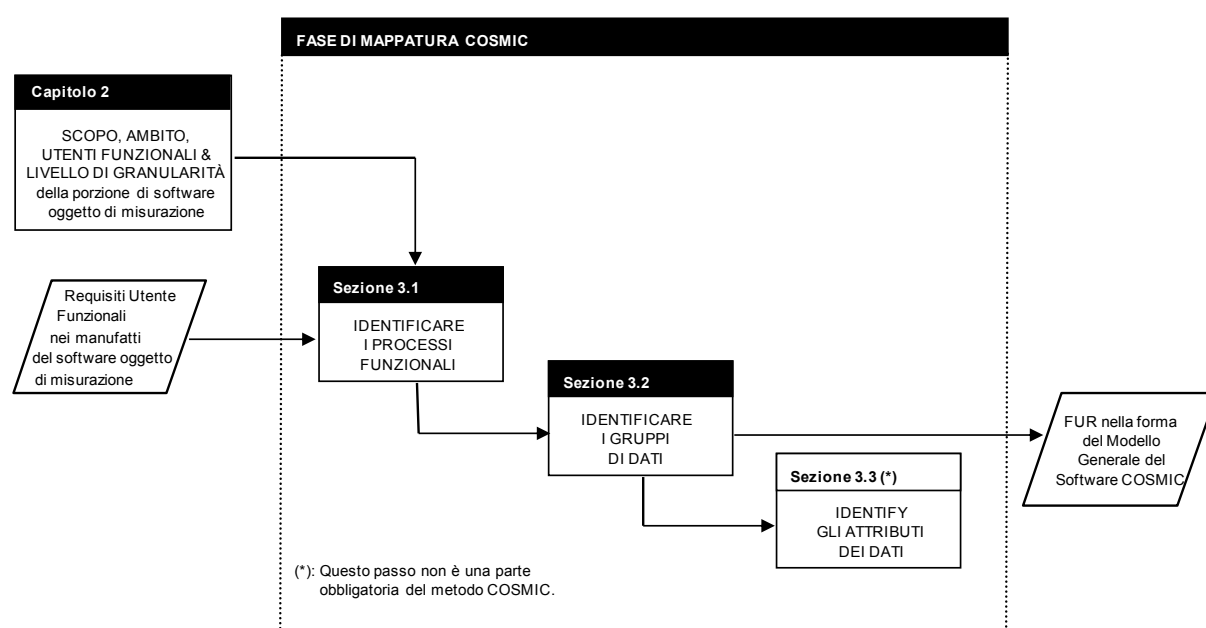


Figura 3.0 – Processo generale della Fase di Mappatura COSMIC.

Ogni passo di questo metodo è oggetto di una specifica sezione (individuata dal titolo del passo in Fig. 3.0) in cui sono presentate le definizioni e le regole, insieme con alcuni principi guida ed esempi.

Il metodo generale schematizzato in Fig. 3.0 è progettato per essere applicabile a una gamma molto ampia di manufatti del software. Una procedura più sistematica e dettagliata fornirebbe regole precise di mappatura per un più ampio insieme di manufatti altamente specifici, riducendo così il livello di ambiguità nella produzione del Modello Generale del Software COSMIC. Una simile procedura sarebbe, per definizione, altamente dipendente dalla natura dei manufatti, la quale, a sua volta, dipende dalla metodologia di ingegneria del software utilizzata in ogni organizzazione.

Il documento “Guideline for Sizing Business Application Software” fornisce indicazioni sulla mappatura da vari metodi di analisi dei dati e di gestione dei requisiti, utilizzati nel dominio degli applicativi aziendali, sui concetti del metodo COSMIC.

3.1 Applicare il Modello Generale del Software

PRINCIPIO – Applicare il Modello Generale del Software COSMIC

Il Modello Generale del Software COSMIC deve essere applicato ai requisiti utente funzionali di ciascuna porzione distinta di software per la quale è stato definito un distinto ambito di misurazione.

“Applicare il Modello Generale del Software COSMIC” significa identificare l’insieme degli eventi d’innescio percepiti da ognuno dei (tipi di) utenti funzionali identificati nei FUR e successivamente identificare i corrispondenti processi funzionali, oggetti di interesse, gruppi di dati e movimenti di dati che devono essere forniti per rispondere a tali eventi.

Le Fig. 3.1.1 e 3.1.2 sottostanti mostrano l’applicazione del principio g) del Modello del Contesto del Software e dei principi del Modello Generale del Software, rispettivamente, a una porzione di un software applicativo aziendale e a una porzione tipica di un software integrato (“embedded”) real-time.

Mentre le Fig. 2.2.3.1 e 2.2.3.2 precedenti mostravano, rispettivamente, le viste fisiche delle architetture software a strati, le Fig. 3.1.1 e 3.1.2 mostrano una vista logica delle relazioni dei vari concetti definiti nei modelli COSMIC. In questa vista logica gli utenti funzionali interagiscono con il software oggetto di misurazione attraverso un confine tramite movimenti di dati di Entry e Exit. Il software muove i dati da e verso la memoria permanente tramite movimenti di dati di Write e Read, rispettivamente. In tali viste logiche, si trascura tutto l’hardware e il software necessario per consentire che avvengano tali scambi tra il software oggetto di misurazione, i suoi utenti funzionali e la memoria permanente.

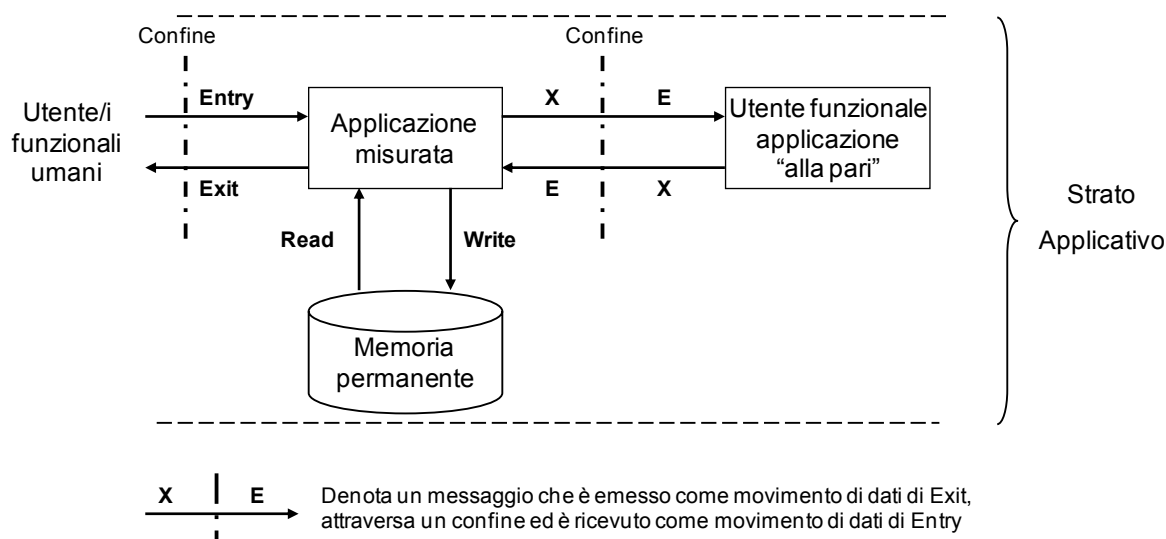


Figura 3.1.1 – Un applicativo aziendale con sia utenti funzionali umani che un'altra applicazione “alla pari” come propri utenti funzionali (vista logica).

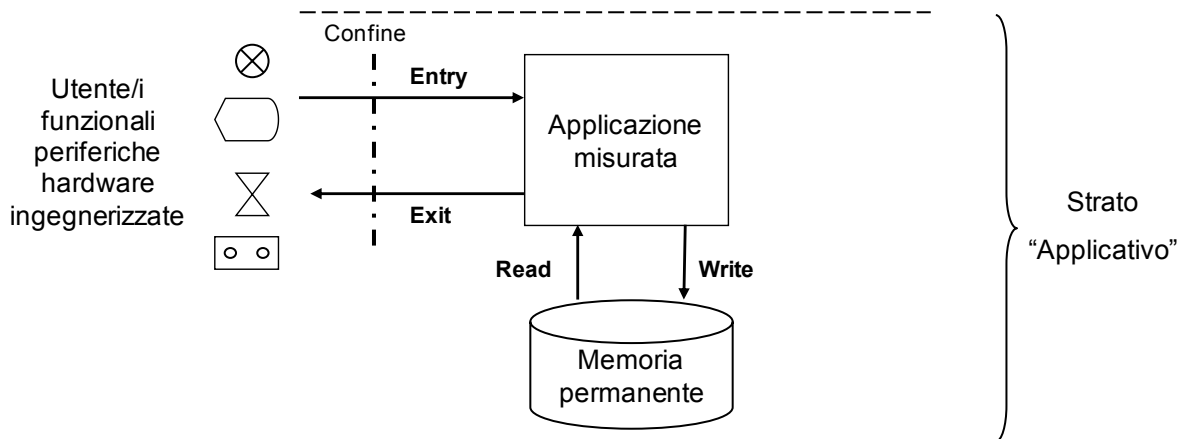


Figura 3.1.2 – Un'applicazione software integrata real-time con varie periferiche hardware ingegnerizzate come suoi utenti funzionali (vista logica).

3.2 Identificare i processi funzionali

Questo passo consiste nell'identificare l'insieme dei processi funzionali della porzione di software oggetto di misurazione, a partire dai suoi Requisiti Utente Funzionali (FUR, Functional User Requirements).

3.2.1 Definizioni

DEFINIZIONE – Processo funzionale

Un processo funzionale è una componente elementare di un insieme di Requisiti Utente Funzionali, comprendente un insieme di movimenti di dati unico, compatto e indipendentemente eseguibile. È innescato da un movimento di dati (un Entry) proveniente da un utente funzionale che informa la porzione di software che l'utente funzionale ha identificato un evento d'innescio. È completo quando ha eseguito tutto ciò che si richiede di fare in risposta all'evento d'innescio.

NOTA. Oltre a informare la porzione di software che si è verificato l'evento d'innescio, l'"Entry di innescio" può includere dei dati relativi all'oggetto d'interesse associato con l'evento.

DEFINIZIONE – Evento di innescio

Un evento (qualcosa che accade) che induce un utente funzionale della porzione di software a attivare ("innescare") uno o più processi funzionali. In un insieme di Requisiti Utente Funzionali, ogni evento che induce un utente funzionale a innescare un processo funzionale:

- non può essere suddiviso per quell'insieme di FUR, E
- o è avvenuto, o non è avvenuto.

NOTA. Eventi di clock e di temporizzazione possono essere eventi di innescio.

La relazione tra un evento d'innescò, l'utente funzionale ed il movimento di dati di Entry che innescò un processo funzionale è schematizzato in Fig. 3.2.1 seguente. L'interpretazione di questo diagramma è: *un evento è percepito da un utente funzionale e l'utente funzionale innescò un processo funzionale.*

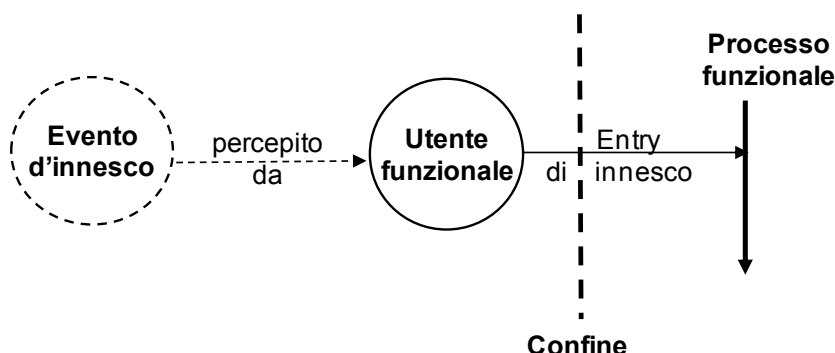


Figura 3.2.1 – Relazione tra evento d'innescò, utente funzionale e processo funzionale.

L'Entry d'innescò è solitamente un messaggio non ambiguo esplicito che informa il software che l'utente funzionale ha identificato un evento d'innescò. L'Entry d'innescò spesso include anche dei dati relativi a un oggetto d'interesse associato all'evento. Dopo che l'Entry di innescò è stato ricevuto, un processo funzionale può richiedere di ricevere e elaborare altri Entry descriventi altri oggetti di interesse.

Se un utente funzionale invia dati non corretti, per es. perché un utente-sensore è mal funzionante o perché un ordine inserito da un utente umano contiene degli errori, è generalmente compito del processo funzionare stabilire se l'evento sia effettivamente avvenuto e/o i dati inseriti siano effettivamente validi e come rispondere.

3.2.2 Approccio all'identificazione dei processi funzionali

L'approccio per identificare i processi funzionali dipende dai manufatti del software disponibili al misuratore, che a loro volta dipendono dal momento del ciclo di vita del software in cui è richiesta la misurazione e dai metodi di analisi, progettazione e sviluppo del software utilizzati. Poiché questi ultimi variano enormemente, perfino in un dato dominio software, è fuori dell'ambito di questo Manuale di Misurazione fornire uno o più processi generali per identificare i processi funzionali.

Il documento "Guideline for Sizing Business Application Software", sez. 4.4, offre ulteriori regole ed esempi su come identificare e distinguere i processi funzionali nel dominio del software applicativo aziendale.

Il consiglio più importante è che è quasi sempre utile cercare di identificare gli eventi distinti nel mondo degli utenti funzionali a cui il software deve rispondere, poiché ognuno di tali (tipi di) eventi dà luogo solitamente ad un (ma in qualche caso a più di un) (tipo di) processo funzionale. Gli eventi si possono identificare nei diagrammi di stato e nei diagrammi del ciclo di vita delle entità, poiché ogni transizione che il software deve trattare corrisponde ad un evento.

Si utilizzino le seguenti regole per verificare che i candidati processi funzionali siano stati propriamente identificati.

REGOLE – Processo funzionale
a) Un processo funzionale deve essere derivato da almeno un Requisito Utente Funzionale identificabile nell'ambito prestabilito.
b) Un (tipo di) processo funzionale deve essere eseguito quando si verifica un (tipo di) evento d'innescò identificabile.
c) Uno specifico (tipo di) evento può innescare uno o più (tipi di) processi funzionali che si svolgono in parallelo. Uno specifico (tipo di) processo funzionale può essere innescato da più di un (tipo di) evento.

- d) Un processo funzionale deve comprendere almeno due movimenti di dati, un Entry più o un Exit o un Write.
- e) Un processo funzionale deve appartenere interamente al software di uno ed un solo strato.
- f) Nel contesto del software real-time, un processo funzionale deve considerarsi terminato quando raggiunge uno stato di attesa auto-indotto (cioè il processo funzionale ha svolto tutto quanto gli è richiesto di fare in risposta all'evento d'Innesco e attende finché non riceve il successivo Entry di innesco).
- g) Si deve identificare un solo (tipo di) processo funzionale anche se i suoi FUR consentono che esso possa svolgersi con differenti sottoinsiemi del suo numero massimo di attributi dei dati di input, e anche se tali variazioni e/o differenti valori dei dati di input possono dar luogo a differenti percorsi di elaborazione nel corso del processo funzionale.
- h) Si dovrebbero distinguere (tipi di) eventi distinti e quindi (tipi di) processi funzionali distinti nei seguenti casi:
 - Quando delle decisioni hanno come risultato eventi distinti che sono separati nel tempo (per es. inserire i dati di un ordine oggi e confermare successivamente l'accettazione dell'ordine, con la necessità di una decisione separata, dovrebbe essere considerato indice di due processi funzionali distinti).
 - Quando le responsabilità delle attività sono distinte (per es. in un sistema del personale, in cui la responsabilità della gestione dei dati anagrafici di base è distinta da quella della gestione dei dati degli stipendi, indice di due processi funzionali distinti; oppure per un pacchetto software implementato in cui esiste una funzionalità, accessibile all'amministratore di sistema, per la gestione dei parametri del pacchetto, che è separata dalle funzionalità accessibili all'utente funzionale „normale“).

3.2.3 Esempi nel dominio degli applicativi aziendali

- a) Gli eventi d'Innesco di un applicativo aziendale on-line di solito avvengono nel mondo fisico degli utenti funzionali umani dell'applicativo. L'utente umano trasmette l'occorrenza dell'evento a un processo funzionale inserendo i dati relativi all'evento stesso.

ESEMPIO. In un'azienda si riceve un ordine (evento di innesco), ciò fa sì che un impiegato (utente funzionale) inserisca i dati dell'ordine (Entry di innesco che trasporta i dati relativi all'oggetto d'interesse "ordine") quale primo movimento di dati del processo funzionale "inserimento ordine".

- b) Talvolta, per un'applicazione A può esistere un'applicazione alla pari B che deve inviare dati a, o ricevere dati da, l'applicazione A. In tal caso, se l'applicazione B innesca un processo funzionale ogniqualevolta deve inviare a o ricevere dati dall'applicazione A, allora l'applicazione B è un utente funzionale dell'applicazione A.

ESEMPIO. Supponiamo che all'atto della ricezione di un ordine nell'esempio a), l'applicazione degli ordini debba inviare i dettagli del cliente ad un'applicazione centralizzata di registrazione dei clienti, che è oggetto di misurazione. L'applicazione degli ordini è ora divenuta un utente funzionale dell'applicazione centralizzata. L'applicazione degli ordini si accorge dell'evento di ricezione dei dati del cliente e innesca un processo funzionale nell'applicazione centralizzata di registrazione dei clienti per memorizzare tali dati, inviando i dati relativi all'oggetto d'interesse "cliente" come Entry di innesco all'applicazione centralizzata.

- c) In linea di principio, non vi è alcuna differenza nell'analisi di un processo funzionale, laddove esso richieda di essere elaborato on-line o in modalità batch. L'elaborazione notturna è una decisione implementativa tecnica e "tutto ciò che si richiede di fare" non è avvenuto fino a quando il processo di elaborazione notturna non sarà completato. Un flusso di elaborazione batch consiste di uno o più (tipi di) processi funzionali, ognuno dei quali dovrebbe essere analizzato "dall'inizio alla fine" indipendentemente da qualsiasi altro processo funzionale dello stesso flusso. Ogni processo funzionale ha il proprio Entry di innesco che deve essere misurato.

ESEMPIO. Supponiamo che gli ordini dell'esempio a) siano inseriti on-line, ma archiviati per l'elaborazione batch notturna automatica. L'utente funzionale è sempre l'umano che ha inserito i dati on-line; l'Entry di innesco è sempre dato dai dati dell'ordine. Vi è un solo processo funzionale di inserimento e relativa elaborazione degli ordini.

- d) Segnali periodici dati da un clock (“battiti”, “clock ticks”) possono innescare fisicamente un processo funzionale.

ESEMPIO. Supponiamo di avere un FUR relativo a un processo batch di fine anno per riportare i risultati aziendali dell'anno concluso e azzerare le posizioni per l'avvio del nuovo anno. Fisicamente, un segnale di clock di fine anno generato dal sistema operativo induce all'avvio il flusso batch, consistente in uno o più processi funzionali. Quei processi funzionali del flusso che utilizzano i dati di input del flusso stesso dovrebbero essere analizzati come al solito (per es. i dati di input di un qualsiasi processo funzionale comprendono uno o più Entry, il primo dei quali è l'Entry di innesco per tale processo).

Assumiamo tuttavia che esista un particolare processo funzionale nel flusso che non necessiti di alcun dato di input per produrre il proprio insieme di report. Fisicamente, l'utente funzionale (umano) ha delegato al sistema operativo il compito di innescare tale processo funzionale. Poiché ogni processo funzionale deve avere almeno un Entry di innesco, possiamo assegnare tale ruolo per questo processo al segnale di clock di fine anno che ha avviato il flusso batch. Questo processo funzionale può poi richiedere vari Read e numerosi Exit per produrre i propri report. Logicamente, l'analisi di questo esempio non sarebbe diversa se l'utente funzionale avviasse la produzione di uno o più report con il click del mouse o una voce di menu on-line, piuttosto che delegando la produzione di report al sistema operativo tramite un flusso batch.

- e) Un singolo evento può innescare uno o più processi funzionali che si svolgono in modo indipendente.

ESEMPIO. Un segnale di clock di “fine settimana” induce l'avvio sia della produzione di report, sia del processo di verifica della scadenza di limiti di tempo in un sistema di workflow.

- f) Un singolo processo funzionale può essere attivato da più di un tipo di evento d'innesco.

ESEMPIO. In un sistema bancario, un estratto conto potrebbe essere innescato da un processo batch di “fine mese”, come anche da una specifica richiesta dell'utente.

Per vari esempi su come distinguere gli eventi di innesco e i processi funzionali nei flussi batch, v. il documento “Guideline for Sizing Business Application Software”, sez. 4.6.3.

3.2.4 Esempi nel dominio delle applicazioni real-time

- a) Un evento di innesco è tipicamente rilevato da un sensore.

ESEMPIO 1. Quando la temperatura raggiunge un certo valore (evento di innesco), un sensore (utente funzionale) è indotto a inviare un segnale (movimento dato di Entry di innesco) per accendere una spia di segnalazione (processo funzionale).

ESEMPIO 2. Un velivolo militare ha un sensore che rileva l'evento “missile in avvicinamento”. Il sensore è un utente funzionale del software integrato che deve rispondere alla minaccia. Per questo software, un evento si verifica solo quando il sensore rileva qualcosa ed è proprio il sensore (l'utente funzionale) che innesca il software inviandogli un messaggio (Entry di innesco), dicendo per es. “il sensore n. 2 ha rilevato un missile”, oltre forse a un flusso di dati relativamente a quanto velocemente si sta avvicinando il missile e alle sue coordinate. Il missile è l'oggetto d'interesse.

- b) Segnali periodici da un clock (“battiti”, “clock ticks”) possono innescare un processo funzionale.

ESEMPIO. In un software di controllo di processo real-time, un battito (evento d'innesco) del clock (utente funzionale) induce il clock a inviare un segnale (Entry di innesco), un messaggio a 1 bit che dice a un processo funzionale di ripetere il proprio normale ciclo di controllo. Il processo funzionale allora legge vari sensori, ricevendo i dati relativi a degli oggetti di interesse, e attua ogni azione necessaria. Non vi sono ulteriori dati associati al battito del clock.

- c) Un singolo evento può innescare uno o più processi funzionali che si svolgono in modo indipendente e parallelo.

ESEMPIO. Una condizione di emergenza rilevata in una centrale nucleare può innescare processi funzionali indipendenti che in diversi punti della centrale per abbassare le barre di controllo, attivare i sistemi di raffreddamento, chiudere le valvole, attivare allarmi sonori per avvisare gli operatori, ecc.

- d) Un singolo processo funzionale può essere innescato da più di un tipo di evento d'innescamento.
ESEMPIO. Il rientro del carrello di un velivolo può essere innescato dal sensore di "massa in volo", da un comando del pilota.

3.2.5 Ulteriori considerazioni sui processi funzionali distinti

Il software distingue gli eventi e definisce i corrispondenti processi funzionali solamente in base ai suoi FUR. Quando si misura il software, può essere talvolta difficile stabilire quali siano i distinti eventi che il software deve riconoscere. È questo il caso in particolare in cui i FUR originali non sono più disponibili e per es. lo sviluppatore può aver trovato economicamente vantaggioso unire insieme vari requisiti. Per l'analisi può essere utile esaminare in che modo sono organizzati i dati di input (vedi sotto) oppure esaminare i menu di alcuni software installati per aiutare a distinguere gli eventi separati a cui il software deve rispondere e i corrispondenti processi funzionali.

ESEMPIO 1. Quando vi è un requisito utente funzionale relativo ai crediti di imposta per un nuovo figlio e anche ai "crediti di imposta sul lavoro" sotto una certa soglia di reddito, questi sono requisiti per cui il software deve rispondere a due eventi che sono distinti del mondo degli utenti funzionali umani. Ci dovrebbero quindi essere due processi funzionali, anche laddove si potrebbe utilizzare un solo modulo delle tasse per raccogliere i dati relativi a entrambe le casistiche.

ESEMPIO 2. Se il reddito in una istanza (per una persona) supera il limite del credito d'imposta sul lavoro, e in un'altra istanza (per un'altra persona) no, questa differenza non dà luogo a due processi funzionali distinti, ma indica semplicemente due differenti condizioni a cui si viene incontro nello stesso unico processo funzionale.

ESEMPIO 3. Come esempio della regola g), se un'occorrenza di uno specifico evento innesca l'Entry di un gruppo di dati che comprende gli attributi dei dati A, B e C, e il FUR consente che un'altra occorrenza dello stesso evento inneschi un Entry di un gruppo di dati che ha valori per i soli attributi A e B, questo non comporta l'identificazione di due (tipi di) processi funzionali. Si identificano un solo Entry e un solo (tipo di) processo funzionale, che muove e manipola gli attributi dei dati A, B e C.

Una volta identificato, ogni processo funzionale può essere registrato su una singola linea, sotto il relativo strato o componente alla pari, nella matrice del Modello Generale del Software (App. A), sotto la corrispondente etichetta.

3.3 Identificare gli oggetti di interesse e i gruppi di dati

3.3.1 Definizioni e principi

Questo passo consiste nell'identificare i gruppi di dati referenziati dalla porzione di software oggetto di misurazione. Per identificare i gruppi di dati, specialmente nel dominio del software applicativo aziendale, è solitamente utile identificare gli oggetti di interesse e probabilmente anche i loro attributi. I gruppi di dati sono mossi attraverso "movimenti di dati", che sono identificati nel prossimo capitolo.

DEFINIZIONE – Oggetto d'interesse

Qualsiasi “cosa” che è identificata dal punto di vista dei Requisiti Utente Funzionali. Può essere qualsiasi cosa fisica, così come qualsiasi oggetto concettuale o parte di un oggetto concettuale appartenente al mondo dell'utente funzionale, riguardo al quale si richiede che il software elabori e/o memorizzi dei dati.

NOTA. Nel metodo COSMIC, si utilizza il termine “oggetto d'interesse” per evitare termini legati a specifici metodi di ingegneria del software. Questo termine non implica “oggetti” nell'accezione dei metodi Object Oriented.

DEFINIZIONE – Gruppo di dati

Un gruppo di dati è un insieme distinto, non vuoto, non ordinato e non ridondante di attributi dei dati, in cui ogni attributo dei dati descrive un aspetto complementare del medesimo oggetto d'interesse.

DEFINIZIONE – Memoria permanente

La memoria permanente è la memoria che permette a un processo funzionale di memorizzare un gruppo di dati oltre la vita del processo funzionale e/o dalla quale un processo funzionale può reperire un gruppo di dati memorizzato da un altro processo funzionale, da un'occorrenza precedente del medesimo processo funzionale o da qualche altro processo.

NOTA 1. Nel modello COSMIC, essendo situata dal lato software del confine, la memoria permanente non è considerata essere un utente del software.

NOTA 2. Un esempio di “qualche altro processo” sarebbe la produzione di una memoria di sola lettura (Read-Only Memory, ROM).

Una volta identificato, ogni candidato gruppo di dati deve soddisfare i seguenti principi:

PRINCIPI – Gruppo di dati

- a) Ogni gruppo di dati identificato deve essere unico e distinguibile tramite la sua raccolta di attributi dei dati unica.
- b) Ogni gruppo di dati deve essere direttamente collegato a un oggetto d'interesse nei Requisiti Funzionali Utente del software.
- c) Un gruppo di dati deve essere materializzato nel sistema informatico che supporta il software.

Una volta identificato, ogni gruppo di dati può essere registrato in una colonna individuale nella matrice del Modello Generale del Software (App. A), sotto l'etichetta corrispondente.

3.3.2 Sulla materializzazione di un gruppo di dati

In pratica, la materializzazione di un gruppo di dati può assumere molte forme, per es. come:

- a) una struttura fisica di record su di una periferica di memoria permanente (file, tabella di database, memoria ROM, ecc.);
- b) una struttura fisica nella memoria volatile del computer (struttura dati allocata dinamicamente o tramite un blocco pre-allocato di spazio di memoria);
- c) la presentazione raggruppata di attributi di dati funzionalmente collegati su di una periferica di input/output (schermo video, report stampato, pannello di controllo a display, ecc.);
- d) un messaggio in trasmissione tra una periferica e un computer, o in una rete, ecc.

3.3.3 Sull'identificazione di oggetti di interesse e gruppi dati

La definizione ed i principi relativi agli oggetti di interesse e ai gruppi di dati sono intenzionalmente generici in modo da poter essere applicabili al più ampio range possibile di software. Questa qualità comporta talvolta una certa difficoltà nell'applicare la definizione e i principi quando si misura una specifica porzione di software. Tuttavia, i seguenti casi ed esempi sono concepiti per assistere nell'applicazione dei principi a casi specifici.

Quando si affronta la necessità di analizzare un gruppo di attributi di dati che è mosso in ingresso o in uscita da un processo funzionale o da un processo funzionale da o verso la memoria permanente, è *criticamente importante* decidere se tutti gli attributi trasportano informazioni riguardanti un singolo "oggetto d'interesse", poiché è quest'ultimo che determina il numero di "gruppi di dati" distinti come definito dal metodo COSMIC. Per es. se gli attributi dei dati in input a un processo funzionale sono attributi di tre oggetti di interesse distinti, si devono identificare tre movimenti di dati di "Entry" distinti.

CASO 1. Dominio degli applicativi aziendali

- Nel dominio del software applicativo aziendale, un oggetto d'interesse potrebbe essere l'"impiegato" (fisico) oppure l'"ordine" (concettuale), assumendo che il software debba memorizzare dati sugli impiegati e sugli ordini. Nel caso dell'"ordine", discende solitamente dai FUR relativi agli ordini con righe multiple che si identificano due oggetti di interesse: "ordine" e "riga ordine". I gruppi di dati corrispondenti potrebbero essere denominati "dati dell'ordine" e "dati della riga dell'ordine".
- Si formano dei gruppi di dati ogni qualvolta si è in presenza di una interrogazione ad hoc che richiede dei dati riguardanti una certa "cosa", i cui dati non sono conservati nella memoria permanente, ma che possono essere derivati da altri dati conservati nella memoria permanente. Il movimento di dati di Entry di un'interrogazione ad hoc (i criteri di selezione per derivare i dati richiesti) ed il movimento di dati di Exit (contenente gli attributi dei dati desiderati) muovono entrambi i dati relativi a questa certa "cosa". Questi dati costituiscono gruppi di dati temporanei che non sopravvivono al termine dell'esecuzione del processo funzionale. Essi sono comunque gruppi di dati validi poiché attraversano il confine tra il software e il proprio utente/i.

ESEMPIO. Sia data un'interrogazione ad hoc sul database del personale per estrarre la lista dei nominativi di tutti gli impiegati con più di 35 anni di età. L'Entry muove un gruppo di dati contenente i criteri di selezione. L'Exit muove un gruppo di dati contenente il singolo attributo "nominativo"; l'"oggetto d'interesse" (o "cosa") è "tutti gli impiegati con più di 35 anni di età". È importante, quando si registra il processo funzionale, dare un nome esplicito al gruppo di dati temporaneo in relazione al suo oggetto d'interesse, piuttosto che collegarlo all'oggetto/i di interesse da cui si deriva il risultato dell'interrogazione ad hoc.

Per una discussione dettagliata dei metodi di analisi dei dati per determinare gli oggetti di interesse e i gruppi di dati distinti, si rimanda il lettore al documento "Guideline for Sizing Business Application Software".

CASO 2. Dominio del software real-time

La pratica di misurazione del software real-time ha mostrato come i gruppi di dati per questo tipo di software spesso assumano le seguenti forme:

- Movimenti di dati che costituiscono Entry da periferiche fisiche tipicamente contengono dati relativi allo stato di un singolo oggetto d'interesse, come il fatto che una valvola sia aperta o chiusa, o indicano il momento in cui i dati nella memoria volatile a breve termine sono validi o non validi, o contengono dati che indicano che un evento critico si è verificato e causa un'interruzione. Allo stesso modo, un comando di Exit per accendere o spegnere una spia di allarme trasporta dati relativi a un singolo oggetto d'interesse.
- Uno switch di messaggi può ricevere un gruppo di dati di messaggio come un Entry e inoltrarlo immutato come un Exit, in base ai FUR di una specifica porzione di software. Gli attributi del gruppo di dati del messaggio potrebbero essere per es. "mittente, destinatario, codice_percorso, contenuto_messaggio"; l'oggetto d'interesse del messaggio è il "messaggio".
- Una struttura dati comune, rappresentante oggetti di interesse menzionati nei Requisiti Utente Funzionali, che può essere mantenuta da processi funzionali e che è accessibile alla maggior parte dei processi funzionali che si trovano nel software misurato.
- Una struttura dati referenziata, rappresentante grafici o tabelle di valori trovati nei Requisiti Utente Funzionali, che è conservata nella memoria permanente (per es. nella memoria ROM) e è accessibile alla maggior parte dei processi funzionali che si trovano nel software misurato.
- File, comunemente classificati come „file piatti“, rappresentanti oggetti di interesse menzionati nei Requisiti Utente Funzionali, che sono conservati su una periferica di memoria permanente.

3.3.4 Gruppi di dati che non sono candidati ad essere movimenti di dati

Qualunque dato che appare in schermate di input o output o report e che non è collegato a un oggetto d'interesse per un utente funzionale non dovrebbe essere identificato come indice di un movimento di dati, e quindi non dovrebbe essere misurato. Per esempio:

- Dati a livello generale dell'applicazione, come testata e pie" di pagina (nome azienda, nome applicazione, data di sistema, ecc.) che compaiono su tutte le schermate.
- Comandi di controllo (un concetto definito solo nel dominio degli applicativi aziendali) che permettono a un utente funzionale di controllare l'utilizzo del software, piuttosto che muovere dati, per es. pagina su/giù, click su "OK" per riscontro di un messaggio di errore, ecc. – v. sez. 4.1.10.

Il Modello Generale del Software COSMIC assume che qualsiasi manipolazione di dati in un processo funzionale è associata ad uno dei quattro tipi di movimento di dati – v. sez. 4.1.6. Quindi, nessun movimento o manipolazione di dati all'interno di un processo funzionale può essere candidato come movimento di dati in aggiunta a Entry, Exit, Read e Write. (Per esempi di manipolazioni e movimenti di dati che potrebbero essere mal interpretati come movimenti di dati, v. sez. 4.1.4, principio c) per un Read, e sez. 4.1.5, principio d) per un Write).

3.3.5 L'utente funzionale come oggetto d'interesse

In molti semplici casi di software real-time come descritto nel primo punto del caso 2 in sez. 3.3.3, la periferica fisica, un utente funzionale, è indistinguibile dall'oggetto d'interesse del movimento di dati che essa genera o riceve. In tali casi, è di scarso valore aggiunto documentare un oggetto d'interesse come se fosse qualcosa di separato dall'utente funzionale. Il punto importante è utilizzare questi concetti, quando utili, per distinguere gruppi di dati distinti, e quindi movimenti di dati distinti.

ESEMPIO. Consideriamo un sensore di temperatura "A" che, quando interrogato da un processo funzionale, invia la temperatura corrente al processo. L'utente funzionale è il sensore "A"; il nome del messaggio di Entry potrebbe essere "temperatura corrente di A"; l'oggetto d'interesse di questo messaggio potrebbe anche essere visto come "il sensore A". Teoricamente, l'oggetto d'interesse non è "il sensore A" ma "il materiale o oggetto la cui temperatura è misurata dal sensore A". In pratica comunque è di scarso valore aggiunto documentare tali distinzioni fini e può non valere la pena identificare separatamente l'oggetto d'interesse.

3.4 Identificare gli attributi dei dati (opzionale)

Questa sezione tratta l'identificazione degli attributi dei dati referenziati dalla porzione di software oggetto di misurazione. In questa versione del metodo di misurazione non è obbligatorio identificare gli attributi dei dati. Tuttavia può essere utile analizzare e identificare gli attributi dei dati nel corso dell'attività di distinzione dei gruppi di dati e degli oggetti di interesse. Gli attributi dei dati potrebbero anche essere identificati se si richiede una sotto-unità di misura della dimensione, come proposto in sez. 4.5 "Estendere il metodo di misurazione COSMIC".

3.4.1 Definizione

DEFINIZIONE – Attributo dei dati
Un attributo dei dati è la più piccola porzione di informazione, in un gruppo di dati identificato, che trasporta un significato dal punto di vista dei Requisiti Utente Funzionali del software.

ESEMPIO 1. Attributi dei dati nel contesto del dominio degli applicativi aziendali:

- Elementi dei dati definiti nel dizionario dei dati.
- Attributi dei dati presenti in un modello concettuale o logico dei dati.

ESEMPIO 2. Attributi dei dati nel contesto del software applicativo real-time:

- Attributi dei dati di un segnale ricevuto da un sensore.
- Attributi dei dati di un messaggio in trasmissione.

3.4.2 Sulla relazione tra attributi dei dati e gruppi di dati

In teoria, un gruppo di dati potrebbe contenere anche solo un dato, se ciò è sufficiente dal punto di vista dei Requisiti Utente Funzionali a descrivere l'oggetto d'interesse. In pratica, casi del genere si presentano di solito nel software applicativo real-time (per es. l'input di un clock); sono meno frequenti nel software applicativo aziendale.

LA FASE DI MISURAZIONE

Questo capitolo presenta le regole e il metodo della fase di misurazione del processo di misurazione COSMIC. Il metodo generale per misurare una porzione di software, quando i suoi Requisiti Utente Funzionali siano stati espressi secondo il Modello Generale del Software COSMIC, è riassunto in Fig. 4.0 seguente.

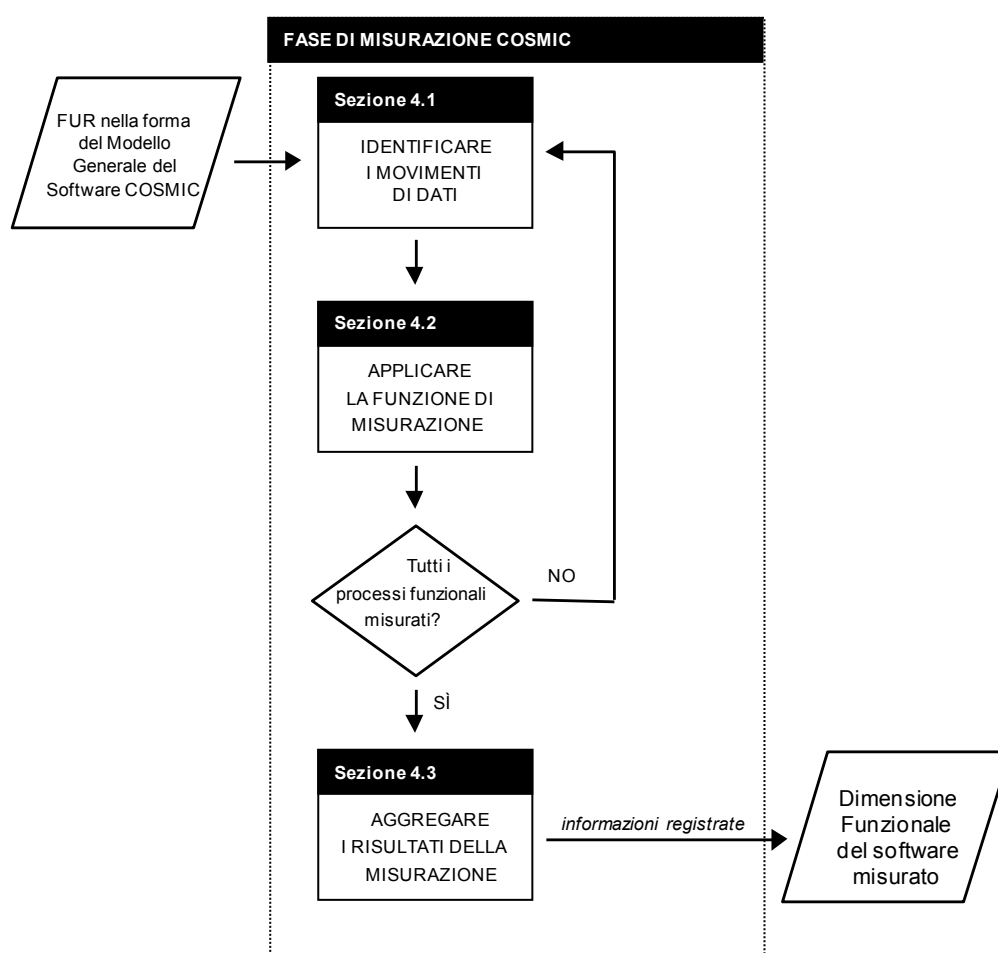


Figura 4.0 – Processo generale della Fase di Misurazione COSMIC.

Ogni passo del metodo è oggetto di una specifica sezione di questo capitolo, in cui si presentano le definizioni e i principi da applicare, insieme a regole ed esempi.

4.1 Identificare i movimenti di dati

Questo passo consiste nell'identificare i movimenti di dati (Entry, Exit, Read e Write) di ogni processo funzionale.

4.1.1 Definizione dei tipi di movimento di dati

DEFINIZIONE – Movimento di dati

Una componente funzionale di base che muove un unico gruppo di dati.

NOTA 1. Esistono quattro sotto-tipi di movimento di dati: Entry, Exit, Read e Write (tipi).

NOTA 2. Per scopi di misurazione, ogni sotto-tipo di movimento di dati è considerato includere certe manipolazioni dei dati ad esso associate – v. sez. 4.1.6 per dettagli.

NOTA 3. Più precisamente, è un'"*occorrenza*" di un movimento di dati, non il "*tipo*" di movimento di dati, che *muove* effettivamente le *occorrenze* del gruppo di dati (non i *tipi*). Questo commento si applica anche alle definizioni di Entry, Exit, Read e Write.

DEFINIZIONE – Entry (E)

Un movimento di dati che muove un gruppo di dati da un utente funzionale attraverso il confine nel processo funzionale che lo richiede.

NOTA. Un Entry è considerato includere certe manipolazioni dei dati ad esso associate (v. sez. 4.1.6).

DEFINIZIONE – Exit (X)

Un movimento di dati che muove un gruppo di dati da un processo funzionale attraverso il confine all'utente funzionale che lo richiede.

NOTA. Un Exit è considerato includere certe manipolazioni dei dati ad esso associate (v. sez. 4.1.6).

DEFINIZIONE – Read (R)

Un movimento di dati che muove un gruppo di dati dalla memoria permanente verso il processo funzionale che lo richiede.

NOTA. Un Read è considerato includere certe manipolazioni dei dati ad esso associate (v. sez. 4.1.6).

DEFINIZIONE – Write (W)

Un movimento di dati che muove un gruppo di dati situato all'interno di un processo funzionale verso la memoria permanente.

NOTA. Un Write è considerato includere certe manipolazioni dei dati ad esso associate (v. sez. 4.1.6).

La Fig. 4.1.1 seguente illustra le relazioni generali tra i quattro tipi di movimento di dati, il processo funzionale a cui essi appartengono e il confine del software misurato.

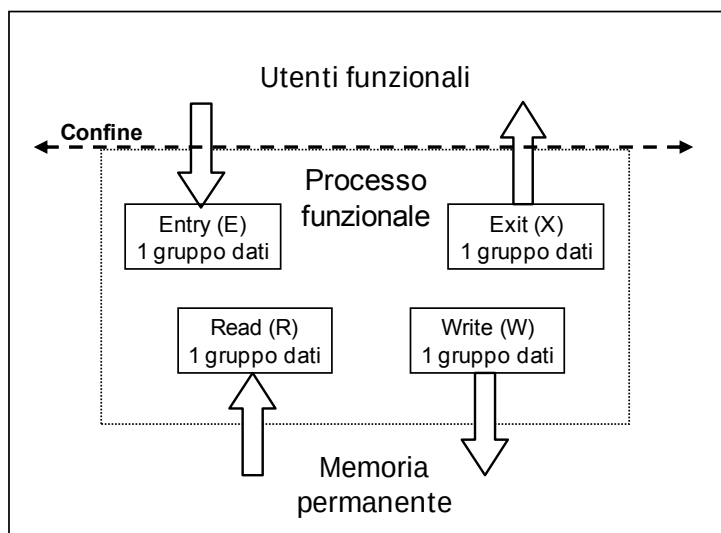


Figura 4.1.1 – I quattro tipi di movimento di dati e le loro relazioni con il processo funzionale e i gruppi di dati.

4.1.2 Identificare gli Entry (E)

Una volta identificato, un candidato movimento di dati di tipo Entry deve soddisfare i seguenti principi:

PRINCIPI – Entry (E)

- Un Entry deve muovere un unico gruppo di dati, descrivente un unico oggetto d'interesse, da un utente funzionale attraverso il confine e nel processo funzionale di cui fa parte l'Entry. Se l'input di un processo funzionale comprende più di un gruppo di dati, si identifichi un Entry per ciascun gruppo di dati distinto in input. (V. anche sez. 4.1.7 sulla "Unicità dei movimenti di dati".)
- Un Entry non deve eseguire output di dati attraverso il confine, né leggere o scrivere dati.
- Un movimento di dati di Entry deve includere qualsiasi funzionalità di "richiesta di inserimento", *eccetto* dove si debba specificare esplicitamente all'utente funzionale quali dati sono richiesti, nel qual caso si richiede una coppia Exit/Entry per ottenere ciascun gruppo di dati. (V. anche sez. 4.1.9.)

Le seguenti regole aiutano a convalidare lo stato di un tipo di movimento di dati candidato Entry:

REGOLE – Entry (E)

- Il gruppo di dati di un Entry d'innescò può consistere di solo un attributo che informa semplicemente il software che "un evento Y è avvenuto". Molto spesso, specialmente nel software applicativo aziendale, il gruppo dati dell'Entry d'innescò ha vari attributi che informano il software che "un evento Y è avvenuto e questi sono i dati riguardanti questo particolare evento".
- Segnali periodici di clock che sono eventi d'innescò devono essere sempre esterni al software oggetto di misurazione. Quindi per es. un evento periodico di clock che avviene ogni 3 secondi deve essere associato a un Entry che muove un gruppo dati con un solo attributo. Si noti che non fa alcuna differenza se l'evento d'innescò è generato periodicamente dall'hardware o da un'altra porzione di software esterna al

confine del software oggetto di misurazione.

- c) A meno che non sia necessario uno specifico processo funzionale, ottenere il valore di tempo dal clock di sistema non deve essere considerato come scatenante un Entry.
- d) Se un'occorrenza di uno specifico evento innesca l'Entry di un gruppo di dati contenente fino a "n" attributi di un particolare oggetto d'interesse e i FUR consentono che altre occorrenze dello stesso evento possano innescare un Entry di un gruppo di dati valorizzato solo per un sottoinsieme degli "n" attributi dell'oggetto d'interesse, allora si deve identificare un solo Entry, comprendente tutti gli "n" attributi dei dati.

Un esempio che illustra la regola c): quando un processo funzionale scrive un "timestamp", non si identifica alcun Entry per ottenere il valore di clock del sistema.

Una volta identificato, ogni movimento di dati di Entry può essere registrato contrassegnando con una "E" la casella corrispondente della matrice del Modello Generale del Software (App. A).

4.1.3 Identificare gli Exit (X)

Una volta identificato, un candidato movimento di dati di Exit deve soddisfare i seguenti principi:

PRINCIPI – Exit (X)

- a) Un Exit deve muovere un unico gruppo di dati, descrivente un unico oggetto d'interesse, dal processo funzionale di cui fa parte l'Exit attraverso il confine verso un utente funzionale. Se l'output di un processo funzionale comprende più di un gruppo di dati, si identifichi un Exit per ciascun gruppo di dati distinto in output. (V. anche sez. 4.1.7 sulla "Unicità dei movimenti di dati".)
- b) Un Exit non deve eseguire input di dati attraverso il confine, né leggere o scrivere dati.

Le seguenti regole potrebbero risultare utili per convalidare lo stato di un tipo di movimento di dati candidato Exit:

REGOLE – Exit (X)

- a) Tutti i messaggi generati e emessi dal software privi di dati utente (per es. messaggi d'errore) devono essere considerati come valori di un solo attributo di un unico oggetto d'interesse (che potrebbe essere chiamato "indice d'errore"). Quindi, si deve identificare un unico Exit per rappresentare tutte queste occorrenze di messaggi all'interno di ogni processo funzionale in cui siano richiesti dai FUR.
- b) Se un Exit di un processo funzionale muove un gruppo di dati contenente fino a "n" attributi di un particolare oggetto d'interesse e i FUR consentono il processo funzionale un'occorrenza di Exit che muove un gruppo di dati valorizzato solo per un sottoinsieme degli "n" attributi dell'oggetto d'interesse, allora si deve identificare un solo Exit, comprendente tutti gli "n" attributi dei dati.

Esempi della precedente regola a) sono i seguenti:

ESEMPIO 1. In un dialogo uomo-computer, esempi di messaggi di errore possibili durante la validazione dei dati digitati, potrebbero essere "errore di formato", "cliente non trovato", "errore: spuntare l'indicazione di avere letto il paragrafo termini e condizioni", "superato il limite di credito", ecc. Tutti questi tipi di errori devono essere considerati come occorrenze di un Exit in ogni processo funzionale dove tali messaggi sono presenti (che potrebbe essere chiamato "messaggi di errore").

ESEMPIO 2. Messaggi di errore inviati a un utente umano, ma non generati dal software applicativo, devono essere completamente ignorati nella misurazione dell'applicazione. Un esempio di tale messaggio, passato dal sistema operativo, potrebbe essere "stampante X non disponibile".

ESEMPIO 3. In un dialogo computer-utente umano, se un messaggio è l'output di una situazione di errore ma contiene dati dell'utente funzionale, allora dovrebbe essere contato come un Exit nel processo funzionale in cui avviene. Un esempio di tale messaggio potrebbe essere "Attenzione: l'importo che desideri prelevare eccede la disponibilità di 100\$" (dove "100\$" è una variabile calcolata). In questo esempio, l'Exit contiene un gruppo di dati sul conto corrente bancario del cliente.

ESEMPIO 4. In un sistema real-time, un processo funzionale che controlla periodicamente il corretto funzionamento di tutte le periferiche hardware potrebbe inviare un messaggio che dice "Il sensore X è guasto" dove "X" è una variabile. Questo messaggio dovrebbe essere identificato come un Exit nel processo funzionale (indipendentemente dal valore di "X").

ESEMPIO 5. Consideriamo i processi funzionali "A" e "B". "A" può potenzialmente inviare 2 distinti messaggi di conferma e 5 messaggi di errore ai propri utenti funzionali e "B" può inviare 8 messaggi di errore ai propri utenti funzionali. In questo esempio, si identificherebbero un Exit nell'ambito del processo funzionale "A" (che gestisce $5 + 2 = 7$ messaggi) e un distinto Exit nell'ambito del processo funzionale "B" (che gestisce 8 messaggi).

Una volta identificato, ogni movimento di dati di Exit può essere registrato contrassegnando con una "X" la casella corrispondente della matrice del Modello Generale del Software (App. A).

4.1.4 Identificare i Read (R)

Una volta identificato, un candidato movimento di dati di Read deve soddisfare i seguenti principi:

PRINCIPI – Read (R)	
a)	Un Read deve muovere un unico gruppo di dati, descrivente un unico oggetto d'interesse, dalla memoria permanente al processo funzionale di cui fa parte il Read. Se il processo funzionale deve reperire più di un gruppo di dati dalla memoria permanente, si identifichi un Read per ogni gruppo di dati distinto reperito. (V. anche sez. 4.1.7 sull'"Unicità dei movimenti di dati".)
b)	Un Read non deve ricevere o inviare dati attraverso il confine, né scrivere dati.
c)	Durante un processo funzionale, il movimento o la manipolazione di costanti o variabili che sono interne al processo funzionale e che possono essere cambiate solo da un programmatore, l'elaborazione di risultati intermedi in un calcolo, o di dati memorizzati da un processo funzionale risultanti solo dall'implementazione, invece che dai FUR, non devono essere considerati come movimenti di dati di Read.
d)	Un movimento di dati di Read deve includere sempre qualsiasi funzionalità di "richiesta di lettura" (quindi non si deve mai contare un movimento di dati distinto per una qualsiasi funzionalità di "richiesta di lettura"). (V. anche sez. 4.1.9.)

Una volta identificato, ogni movimento di dati di Read può essere registrato contrassegnando con una "R" la casella corrispondente della matrice del Modello Generale del Software (App. A).

4.1.5 Identificare i Write (W)

Una volta identificato, un candidato movimento di dati di Write deve soddisfare i seguenti principi:

PRINCIPI – Write (W)	
a)	Un Write deve muovere un unico gruppo di dati, descrivente un unico oggetto d'interesse, dal processo funzionale di cui fa parte il Write verso la memoria permanente. Se il processo funzionale deve muovere più di un gruppo di dati verso la memoria permanente, si identifichi un Write per ogni gruppo di dati distinto mosso verso la memoria permanente. (V. anche sez. 4.1.7 sull'"Unicità dei movimenti di dati".)
b)	Un Write non deve ricevere o inviare dati attraverso il confine, né leggere dati.
c)	Un requisito di cancellare un gruppo di dati dalla memoria permanente deve essere

misurato come un singolo movimento di dati di Write.

- d) Durante un processo funzionale, il movimento o la manipolazione di dati che non permangono quando il processo funzionale è completato, l'aggiornamento di variabili interne al processo funzionale o la produzione di risultati intermedi in un calcolo non devono essere considerati come movimenti di dati di Write.

Una volta identificato, ogni movimento di dati di Write può essere registrato contrassegnando con una "W" la casella corrispondente della matrice del Modello Generale del Software (App. A).

4.1.6 Sulle manipolazioni dei dati associate ai movimenti di dati

Come definito dal principio d) del Modello Generale del Software (v. sez. 1.4), i sotto-processi sono o movimenti di dati o manipolazioni di dati. Tuttavia, per attuale convenzione COSMIC (v. principio j) del Modello Generale del Software), non è riconosciuta l'esistenza separata dei sotto-processi di manipolazione dei dati.

DEFINIZIONE – Manipolazione dei dati

Qualsiasi cosa che succede ai dati che sia differente da un movimento dei dati dentro o fuori di un processo funzionale, o tra un processo funzionale e la memoria permanente.

Il seguente principio determina come il metodo COSMIC tratta la manipolazione dei dati.

PRINCIPIO – Manipolazione dei dati associata ai movimenti di dati

Tutte le manipolazioni dei dati in un processo funzionale devono essere associate ai quattro tipi di movimenti di dati (E, X, R e W). Per convenzione, si assume che i movimenti di dati di un processo funzionale rappresentino anche la manipolazione dei dati del processo funzionale.

La necessità di definire quali tipi di manipolazione di dati sono associati con quali tipi di movimento di dati sorge solamente quando si misurano le *modifiche* del software (v. sez. 4.4). Una tipica richiesta di cambiamento impatta sia gli attributi mossi che la manipolazione associata con un particolare movimento di dati, ma *può* impattare solo la manipolazione del dato e non il movimento del dato. Anche un tale cambiamento deve poter essere identificato e misurato. Quindi quando c'è una qualsiasi richiesta di cambiare una manipolazione di dati in un processo funzionale, il misuratore deve identificare quale movimento di dati è associato con il cambiamento della manipolazione del dato.

Seguono delle linee guida generali per identificare la manipolazione dei dati rappresentata da ciascuno dei movimenti di dati.

Movimento di dati di Entry

Un Entry include qualsiasi manipolazione dei dati:

- per assicurare che il gruppo di dati sia trattato da una funzionalità utente (es. manipolazione di formattazione e presentazione), e/o
- per essere validato,

ma nessuna manipolazione che implichi un altro movimento di dati, nè alcuna manipolazione successiva all'inserimento e alla validazione del gruppo di dati.

Per es. un Entry include tutta la manipolazione, la formattazione e la presentazione a video dei dati inseriti, ECCETTO qualsiasi Read che potrebbe essere richiesto per validare eventuali codici in input o per ottenere eventuali descrizioni associate.

Un movimento di dati di Entry include ogni "richiesta di input" eccetto quando il processo funzionale necessita di informare l'utente funzionale quale dato spedire (v. sez. 4.1.9 per quali dati spedire e sez. 4.1.10 per il trattamento di una schermata vuota di input).

Movimento di dati di Exit

Un Exit include qualsiasi manipolazione dei dati:

- per creare gli attributi di un gruppo di dati per essere inviato all'esterno, e/o
- per assicurare che il gruppo di dati sia inviato all'esterno (es. manipolazione di formattazione e presentazione) e sia instradato verso il corretto utente funzionale,

ma nessuna manipolazione che implica un altro movimento di dati.

Per es. un EXIT include tutta l'elaborazione per formattare e preparare per la stampa particolari attributi dei dati, inclusi i campi di testata leggibili dall'utente umano¹³, ECCETTO qualsiasi Read o Entry che potrebbe essere richiesto per fornire i valori o le descrizioni di qualche dato da stampare.

Movimento di dati di Read

Un Read include tutta l'elaborazione e/o i calcoli necessari per reperire un gruppo di dati dalla memoria permanente, ma nessuna manipolazione che implichi un altro tipo di movimento di dati e nemmeno altre manipolazioni dopo che il Read è stato completato con successo.

Per es. un Read include tutti i calcoli matematici e il processo logico richiesto per reperire un gruppo di dati dalla memoria permanente, ma non la manipolazione di questi attributi dopo che il gruppo di dati è stato reperito.

Un Read include anche sempre ogni funzione di "richiesta di Read" (v. sez. 4.1.9).

Movimento di dati di Write

Un Write include tutta l'elaborazione e/o i calcoli per creare un gruppo di dati che deve essere scritto, ma nessuna manipolazione che implichi un altro tipo di movimento di dati e nemmeno altre manipolazioni dopo che il Write è stato completato con successo.

Per es. un Write include tutti i calcoli matematici e il processo logico richiesto per creare o aggiornare un gruppo di dati per essere scritto, o per essere cancellato, ECCETTO qualsiasi Read o Entry che potrebbe essere richiesto per fornire i valori di eventuali attributi dei dati facenti parte del gruppo da scrivere o cancellare.

4.1.7 Unicità dei movimenti di dati e possibili eccezioni¹⁴

Il Modello Generale del Software assume che *normalmente* in un qualsiasi processo funzionale *tutti* i dati che descrivono un qualsiasi oggetto d'interesse siano inseriti con un solo movimento di dati di tipo Entry e/o siano letti con un solo movimento di dati di tipo Read e/o siano scritti con un solo movimento di dati di tipo Write e/o siano inviati all'esterno con un solo movimento di dati di tipo Exit. Il modello assume inoltre che tutte le manipolazioni dei dati, risultanti da tutti i possibili valori degli attributi del gruppo di dati oggetto del movimento, siano associate con quest'unico movimento di dati.

Per illustrare quest'ultima assunzione, consideriamo due occorrenze di un dato (tipo di) processo funzionale. Supponiamo che nella prima occorrenza i valori di alcuni attributi mossi portino a un (tipo di) sotto-processo di manipolazione dei dati "A" e che in un'altra occorrenza dello stesso processo funzionale i valori degli attributi portino a un differente (tipo di) sotto-processo di manipolazione dei dati "B". In tali circostanze, entrambi i sotto processi di manipolazione dei dati "A" e "B" dovrebbero essere associati allo stesso movimento di dati e quindi si dovrebbe normalmente identificare e contare quell'unico movimento di dati per quel processo funzionale.

¹³ Questo esempio vale quando si misura il software applicativo per l'uso da parte di utenti umani, a prescindere dal dominio. Non varrebbe ovviamente quando si misura la dimensione di oggetti riusabili, che supportino la visualizzazione di singole intestazioni di campo in schermate di input o di output.

¹⁴ Questa sezione era intitolata "De-duplicazione dei Movimenti di dati" nella precedente versione 2.2 del Manuale di Misurazione. Si è ritenuto il termine "de-duplicazione" di scarso aiuto, perciò la terminologia è stata variata.

Ci possono, tuttavia, essere circostanze *eccezionali* in cui si può richiedere (nei FUR) che *diversi* tipi di gruppi dati descriventi un dato oggetto d'interesse siano mossi con un movimento di dati del medesimo tipo (E, R, W, X) nello stesso processo funzionale. Alternativamente, e di nuovo in via eccezionale, si può richiedere che il medesimo gruppo di dati sia mosso con il medesimo tipo di movimento di dati (E, R, W o X) nello stesso processo funzionale, ma con differenti manipolazioni associate dei dati.

Le seguenti regole ed esempi coprono la normale situazione, possibili valide eccezioni ed esempi che potrebbero sembrare validi, ma che validi non sono.

REGOLE – Unicità dei movimenti di dati e possibili eccezioni	
a)	A meno che i Requisiti Funzionali Utente non specifichino altrimenti, tutti gli attributi dei dati descriventi un qualsiasi oggetto d'interesse richiesto in input in un processo funzionale, e tutte le manipolazioni associate, devono essere identificati e contati come un unico (tipo di) Entry. (NOTA. Un processo funzionale può, naturalmente, dover gestire molteplici (tipi di) Entry, ciascuno dei quali muove un gruppo di dati descrivente un diverso (tipo di) oggetto d'interesse.) La stessa regola si applica analogamente a qualsiasi movimento di dati di Read, Write o Exit in un qualsiasi processo funzionale.
b)	Si possono identificare e contare più di un (tipo di) movimento di dati di Entry, ciascuno dei quali muove un gruppo di dati descrivente lo stesso (tipo di) oggetto d'interesse in un certo processo funzionale, se vi è un Requisito Utente Funzionale per questi molteplici Entry. Analogamente, si possono identificare e contare più di un (tipo di) Entry, che muovono lo stesso (tipo di) gruppo di dati nello stesso processo funzionale, ma ciascuno con differenti (tipi di) manipolazioni associate di dati, se vi è un Requisito Utente Funzionale per questi molteplici Entry. Tali FUR possono presentarsi quando, in un unico processo funzionale, i molteplici Entry sono originati da utenti funzionali differenti, che inseriscono gruppi di dati differenti (ciascuno dei quali descrivente lo stesso oggetto d'interesse). La stessa regola si applica analogamente a qualsiasi movimento di dati di Read, Write o Exit in un qualsiasi processo funzionale.
c)	<i>Occorrenze</i> ripetute di un tipo di movimento di dati (per es. muovere lo stesso gruppo di dati con la stessa manipolazione dei dati) non devono essere identificate e contate più di una volta in un qualsiasi processo funzionale.
d)	Se le <i>occorrenze</i> ripetute di un tipo di movimento di dati in un determinato processo funzionale differiscono nel trattamento dei dati associato, in quanto differenti <i>valori</i> degli attributi dei dati del gruppo di dati mosso danno luogo a percorsi di elaborazione differenti, non si deve identificare e contare il tipo di movimento di dati più di una volta in quel processo.

I seguenti esempi illustrano le regole precedenti.

ESEMPIO 1, regola a). In un qualsiasi processo funzionale, tutti i Read di dati che descrivono uno specifico oggetto d'interesse possono essere considerati logicamente come il reperimento di tutti gli attributi che descrivono tale oggetto d'interesse (es. l'intero "vettore di stato" di quell'oggetto d'interesse). Quindi normalmente un solo Read dei dati su un qualsiasi oggetto d'interesse è funzionalmente necessario e dovrebbe essere identificato in ogni processo funzionale.

ESEMPIO 2, regole a) e b). Proseguendo con l'esempio 1, poiché qualsiasi dato che è letto deve essere stato reso permanente da un comando di Write, normalmente si dovrebbe identificare un Write che muove un gruppo di dati contenente tutti gli attributi dei dati di un oggetto d'interesse, che si richiede di rendere permanente in un dato processo funzionale, come da regola a). È possibile,

tuttavia che esista un FUR per un singolo processo funzionale che richiede di scrivere due differenti gruppi di dati che descrivono lo stesso oggetto d'interesse, per es. per usi successivi da parte di due diversi utenti funzionali in altri processi funzionali. Questo sarebbe un esempio che applica la regola b), quando due Write di dati che descrivono lo stesso oggetto d'interesse possono essere identificati e contati nello stesso processo funzionale.

ESEMPIO 3, regola b). È possibile che esista un FUR per un singolo processo funzionale che richiede di produrre due o più Exit che movimentano diversi gruppi di dati che descrivono lo stesso oggetto d'interesse, pensati per diversi utenti funzionali diversi. Per es. quando un nuovo impiegato entra in azienda, è prodotto un report da fare firmare all'impiegato a convalida dei suoi dati personali ed un messaggio è inviato alla Sicurezza per autorizzare l'impiegato all'ingresso nell'edificio.

ESEMPIO 4, regola c). Supponiamo che i FUR richiedano un Read che in pratica implica il reperimento di molte occorrenze, come la ricerca all'interno di un archivio. Ai fini del dimensionamento, si identifichi un solo Read.

ESEMPIO 5, regola c). Supponiamo che in un processo funzionale real-time, i FUR richiedano che i dati debbano essere inseriti da uno specifico utente funzionale, per es. da una periferica hardware, duplicato a un fissato intervallo di tempo al fine di misurare il tasso di variazione o per controllare se un valore è cambiato durante il processo. Poiché i due Entry sono identici in termini del gruppo di dati movimentato e del trattamento associato ai dati, solamente un Entry dovrebbe essere identificato. (V. sez. 4.1.6 per i tipi di manipolazione di dati che sono da considerare associati con un Entry.)

ESEMPIO 6, regola c). V. sez. 4.1.2, regola d), per gli Entry, e sez. 4.1.3, regola b), per gli Exit.

ESEMPIO 7, regola d). Supponiamo un processo funzionale a cui siano richiesti diversi trattamenti dei dati in funzione dei valori degli attributi dei dati di un Entry. Ai fini del dimensionamento, identificare un solo Entry.

ESEMPIO 8. Supponiamo che i FUR richiedano il Read di un gruppo di dati, ma che lo sviluppatore decida di implementarlo con due comandi per reperire diversi sottoinsiemi degli attributi dei dati del medesimo oggetto d'interesse dalla memoria permanente in punti diversi del processo funzionale. Identificare un solo Read.

4.1.8 Quando un processo funzionale muove dati da o verso la memoria permanente

Questa sezione spiega i movimenti di dati coinvolti quando un processo funzionale di una porzione di software applicativo richiede di muovere dati da o verso la memoria permanente, locale o remota. I casi mostrano inoltre come sono trattate le esigenze di memorizzazione dell'applicazione da parte di altri software che supportano l'applicazione in un altro strato, come per es. il software di driver della periferica di memoria permanente.

I casi proposti illustrano l'applicazione del principio g) del Modello di Contesto del Software e i principi del Modello Generale del Software. La chiave per l'interpretazione dei casi è che quei principi devono essere applicati separatamente ad ogni porzione di software che deve essere misurato.

Il primo caso riguarda i movimenti di dati di un'interrogazione in un'applicazione dove il requisito di reperire dati permanenti è gestito dal software di driver della periferica locale in un altro strato. Il secondo caso mostra come i movimenti di dati cambino, quando il requisito di reperire i dati è soddisfatto in primo luogo da una porzione di software alla pari nello stesso strato dell'applicazione.

In termini pratici, i casi sono applicabili quando l'attività è di misurare due porzioni di software che hanno sia una relazione gerarchica (es. quando due porzioni sono in strati differenti) oppure hanno una relazione client-server (es. quando due porzioni sono nello stesso strato). I casi mostrano come sono modellati i movimenti di dati che sono fisicamente scambiati tra le due porzioni di software oggetto di misurazione.

I casi sono illustrati usando le convenzioni dei Diagrammi di Sequenza dei Messaggi ("Message Sequence Diagrams"). La notazione di questi diagrammi è la seguente:

- Un freccia verticale in grassetto che punta in basso rappresenta un processo funzionale.
- Le frecce orizzontali rappresentano i movimenti di dati, etichettate E, X, R o W per Entry, Exit, Read e Write, rispettivamente. Entry e Read sono mostrate come frecce che entrano nel processo funzionale e Exit e Write come frecce uscenti; esse si presentano nella sequenza richiesta, dall'alto verso il basso, del processo funzionale.
- Una linea punteggiata verticale rappresenta un confine.

CASO 1. Quando un processo funzionale deve muovere dati da o verso la memoria permanente locale

Il caso 1 implica due porzioni di software, cioè una porzione di software applicativo "A" e una porzione separata di software "B" che è il driver di una periferica di memorizzazione fisica, utilizzata dal software applicativo. (Per semplicità ignoriamo la probabile presenza di un sistema operativo; il sistema operativo effettivamente trasmette le richieste dell'applicazione al software di driver della periferica e restituisce i risultati delle richieste.)

Il concetto di strato ci dice che due porzioni di software sono in strati diversi: lo strato applicativo e lo strato del driver della periferica. Fisicamente, c'è una relazione gerarchica tra le due porzioni e (ignorando il sistema operativo) un'interfaccia fisica tra i software nei due strati, come mostrato per es. in Fig. 2.2.3.1.

Normalmente, i FUR dell'applicazione "A" specificheranno dei processi funzionali che includono la necessità di vari Read e Write dalla/nella memoria permanente, ma non si preoccupano di come questi Read e Write saranno trattati dalle altre infrastrutture software.

Applicando i modelli COSMIC alle due porzioni di software, gli utenti funzionali del software A nello strato applicativo potrebbero essere per es. degli utenti umani, mentre l'utente funzionale del software B nello strato del driver è la porzione di software applicativo A (ignorando il sistema operativo).

Supponiamo che un processo funzionale di interrogazione "PF A" del software A nello strato applicativo richieda un movimento di dati di Read. La Fig. 4.1.8.1 (a) mostra il modello COSMIC dell'interrogazione dell'applicazione. Il reperimento fisico dei dati richiesti dalla memoria permanente è gestito da un processo funzionale "PF B" del software B nello strato del driver della periferica. La Fig. 4.1.8.1 (b) mostra il modello per questo processo funzionale del driver della periferica.

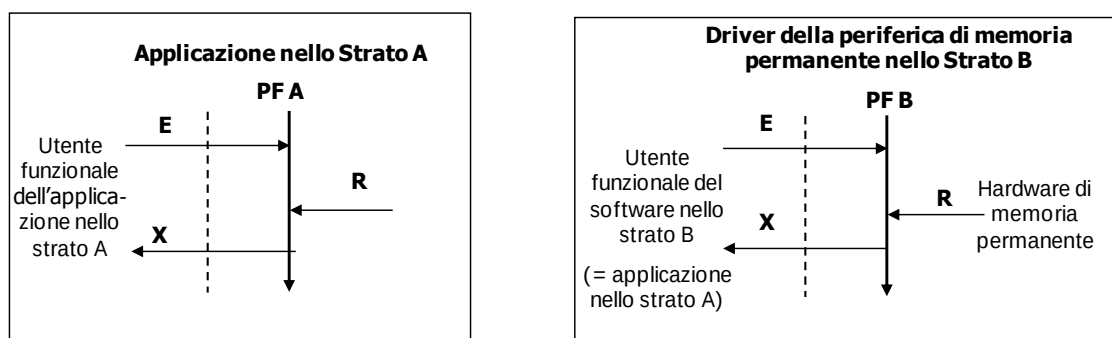


Figure 4.1.8.1 (a) e (b) – Soluzione per una richiesta di Read dal software "A" nello strato applicativo al software "B" nello strato del driver della periferica.

La Fig. 4.1.8.1 (a) mostra l'interrogazione innescata da un Entry, seguita da un Read e quindi da un Exit con il risultato dell'interrogazione. Il "PF A" non ha conoscenza di dove il dato è reperito, nè che in realtà il Read è delegato a qualche software di driver di periferica.

La Fig. 4.1.8.1 (b) mostra che funzionalmente la richiesta di Read dell'applicazione A è ricevuta come un trigger per l'Entry al processo funzionale "PF B", che quindi reperisce i dati richiesti dalla periferica fisica di memoria permanente e ritorna i dati all'applicazione come un Exit. L'applicazione A è dunque un utente funzionale del software del driver della periferica, B.

L'apparente disallineamento tra il numero dei movimenti di dati di "Read" del software nello strato applicativo e le coppie "Entry/Exit" del software nello strato del driver della periferica è dovuto al fatto che per convenzione, un movimento di dati di Read è considerato includere ogni funzionalità di „richiesta di read“.

Modelli esattamente analoghi dovrebbero applicarsi se al processo funzionale "PF A" fosse richiesto di rendere permanente qualche dato tramite un movimento di dati di Write. In questo caso, l'Exit del processo funzionale "PF B" del software del driver della periferica dovrebbe contenere il "return code" o il messaggio di errore.

CASO 2. Quando un processo funzionale deve ottenere dati da una porzione di software alla pari

In questo caso si assume che le due porzioni di software alla pari, oggetto di misurazione, abbiano una relazione "client/server", per es. dove una porzione, il "client", ottiene servizi e/o dati, da un'altra porzione, il "server". La Fig. 4.1.8.2 mostra un esempio di tale relazione, in cui le due porzioni di software sono le componenti principali (alla pari) della medesima applicazione. Si avrebbe la stessa relazione e varrebbe lo stesso diagramma anche se le due porzioni fossero applicazioni alla pari distinte, dove una necessitasse di ottenere dei dati dall'altra.

Fisicamente, le due componenti alla pari, potrebbero essere eseguite su due computer; in questo caso, in un architettura software come quella in Fig. 2.2.3., essi si scambiarebbero informazioni attraverso i rispettivi sistemi operativi e ogni altro strato software. Ma logicamente, applicando i modelli COSMIC, i due componenti scambiano dati attraverso "Entry/Exit alla pari". Tutti i software e gli hardware che intervengono, sono ignorati in questo modello (come mostrato nella parte destra della Fig. 3.1.1).

La Fig. 4.1.8.2 mostra che il processo funzionale "PF C1" della componente client "C1" è attivato da un Entry dal suo utente funzionale che comprende per es. i parametri della interrogazione. Il FUR della componente "C1" riconoscerà che questa componente deve interrogare la componente server "C2" per richiedere dati, e deve dirgli quali dati sono richiesti.

Per ottenere i dati richiesti quindi, "PF C1" invia un Exit alla componente "C2" contenente i parametri di richiesta della interrogazione. Componente C1 è ora un utente funzionale della componente "C2"; quindi esiste un confine tra le due componenti. Questo movimento di dati di Exit attraversa il confine tra "C1" e "C2" e diventa così l'Entry di innesco di un processo funzionale "PF C2" nella componente C2. Il processo funzionale C2 ottiene i dati richiesti attraverso un Read, e invia i dati indietro verso C1 tramite un Exit. Il processo funzionale "PF C1" della componente "C1" riceve questo movimento di dati come un Entry. Il "PF C1" quindi passa i dati attraverso un Exit per soddisfare l'interrogazione del suo utente funzionale. Questa interrogazione del Caso 2 richiede quindi 7 movimenti di dati per soddisfare la richiesta di interrogazione. Si confronti questo caso con i 3 movimenti di dati (1 x E, 1 x R e 1 x X) che sarebbero richiesti se la componente "C1" fosse stata capace di reperire i dati dalla memoria permanente "locale" come mostrato in Fig. 4.1.8.1 (a).

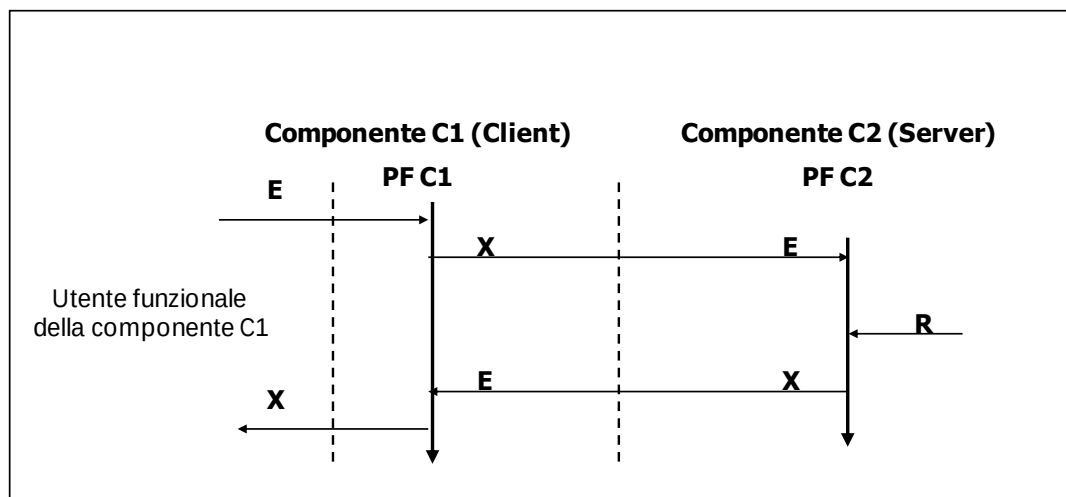


Figura 4.1.8.2 – Scambi di dati tra componenti alla pari nello stesso strato.

Naturalmente, la componente “C2” userà probabilmente i servizi di un software di driver della periferica di memoria permanente, in uno strato inferiore dell’architettura software, per reperire i dati dall’hardware, come mostrato nel Caso 1.

Confrontando i Casi 1 e 2, vediamo che nel Caso 1 i modelli dell’applicazione A e del driver di periferica B non possono combinarsi tra loro come nel caso 2. Questo è il motivo per cui un Read non attraversa il confine. La Fig. 4.1.8.1 (b) mostra che l’applicazione A è un utente funzionale del software di driver della periferica B. Ma l’inverso non è vero, quindi dimostrando la natura gerarchica del software nei differenti strati.

Per contro, la Fig. 4.1.8.2 può mostrare i due componenti in un modello perché essi scambiano dati “alla pari”. Componente C1 è un utente funzionale della componente C2, e viceversa, a essi condividono un confine in comune.

Si noti che in questi casi abbiamo ignorato, per semplicità, la possibilità di un “return code” che accompagni il movimento di dati Read, che potrebbe portare alla generazione di un messaggio di errore da contare come Exit in aggiunta all’Exit che contiene il risultato dell’interrogazione nell’applicazione A o nella componente C1.

4.1.9 Quando un processo funzionale richiede dati da un utente funzionale

In base al principio c) per l’Entry (v. sez. 4.1.2), se un processo funzionale deve ottenere dati da un utente funzionale ci sono due casi. Se il processo funzionale non necessita di dire all’utente funzionale quali dati inviare, un solo Entry è sufficiente (per oggetto d’interesse). Se il processo funzionale necessita di dire all’utente funzionale quali dati inviare, è necessaria una coppia Exit/Entry. Si applicano le seguenti regole:

REGOLE – Quando un processo funzionale richiede dati da un utente funzionale

- a) Un processo funzionale deve ottenere un gruppo di dati tramite un movimento di dati di Entry da parte di un utente funzionale, *quando non deve specificare all’utente funzionale quali dati inviare*, come in uno qualsiasi dei seguenti tre casi:
 - Quando un utente funzionale invia un Entry d’innesco che avvia il processo funzionale.
 - Quando un processo funzionale, essendo iniziato, è in attesa, aspettando il successivo Entry da parte dell’utente funzionale (tipicamente, nel software applicativo aziendale, da parte di un utente funzionale umano)

- Quando un processo funzionale, essendo iniziato, richiede all'utente funzionale "inviarmi i tuoi dati ora, se ne hai" e l'utente funzionale invia i dati (tipicamente noto come "polling" nel software real-time).

Nell'ultimo caso, per convenzione, non si deve identificare alcun Exit da parte del processo funzionale per ottenere i dati richiesti. Il processo funzionale deve semplicemente inviare un messaggio di sollecito ("prompt message") all'utente funzionale per inserire i propri dati e la funzionalità di tale messaggio di sollecito è considerata parte dell'Entry. L'utente funzionale sa cosa deve inviare e il processo funzionale sa cosa deve attendere. È necessario solo un Entry in questo caso.

V. anche principio c) per l'Entry, sez. 4.1.2.

- b) Laddove un processo funzionale necessita di ottenere i servizi di un utente funzionale (per es. per ottenere dei dati) e occorre specificare all'utente funzionale cosa inviare (tipicamente dove l'utente funzionale è un'altra porzione di software, esterna all'ambito del software oggetto di misurazione), si deve identificare una coppia di movimenti di dati Exit/Entry. L'Exit contiene la richiesta dei dati specifici; l'Entry contiene i dati di risposta.

ESEMPIO regola a), terzo punto. Supponiamo un processo funzionale di un software real-time di controllo di processo a cui è richiesto di verificare una serie di sensori "semplici" ("dumb") identici. Il processo funzionale emette un (tipo) Entry. (Poiché i sensori sono identici, si identifica e si conta un solo (tipo di) Entry anche se ci sono occorrenze multiple.)

NOTA. È interessante rielaborare questo esempio per ottenere dati da un Entry e confrontarlo con il Caso 1 di sez. 4.1.8, in cui un processo funzionale deve ottenere dati da un Read. Supponiamo che l'Entry debba essere passato in realtà a una porzione di software del driver di periferica in uno strato inferiore dell'architettura software, che ottiene fisicamente i dati richiesti dal sensore come illustrato in Fig. 2.2.3.2. I processi funzionali del software di controllo di processo e del software di driver di periferica per i sensori "semplici" dovrebbero risultare come mostrato in Fig. 4.1.9.1 (a) e (b) seguenti.

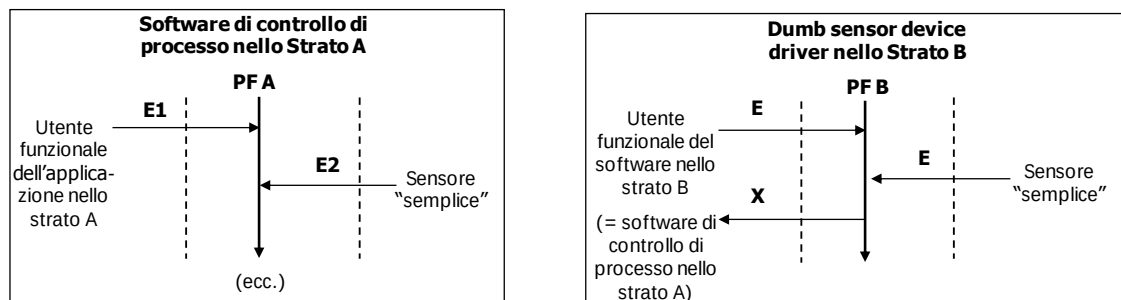


Figure 4.1.9.1 (a) e (b) – Soluzione per un Entry inviato dal software "A" nel processo di controllo nello strato applicativo al software "B" nello strato del driver di periferica del sensore "semplice".

La Fig. 4.1.9.1 (a) mostra che il processo funzionale del software di controllo di processo "PF A" è attivato da un Entry "E1", per es. da un evento a tempo. Il processo funzionale quindi ottiene l'Entry "E2" dalla matrice di sensori semplici per ricevere le occorrenze multiple delle letture dei sensori. I sensori semplici sono anche utenti funzionali del software di controllo del processo.

La Fig. 4.1.9.1 (b) mostra che il software che controlla le periferiche dei sensori semplici riceve l'Entry (probabilmente in pratica attraverso il sistema operativo) come l'innescò del processo funzionale "PF B". Questo processo funzionale ottiene un Entry dal suo (tipo di) utente funzionale, il (tipo di) sensore "semplice" per ottenere i dati dei sensori che sono dati in risposta al software di controllo di processo

come un Exit. Il processo funzionale del software di controllo di processo quindi continua con la sua elaborazione dei dati dei sensori. Il fatto che ci siano multiple occorrenze di questo ciclo di raccolta di dati da ognuno dei sensori identici è ignorato nel modello.

L'apparente disallineamento tra l'unico Entry del software di controllo del processo e la coppia Entry/Exit del software di driver di periferica è dovuto alla convenzione che si considera che un Entry includa ogni funzionalità della „richiesta di Entry“.

ESEMPIO regola b). Supponiamo che un processo funzionale invii a uno dei suoi utenti funzionali, come una periferica hardware “intelligente” o un'altra porzione di software alla pari, alcuni parametri per una interrogazione o parametri per un calcolo, o dati da essere compressi. La risposta dal utente funzionale è ottenuta attraverso una coppia Exit/Entry, come descritto in sez. 4.1.8, Caso 2.

4.1.10 Comandi di controllo

Un “comando di controllo” è una categoria speciale di movimento di dati che è riconosciuta solamente nel dominio degli applicativi aziendali e che deve essere ignorata nel misurare una dimensione funzionale. La definizione è:

DEFINIZIONE – Comando di controllo
Un comando di controllo è un comando che abilita un utente funzionale a controllare il suo utilizzo del software, ma che non comporta alcun movimento di dati riguardante un oggetto d'interesse.
NOTA. Il termine “comando di controllo” è usato SOLO nel contesto della misurazione del software applicativo aziendale. In tale contesto, un comando di controllo non è un movimento di dati perché il comando non muove dati riguardanti un oggetto d'interesse. Esempi sono i comandi “pagina su/giù”, la pressione dei tasti Tab o Invio, il click su “OK” per confermare un'azione precedente, ecc.

REGOLA – Comandi di controllo nel dominio degli applicativi aziendali
Nel dominio degli applicativi aziendali, i “comandi di controllo” devono essere ignorati poiché non comportano alcun movimento di dati riguardante un oggetto d'interesse.

Esempi di comandi di controllo nel dominio degli applicativi aziendali sono le funzioni che consentono ad un utente funzionale il controllo della visualizzazione (o meno) di una testata o di un sottotale che è stato calcolato, la navigazione in su e in giù tra le schermate fisiche, click “OK” per confermare il messaggio di errore o per confermare qualche dato digitato, ecc. I comandi di controllo includono quindi anche i comandi di menu che consentono all'utente funzionale di navigare tra uno o più specifici processi, ma che non innescano essi stessi alcun processo funzionale, e i comandi che presentano uno schermata vuota per inserire dei dati.

N.B. Al di fuori del dominio degli applicativi aziendali, il concetto di “comando di controllo” non ha particolare significato e ogni segnale o movimento di dati di un oggetto d'interesse proveniente da un utente funzionale deve essere contato, ossia deve essere misurato.

4.2 Applicare la funzione di misurazione

Questo passo consiste nell'applicare la funzione di misurazione COSMIC a ogni movimento di dati identificato in ogni processo funzionale.

DEFINIZIONE – Funzione di misurazione COSMIC

La funzione di misurazione COSMIC è una funzione matematica che assegna un valore al proprio argomento sulla base dello standard di misurazione COSMIC. L'argomento della funzione di misurazione COSMIC è il movimento di dati.

DEFINIZIONE – Standard di misurazione COSMIC

Lo standard di misurazione COSMIC, 1 CFP (COSMIC Function Point), è definito come la dimensione di un singolo movimento di dati.

In base a questa funzione di misurazione, a ogni istanza di movimento di dati (Entry, Exit, Read o Write), che si richiede di aggiungere, modificare o cancellare, e che sia stato identificata in base alla sez. 4.1, si assegna una dimensione numerica di 1 CFP.

4.3 Aggregare i risultati della misurazione

Questo passo consiste nell'aggregare in un unico valore di dimensione funzionale i risultati della funzione di misurazione, applicata a tutti i movimenti di dati identificati. Questo passo è svolto attenendosi alle seguenti regole.

4.3.1 Regole generali di aggregazione

REGOLE – Aggregare i risultati della misurazione

- a) Per qualsiasi processo funzionale, le dimensioni funzionali dei singoli movimenti di dati si devono aggregare in un unico valore di dimensione funzionale in unità di CFP sommandole aritmeticamente tra loro.

$$\text{Dimensione (processo funzionale}_i\text{)} = \sum \text{dim. (Entry}_i\text{)} + \sum \text{dim. (Exit}_i\text{)} + \sum \text{dim. (Read}_i\text{)} + \sum \text{dim. (Write}_i\text{)}$$

- b) Per qualsiasi processo funzionale, la dimensione funzionale delle modifiche ai suoi Requisiti Utente Funzionali si deve aggregare a partire dalle dimensioni dei movimenti di dati che sono stati aggiunti, modificati o cancellati nel processo funzionale, per fornire una dimensione della modifica in unità di CFP, secondo la seguente formula.

$$\text{Dimensione (Modifica (proc. funzionale}_i\text{))} = \sum \text{dim. (movimenti di dati aggiunti}_i\text{)} + \sum \text{dim. (movimenti di dati modificati}_i\text{)} + \sum \text{dim. (movimenti di dati cancellati}_i\text{)}$$

Per ulteriori dettagli sull'aggregazione delle dimensioni funzionali, v. sez. 4.3.2. Per la misurazione della dimensione del software modificato, v. sez. 4.4.

- c) La dimensione di una porzione di software in un dato ambito si deve ottenere aggregando le dimensioni di tutti processi funzionali appartenenti a quella porzione, nel rispetto delle regole e) e f) seguenti.
- d) La dimensione di una qualsiasi modifica di una porzione di software in un dato ambito si deve ottenere aggregando le dimensioni di tutte le modifiche di tutti i

processi funzionali appartenenti a quella porzione, nel rispetto delle regole e) e f) seguenti.

- e) Le dimensioni di porzioni di software o di modifiche di porzioni di software in un qualsiasi strato possono essere sommate tra loro solo se sono tutte misurate al medesimo livello di granularità dei processi funzionali dei rispettivi FUR.
- f) Le dimensioni di porzioni di software e/o di modifiche alle dimensioni di porzioni di software in un qualsiasi strato, o in differenti strati, devono essere sommate tra loro solo se ha senso farlo, alla luce dello scopo della misurazione.
- g) La dimensione di una porzione di software non può ottenersi sommando le dimensioni delle sue componenti (a prescindere dalla particolare scomposizione della porzione) *a meno che* non si eliminino i contributi alla dimensione dei movimenti di dati inter-componenti.
- h) Se il metodo COSMIC viene esteso localmente (per es. per misurare aspetti dimensionali non coperti dal metodo standard), la dimensione misurata tramite l'estensione locale deve allora essere riportata separatamente, come descritto in sez. 5.1 e NON può essere sommata alla dimensione ottenuta tramite il metodo standard, misurata in CFP (v. oltre, sez. 4.5).

ESEMPIO regole b) e c). Una modifica richiesta per una porzione di software potrebbe essere: "Aggiungere un nuovo processo funzionale di dimensione 6 CFP e, in un altro processo funzionale, aggiungere un movimento di dati, modificare altri tre movimenti di dati e cancellare due movimenti di dati." La dimensione totale della modifica richiesta è pari a $6 + 1 + 3 + 2 = 12$ CFP.

ESEMPIO regola f). Se varie parti significative di una porzione di software vengono sviluppate utilizzando tecnologie differenti, da differenti gruppi di progetto, potrebbe non esserci alcun valore pratico nel sommare tra loro le rispettive dimensioni.

ESEMPIO regola g). Se una porzione di software è:

- prima misurata come un tutt'uno, cioè in un unico ambito,
- successivamente misurata per ogni sua componente separatamente, cioè ognuna con un proprio ambito,

la dimensione totale ottenuta sommando le dimensioni di tutte le componenti separate (secondo caso) sarà superiore alla dimensione misurata come un tutt'uno (primo caso), per via del contributo alla dimensione di tutti i movimenti di dati inter-componenti. Questi movimenti di dati inter-componenti non sono visibili quando la porzione è misurata come un tutt'uno e devono perciò essere eliminati per ottenere la dimensione del "tutto". Si veda anche l'esempio relativo alla misurazione a vari livelli di granularità in architetture software pure nel documento "Advanced and Related Topics".

Si noti che, *nel contesto* di ogni strato identificato, la funzione di aggregazione è completamente scalabile. Perciò, si possono generare dei subtotali per singoli processi funzionali o per tutto il software in quello strato, in funzione dello scopo e dell'ambito di ciascuna esecuzione di misurazione e nel rispetto delle regole d), e) e f) di cui sopra.

4.3.2 Ulteriori informazioni sull'aggregazione delle dimensioni funzionali

In un contesto in cui si debba usare la dimensione funzionale come variabile in un modello, per es. per la stima dell'impegno, e il software oggetto di misurazione possieda più di uno strato o di una componente alla pari, l'aggregazione si svolgerà tipicamente per strato o per componente alla pari, poiché questi sono spesso implementati con tecnologie differenti.

Consideriamo per es. un software in cui lo strato di applicazione debba essere implementato utilizzando un linguaggio 3GL e un insieme di librerie esistenti, mentre uno strato di driver potrebbe essere implementato usando il linguaggio assembler. L'impegno unitario associato con la costruzione di ogni strato sarà molto probabilmente differente e, di conseguenza, si predisporrà una stima di impegno separate per ogni strato, sulla base della rispettiva dimensione.

D'altra parte, se un gruppo di progetto deve sviluppare un certo numero di porzioni principali di software ed è interessato alla propria produttività globale, esso può sommare tra loro le ore-persona richieste per sviluppare ogni porzione. Allo stesso modo, esso può sommare tra loro le dimensioni delle porzioni principali che ha sviluppato se (ma solo se) tale dimensioni soddisfano le regole sopra date.

Il motivo per cui le dimensioni di porzioni principali di software in diversi strati di un'architettura stratificata standard, misurate al medesimo livello di granularità dei processi funzionali, possono essere sommate tra loro è che tale architettura possiede un insieme coerentemente definito di utenti funzionali. Ogni strato è un utente funzionale degli strati "inferiori" che esso utilizza e ogni porzione di software in uno strato può essere un utente funzionale di qualsiasi altra porzione di software alla pari nel medesimo strato. I requisiti di una simile architettura impongono che i FUR delle varie porzioni debbano scambiare dei messaggi tra loro. È perciò logico e ragionevole che le dimensioni delle varie porzioni si possano sommare tra loro, sempre rispettando le regole d), e) e f) di cui sopra. Tuttavia, al contrario, la dimensione di una porzione principale di software *non* può ottenersi sommando tra loro le dimensioni dei suoi oggetti componenti riusabili, a meno che non si eliminino i movimenti di dati inter-oggetti, come richiesto dalla regola f) di cui sopra.

L'aggregazione dei risultati della misurazione per tipo di movimento di dati potrebbe essere utile per analizzare il contributo di ogni tipo alla dimensione totale di uno strato e potrebbe perciò essere di ausilio nel caratterizzare la natura funzionale dello strato misurato.

4.4 Ulteriori informazioni sulla dimensione delle modifiche del software

Una "modifica funzionale" di un software esistente è interpretata nel metodo COSMIC come "qualsiasi combinazione di aggiunta di nuovi movimenti di dati o di modifica o cancellazione di movimenti di dati esistenti". I termini "manutenzione evolutiva" ("enhancement") e "manutenzione" ("maintenance") sono spesso usati per indicare ciò che qui chiamiamo "modifica funzionale".

La necessità di una modifica del software può derivare da:

- un nuovo FUR (cioè solamente aggiunte alle funzionalità esistenti), o
- un cambiamento del FUR (includendo eventualmente aggiunte, modifiche e cancellazioni), o
- una necessità di "manutenzione" per correggere un difetto¹⁵.

Le regole per dimensionare una qualsiasi di queste modifiche sono le medesime, ma si avverte il misuratore di distinguere le varie circostanze quando svolge misurazioni e stime di performance.

Quando si sostituisce completamente una porzione di software, per es. riscrivendola, con o senza l'estensione o l'omissione di certe funzionalità, la dimensione funzionale della modifica è la dimensione del software di sostituzione, misurata secondo le normali regole per misurare un nuovo software. Questo caso non è ulteriormente considerato in questa sezione. Il misuratore dovrebbe tuttavia considerare, quando svolge misurazioni e stime di performance, la necessità di distinguere tra progetti per sviluppare software completamente nuovo e progetti per "ri-sviluppare" o "sostituire" software esistente.

Spesso, si elimina una parte obsoleta di un'applicazione ("si disconnette" sarebbe una descrizione migliore) lasciando il codice del programma al suo posto e rimuovendo semplicemente i contatti con le funzionalità obsolete. Se le funzionalità della parte obsolete ammontano a 100 CFP, ma tale parte può essere disconnessa modificando per es. 2 movimenti di dati, 100 e non 2 movimenti di dati dovrebbero identificarsi come la dimensione della modifica funzionale. Misuriamo la dimensione dei requisiti, non la dimensione che è stata implementata.¹⁶

¹⁵ Una normale convenzione della misurazione è che la dimensione funzionale di una porzione di software non cambia se il software deve essere cambiato per correggere un difetto in modo da allineare il software ai suoi FUR. La dimensione funzionale del software cambia se la modifica deve correggere un difetto nei FUR.

¹⁶ Si noti che per scopi di stima può essere opportuno utilizzare una produttività differente per questa parte di modifica funzionale, poichè la disconnessione è piuttosto differente da cancellazioni "reali". In alternativa, per scopi di stima può essere

Si noti la differenza tra la dimensione della modifica funzionale (qui discussa) e la variazione della dimensione funzionale del software. Solitamente, queste differiscono tra loro. La dimensione della seconda è trattata in sez. 4.4.3.

4.4.1 Modifica delle funzionalità

Qualsiasi movimento di dati di un certo tipo (E, X, R e W) comprende due tipi di funzionalità: muove un singolo gruppo di dati e possiede qualche manipolazione di dati associata (per quest'ultima, v. sez. 4.1.6). Quindi, per scopi di misurazione, un movimento di dati si considera essere funzionalmente modificato se:

- il gruppo di dati mosso, e/o
- la manipolazione di dati associata

sono modificati in qualsiasi modo.

Un gruppo di dati è modificato se:

- nuovi attributi sono aggiunti a tale gruppo di dati, e/o
- attributi esistenti sono rimossi dal gruppo di dati, e/o
- uno o più attributi esistenti sono modificati, per es. in significato o formato (ma non nei loro valori).

Una manipolazione di dati è modificata se è variata funzionalmente in qualsiasi modo, per es. variando calcoli, formattazione, presentazione e/o convalida specifica dei dati. "Presentazione" può significare per es. carattere, colore di sfondo, lunghezza del campo, numero di decimali, ecc.

Comandi di controllo e dati generali dell'applicazione non coinvolgono movimenti di dati, poiché non è mosso alcun dato di un oggetto d'interesse. Perciò, modifiche dei comandi di controllo e dei dati generali dell'applicazione non dovrebbero essere misurate. Per es. quando il colore della schermata è modificato per tutte le schermate, tale modifica non dovrebbe essere misurata. (V. sez. 4.1.10 per una spiegazione dei comandi di controllo e dei dati generali dell'applicazione.)

REGOLE – Modifica di un movimento di dati

- a) Se un movimento di dati deve essere modificato a causa di una variazione della manipolazione dei dati associate con il movimento dei dati e/o a causa di una variazione nel numero o nel tipo degli attributi del gruppo di dati mosso, si misura un CFP modificato, indipendentemente dal numero reale di modifiche nell'unico movimento di dati.
- b) Se deve essere modificato un gruppo di dati, i movimenti di dati che muovono il gruppo di dati modificato le cui funzionalità non sono impattate dalla modifica del gruppo di dati non sono identificati come movimenti di dati modificati.

NOTA 1. Una modifica di un valore di un'occorrenza di un attributo, come la modifica di un codice individuale di un attributo i cui valori costituiscono un sistema di codifica, non è una modifica del tipo dell'attributo.

NOTA 2. Una modifica di un qualsiasi dato che compare su schermate di input o output che non sia collegato a un oggetto d'interesse di un utente funzionale non è identificata come un CFP modificato (v. sez. 3.3.4 per esempi di tali dati).

ESEMPIO regole a) e b). Supponiamo un requisito per aggiungere o modificare gli attributi dei dati di un gruppo di dati D_1 , per cui dopo la modifica questo diventi D_2 . Nel processo funzionale "A" in cui si richiede tale modifica, tutti i movimenti di dati impattati dalla modifica dovrebbero essere identificati e contati come modificati. Perciò, per la regola a), se il gruppo di dati modificato D_2 è reso permanente e/o è oggetto di output nel processo funzionale A, si identificano rispettivamente un movimento di dati di Write e/o di Exit come modificati. Tuttavia, è possibile che altri processi funzionali leggano (Read) o

preferibile misurare la dimensione che sarà implementata (2 CFP nell'esempio) piuttosto che la dimensione dei requisiti (100 CFP nell'esempio). Se si misura la "dimensione progettuale" di 2 CFP, questa dovrebbe essere chiaramente documentata e distinta dalla misurazione dei FUR che richiedono che l'applicazione dovrebbe essere ridotta di dimensione di 100 CFP.

inseriscano (Entry) questo stesso gruppo di dati D_2 , ma le loro funzionalità non siano impattate dalla modifica perché non usano i particolari attributi dei dati modificati o aggiunti. Questi processi funzionali continuano a elaborare il gruppo di dati mosso come se fosse ancora D_1 . Perciò, per la regola b), i movimenti di dati degli altri processi funzionali che non sono impattati dalla modifica del/dei movimenti di dati del processo funzionale "A" NON devono essere identificati e contati come modificati.

4.4.2 Dimensione del software funzionalmente modificato

Dopo aver modificato funzionalmente una porzione di software, la sua nuova dimensione totale è uguale alla dimensione originale, più la dimensione funzionale di tutti i movimenti di dati aggiunti, meno la dimensione funzionale di tutti i movimenti di dati rimossi. I movimenti di dati modificati non hanno impatto sulla dimensione della porzione di software poiché esistono sia prima sia dopo che le modifiche sono state apportate.

4.5 Estendere il metodo di misurazione COSMIC

4.5.1 Introduzione

Il metodo di misurazione COSMIC non pretende di misurare tutti gli aspetti della "dimensione" del software. Perciò il metodo di misurazione COSMIC non è attualmente progettato per fornire un modo standard per prendere in considerazione la dimensione di certi tipi di Requisiti Utente Funzionali, in particolare algoritmi matematici complessi o sequenze complesse di regole come si possono trovare nei sistemi esperti. Inoltre, l'impatto del numero degli attributi dei dati per movimento di dati sulla dimensione del software non è colto dal metodo di misurazione. L'impatto sulla dimensione dei sotto-processi di manipolazione dei dati è preso in considerazione tramite un'ipotesi semplificatrice che è valida solo per certi domini software, come definito in sez. 2.1 sull'applicabilità del metodo.

Si potrebbe considerare che altri parametri come la "complessità" (comunque sia definita) contribuiscono alla dimensione funzionale. Un dibattito costruttivo su questo argomento richiederebbe in primo luogo delle definizioni comunemente accettate degli altri elementi nel contesto della nozione mal definita di "dimensione" per come si applica al software. Al momento attuale, tali definizioni sono ancora oggetto di ricerca e di dibattito.

Nondimeno, la misura della dimensione COSMIC è considerata una buona approssimazione per lo scopo dichiarato e il dominio di applicabilità del metodo. Pure, potrebbe aversi che, nel contesto locale di un'organizzazione che usi il metodo di misurazione COSMIC, si voglia tener conto di tali funzionalità in un modo che sia significativo come standard locale. Per questo motivo, il metodo di misurazione COSMIC prevede la predisposizione di estensioni locali. Qualora si faccia uso di tali estensioni locali, i risultati delle misurazioni devono essere presentati attenendosi alla particolare convenzione riportata in sez. 5.1.

Le seguenti sezioni illustrano come estendere il metodo con uno standard locale.

4.5.2 Estensione locale con algoritmi complessi

Se si ritiene necessario prendere in considerazione algoritmi complessi, si può impostare uno standard locale per tali funzionalità eccezionali. In qualsiasi processo funzionale in cui vi sia un sotto-processo funzionale di manipolazione dei dati insolitamente complesso, il misuratore è libero di assegnare i propri Function Point localmente determinati. Un esempio di standard di estensione locale potrebbe essere:

"Nella nostra organizzazione, si assegna un FP Locale agli algoritmi matematici quali [lista di esempi localmente significativi e ben compresi]. Si assegnano due FP Locali a [altra lista di esempi], ecc."

4.5.3 Estensione locale con sotto-unità di misura

Qualora si richieda una maggiore precisione nella misurazione dei movimenti di dati, si può definire una sotto-unità di misura. Per es. un metro può essere suddiviso in 100 centimetri o 1000 millimetri. Per analogia, si potrebbe usare il movimento di un singolo attributo dei dati come sotto-unità di misura. Misurazioni svolte su un piccolo campione di software durante le prove sul campo del metodo COSMIC hanno mostrato che, sul campione misurato, il numero medio di attributi dei dati per movimento di dati non varia di molto tra i quattro tipi di movimenti di dati. Per questo motivo e per

ragioni di semplicità di misurazione, l'unità di misura COSMIC, 1 CFP, è stata fissata al livello del singolo movimento di dati. Tuttavia, occorre chiaramente cautela nel confrontare le dimensioni misurate in CFP di due differenti porzioni di software, laddove il numero medio di attributi dei dati per movimento di dati sia drasticamente differente tra le due porzioni di software.

Chiunque desideri perfezionare il metodo COSMIC introducendo una sotto-unità di misura è libero di farlo, ma dovrà esplicitare chiaramente che le misure di dimensione risultanti non sono espresse in Function Point COSMIC standard.

PRESENTAZIONE DELLE MISURAZIONI

Il Modello Generale del Software può essere rappresentato sotto forma di matrice, laddove le righe rappresentano i processi funzionali (eventualmente raggruppati per strati) e le colonne i gruppi di dati, e le celle contengono i sotto-processi identificati (Entry, Exit, Read e Write). Tale rappresentazione del Modello Generale del Software è illustrata in Appendice A.

I risultati delle misurazioni COSMIC devono essere registrati e archiviati attenendosi alle seguenti convenzioni.

5.1 Etichettatura

Quando si presenta una dimensione funzionale COSMIC, la si dovrebbe etichettare attenendosi alla seguente convenzione, in conformità allo standard ISO/IEC 14143-1:2007.

REGOLA – Etichettatura di una misurazione COSMIC

Un risultato di misurazione COSMIC è riportato come “**x** CFP (v.**y**)”, dove:

- “**x**” rappresenta il valore numerico della dimensione funzionale,
- “v.**y**” rappresenta l’identificativo della versione standard del metodo COSMIC usata per ottenere il valore numerico della dimensione funzionale „**x**”.

ESEMPIO. Un risultato ottenuto con le regole del presente Manuale di Misurazione è indicato come “**x** CFP (v3.0)”.

NOTA. Se si è fatto uso di un metodo di approssimazione locale per ottenere la misurazione, ma, a parte ciò, la misurazione è stata effettuata tramite le convenzioni di una versione standard del metodo COSMIC, si usi la precedente convenzione di etichettatura, ma si annoti altrove l’utilizzo del metodo di approssimazione – v. sez. 5.2.

Qualora si sia fatto uso di estensioni locali, come definito nella precedente sez. 4.5, il risultato della misurazione deve essere presentato come definito di seguito.

REGOLA – Etichettatura di estensioni locali COSMIC

Un risultato di misurazione COSMIC ottenuto mediante estensioni locali deve essere riportato come:

“**x** CFP (v. **y**) + z Local FP”, dove:

- “**x**” rappresenta il valore numerico ottenuto aggregando tutti i singoli risultati di misurazione conformi al metodo COSMIC standard, versione v.y,
- “v.**y**” rappresenta l’identificativo della versione standard del metodo COSMIC usata per ottenere il valore numerico della dimensione funzionale „**x**”,
- “**z**” rappresenta il valore numerico ottenuto aggregando tutti i singoli risultati di misurazione ottenuti mediante estensioni locali del metodo COSMIC.

5.2 Archiviazione

Quando si archiviano i risultati di misurazione COSMIC, si dovrebbero mantenere le seguenti informazioni, per garantire che il risultato sia sempre interpretabile.

REGOLA – Presentazione delle misurazioni COSMIC

In aggiunta alle misurazioni effettive, registrate come indicato in sez. 5.1, per ogni misurazione si dovrebbero registrare i seguenti attributi.

- a) Identificativo della componente software misurata (nome, ID di versione o ID di configurazione).
- b) Le fonti informative usate per identificare i FUR usati per la misurazione.
- c) Il dominio del software.
- d) Lo scopo della misurazione.
- e) Una descrizione dell'ambito della misurazione e la sua relazione con l'ambito complessivo di un gruppo relazionato di misurazioni, qualora ve ne siano (usare le categorie di ambito generiche di sez. 2.2).
- f) Gli utenti funzionali del software.
- g) Il livello di granularità dei FUR e il livello di scomposizione del software.
- h) Il momento nel ciclo di vita del progetto in cui è stata effettuata la misurazione (specialmente se la misurazione è una stima basata su FUR incompleti, o è stata effettuata sulla base di funzionalità effettivamente rilasciate).
- i) Il margine di errore, ricercato o valutato, della misurazione.
- j) Indicazioni se si è applicato il metodo di misurazione COSMIC standard e/o un'approssimazione locale del metodo standard e/o estensioni locali (v. sez. 4.5). Si usino le convenzioni di etichettatura di sez. 5.1 o sez. 5.2.
- k) Un'indicazione se la misurazione è relativa a funzionalità sviluppate o consegnate (le funzionalità "sviluppate" sono ottenute creando nuovo software; le funzionalità "consegnate" includono le funzionalità "sviluppate" e anche le funzionalità ottenute con altri mezzi rispetto alla creazione di nuovo software, cioè incluse tutte le forme di riuso di software esistente, l'uso di parametri esistenti per aggiungere o modificare funzionalità, ecc.).
- l) Un'indicazione se la misurazione è di funzionalità fornite *ex novo* o è il risultato di un'attività "evolutiva" (per es. la somma delle funzionalità aggiunte, modificate e cancellate – v. sez. 4.4).
- m) Una descrizione dell'architettura degli strati in cui è svolta la misurazione, se applicabile.
- n) Il numero di componenti principali, se applicabile, le cui dimensioni sono state sommate tra loro per fornire la dimensione totale registrata.
- o) Per ogni ambito compreso nell'ambito complessivo della misurazione, una matrice di misurazione, come specificato in App. A.
- p) Il nome del misuratore e tutte le eventuali qualifiche di certificazione COSMIC.

Appendice A

APPENDICE A – DOCUMENTARE UNA MISURAZIONE DI DIMENSIONE COSMIC

La seguente matrice può essere usata come documento di raccolta per registrare i risultati di una misurazione per ciascuna delle componenti nell'ambito generale mappato sul Modello Generale del Software oggetto di misurazione. Andrà redatta una matrice per ciascun ambito relativo all'ambito complessivo della misurazione.

		GRUPPI DI DATI										
STRATI	PROCESSI FUNZIONALI	Gruppo di Dati 1	:	:	:	:	:	Gruppo di Dati n	ENTRY (E)	EXIT (X)	READ (R)	WRITE (W)
STRATO "A"												
	Processo funzionale a											
	Processo funzionale b											
	Processo funzionale c											
	Processo funzionale d											
	Processo funzionale e											
		TOTALE – Strato A										
STRATO "B"												
	Processo funzionale f											
	Processo funzionale g											
	Processo funzionale h											
		TOTALE – Strato B										

Figura A.1 – Matrice del Modello Generale del Software.

FASE DI MAPPATURA

- Ogni gruppo di dati identificato è registrato in una data colonna.
- Ogni processo funzionale è registrato in una data riga, raggruppato per strato identificato.

FASE DI MISURAZIONE

- Per ogni processo funzionale identificato, i movimenti di dati identificati sono riportati nella casella corrispondente con la seguente convenzione: "E" per Entry, "X" per Exit, "R" per Read e "W" per Write.
- Per ogni processo funzionale identificato, i movimenti di dati sono quindi sommati per tipo e ogni totale è registrato nella colonna appropriata all'estrema destra della matrice.
- Si può quindi calcolare e registrare il riepilogo della misurazione per ogni strato, nelle caselle con bordi, sulla riga "Totale".

Appendice B

APPENDICE B – SOMMARIO DEI PRINCIPI DEL METODO COSMIC

La seguente tabella enumera tutti i principi del metodo di misurazione COSMIC per consentire riferimenti precisi.

ID	NOME	DESCRIZIONE PRINCIPI
P-01	Modello del Contesto del Software COSMIC	a) Il software è delimitato dall'hardware. b) Il software è tipicamente strutturato in strati. c) Uno strato può contenere una o più porzioni di software "alla pari" separate e ogni porzione di software può ulteriormente consistere di componenti alla pari separate. d) Ogni porzione di software oggetto di misurazione è individuata dal proprio ambito di misurazione, che deve essere confinato interamente in un singolo strato. e) L'ambito di una porzione di software oggetto di misurazione dipende dallo scopo della misurazione. f) Gli utenti funzionali di una porzione di software sono identificati dai requisiti utente funzionali della porzione di software oggetto di misurazione come i mittenti e/o destinatari designati dei dati. g) Una porzione di software interagisce con i propri utenti funzionali tramite movimenti di dati attraverso un confine e può muovere i dati da e verso la memoria permanente all'interno del confine. h) I FUR del software possono essere espressi a differenti livelli di granularità. i) Il livello di granularità al quale si dovrebbero effettuare normalmente le misurazioni è quello dei processi funzionali. j) Se non è possibile misurare al livello di granularità dei processi funzionali, i FUR del software dovrebbero allora essere misurati con un approccio di approssimazione e risalati al livello di granularità dei processi funzionali.
P-02	Modello Generale del Software COSMIC	a) Il software riceve dati di input dai propri utenti funzionali e produce un output e/o un altro risultato per gli utenti funzionali. b) I requisiti utente funzionali di una porzione di software oggetto di misurazione possono essere mappati su processi funzionali unici. c) Ogni processo funzionale consiste di sotto-processi. d) Un sotto-processo può essere o un movimento di dati o una manipolazione di dati. e) Ogni processo funzionale è innescato da un movimento di dati di Entry da parte di un utente funzionale che informa il processo funzionale che l'utente funzionale ha identificato un evento. f) Un movimento di dati muove un singolo gruppo di dati. g) Un gruppo di dati consiste di un insieme unico di attributi dei dati che descrivono un singolo oggetto d'interesse. h) Esistono quattro tipi di movimenti di dati. Un Entry muove un gruppo di dati all'interno del software da parte di un utente funzionale. Un Exit muove un gruppo di dati fuori del software verso un utente funzionale. Un Write muove un gruppo di dati dal software verso la memoria permanente. Un Read muove un gruppo di dati dalla memoria permanente verso il software. i) Un processo funzionale include almeno un movimento di dati di Entry e un movimento di dati o di Write o di Exit, owerosia include un minimo di due movimenti di dati. j) Come approssimazione per scopi di misurazione, i sotto-processi di manipolazione dei dati non sono misurati separatamente; la/e funzionalità di una qualsiasi manipolazione di dati si assumono come incluse nel

ID	NOME	DESCRIZIONE PRINCIPI
		movimento di dati a cui essa è associata.
P-03	Principio di Misurazione COSMIC	La dimensione funzionale di una porzione di software è direttamente proporzionale al numero dei suoi movimenti di dati.
P-04	Strato	a) Il software in uno strato scambia dati con il software in un altro strato attraverso i rispettivi processi funzionali. b) La "dipendenza gerarchica" tra strati è tale che il software in ogni strato può usare i servizi funzionali di qualsiasi software in qualsiasi strato sottostante nella gerarchia. Laddove esistano tali relazioni d'uso, si designa lo strato software che usa l'altro come lo strato "superiore" e ogni strato che contiene il software usato come il suo strato "subordinato". Il software nello strato superiore poggia sui servizi del software contenuti in tali strati subordinati per funzionare correttamente; questi ultimi poggiano a loro volta sul software negli strati a loro subordinati per funzionare correttamente, e così via lungo la gerarchia. Al contrario, il software in uno strato subordinato, insieme con il software in uno qualsiasi degli strati subordinati da cui dipende, può funzionare senza necessità dei servizi del software contenuto in un qualsiasi strato superiore nella gerarchia. c) Il software in uno strato non usa necessariamente tutti i servizi funzionali forniti dal software in uno strato subordinato. d) I dati che sono scambiati tra i software in due strati qualsiasi sono definiti ed interpretati differentemente dai rispettivi FUR delle due porzioni di software, ossia le due porzioni di software riconoscono attributi dei dati differenti e/o sotto-gruppi differenti dei dati che scambiano. Tuttavia, devono anche esistere uno o più attributi o sotto-gruppi di dati definiti in comune per consentire al software nello strato ricevente di interpretare i dati che gli vengono passati dal software nello strato mittente, in funzione delle esigenze del software ricevente.
P-05	Componente alla pari	a) In un insieme di componenti alla pari di una porzione di software in uno strato non esiste dipendenza gerarchica tra le componenti alla pari come avviene tra strati. I FUR di tutte le componenti alla pari di una porzione di software in un qualsiasi strato sono allo stesso "livello" nella gerarchia degli strati. b) Tutte le componenti alla pari di una porzione di software devono cooperare mutualmente affinché la porzione di software possa funzionare correttamente. c) I FUR di componenti alla pari in un qualsiasi strato che si passano o condividono dati in comune, definiscono tali dati in modo identico, cioè riconoscono gli stessi attributi dei dati e sotto-raggruppamenti dei dati che si passano o condividono.
P-06	Applicare il Modello Generale del Software COSMIC	Il Modello Generale del Software COSMIC deve essere applicato ai requisiti utente funzionali di ciascuna porzione distinta di software per la quale è stato definito un distinto ambito di misurazione. "Applicare il Modello Generale del Software COSMIC" significa identificare l'insieme degli eventi d'innescio percepiti da ognuno dei (tipi di) utenti funzionali identificati nei FUR e successivamente identificare i corrispondenti processi funzionali, oggetti di interesse, gruppi di dati e movimenti di dati che devono essere forniti per rispondere a tali eventi.
P-07	Gruppo di dati	a) Ogni gruppo di dati identificato deve essere unico e distinguibile tramite la sua raccolta di attributi dei dati unici. b) Ogni gruppo di dati deve essere direttamente collegato a un oggetto d'interesse nei Requisiti Funzionali Utente del software. c) Un gruppo di dati deve essere materializzato nel sistema informatico che supporta il software.
P-08	Entry (E)	a) Un Entry deve muovere un unico gruppo di dati, descrivente un unico oggetto d'interesse, da un utente funzionale attraverso il confine e nel

ID	NOME	DESCRIZIONE PRINCIPI
		processo funzionale di cui fa parte l'Entry. Se l'input di un processo funzionale comprende più di un gruppo di dati, si identifichi un Entry per ciascun gruppo di dati distinto in input. (V. anche sez. 4.1.7 sulla "Unicità dei movimenti di dati".) b) Un Entry non deve eseguire output di dati attraverso il confine, né leggere o scrivere dati. c) Un movimento di dati di Entry deve includere qualsiasi funzionalità di "richiesta di inserimento", eccetto dove si debba specificare esplicitamente all'utente funzionale quali dati sono richiesti, nel qual caso si richiede una coppia Exit/Entry per ottenere ogni gruppo di dati. (V. anche sez. 4.1.9.)
P-09	Exit (X)	a) Un Exit deve muovere un unico gruppo di dati, descrivente un unico oggetto d'interesse, dal processo funzionale di cui fa parte l'Exit attraverso il confine verso un utente funzionale. Se l'output di un processo funzionale comprende più di un gruppo di dati, si identifichi un Exit per ciascun gruppo di dati distinto in output. (V. anche sez. 4.1.7 sulla "Unicità dei movimenti di dati".) b) Un Exit non deve eseguire input di dati attraverso il confine, né leggere o scrivere dati.
P-10	Read (R)	a) Un Read deve muovere un unico gruppo di dati, descrivente un unico oggetto d'interesse, dalla memoria permanente al processo funzionale di cui fa parte il Read. Se il processo funzionale deve reperire più di un gruppo di dati dalla memoria permanente, si identifichi un Read per ogni gruppo di dati distinto reperito. (V. anche sez. 4.1.7 sull'"Unicità dei movimenti di dati".) b) Un Read non deve ricevere o inviare dati attraverso il confine, né scrivere dati. c) Durante un processo funzionale, il movimento o la manipolazione di costanti o variabili che sono interne al processo funzionale e che possono essere cambiate solo da un programmatore, l'elaborazione di risultati intermedi in un calcolo, o di dati memorizzati da un processo funzionale risultanti solo dall'implementazione, invece che dai FUR, non devono essere considerati come movimenti di dati di Read. d) Un movimento di dati di Read deve includere sempre qualsiasi funzionalità di "richiesta di lettura" (quindi non si deve mai contare un movimento di dati distinto per una qualsiasi funzionalità di "richiesta di lettura"). (V. anche sez. 4.1.9.)
P-11	Write (W)	a) Un Write deve muovere un unico gruppo di dati, descrivente un unico oggetto d'interesse, dal processo funzionale di cui fa parte il Write verso la memoria permanente. Se il processo funzionale deve muovere più di un gruppo di dati verso la memoria permanente, si identifichi un Write per ogni gruppo di dati distinto mosso verso la memoria permanente. (V. anche sez. 4.1.7 sull'"Unicità dei movimenti di dati".) b) Un Write non deve ricevere o inviare dati attraverso il confine, né leggere dati. c) Un requisito di cancellare un gruppo di dati dalla memoria permanente deve essere misurato come un singolo movimento di dati di Write. d) Durante un processo funzionale, il movimento o la manipolazione di dati che non permangono quando il processo funzionale è completato, l'aggiornamento di variabili interne al processo funzionale o la produzione di risultati intermedi in un calcolo non devono essere considerati come movimenti di dati di Write.
P-12	Manipolazione dei dati associata ai movimenti di dati	Tutte le manipolazioni dei dati in un processo funzionale devono essere associate ai quattro tipi di movimenti di dati (E, X, R e W). Per convenzione, si assume che i movimenti di dati di un processo funzionale rappresentino anche la manipolazione dei dati del processo funzionale.

APPENDICE C – SOMMARIO DELLE REGOLE DEL METODO COSMIC

La seguente tabella enumera tutte le regole del metodo di misurazione COSMIC per consentire riferimenti precisi.

ID	NOME	DESCRIZIONE REGOLE
R-01	Ambito	a) L'ambito di una Misurazione della Dimensione Funzionale (Functional Size Measurement, FSM) deve derivare dallo scopo della misurazione. b) L'ambito di una qualsiasi misurazione non deve estendersi oltre uno strato del software oggetto di misurazione.
R-02	Strato	a) Se il software è concepito usando un'architettura a strati stabilita, come si intende in questa sede, allora si dovrebbe usare tale architettura per identificare gli strati per scopi di misurazione. b) Nel dominio del software gestionale (MIS) o aziendale, lo strato "più alto", cioè lo strato che non è subordinato a nessun altro strato, è normalmente definito come lo strato "dell'applicazione". Il software (applicativo) contenuto in tale strato poggia fondamentalmente sui servizi dei software in tutti gli altri strati sottostanti per funzionare correttamente. Nel dominio del software real-time, il software nello "strato più alto" è normalmente definito come il "sistema", come per es. in "software di sistema di controllo del processo", "software di sistema di controllo di volo". c) Non si assuma che un software qualsiasi, evoluto senza alcuna considerazione di progettazione architetturale o alcuna strutturazione, possa essere partizionato in strati secondo il modello COSMIC.
R-03	Utenti funzionali	a) Gli utenti funzionali di una porzione di software oggetto di misurazione derivano dallo scopo della misurazione. b) Quando lo scopo della misurazione di una porzione di software è collegato all'impegno per sviluppare o modificare la porzione di software, allora gli utenti funzionali dovrebbero essere coloro ai quali devono essere fornite le funzionalità nuove o modificate.
R-04	Confine	a) Identifica l'utente/i funzionali che interagiscono con il software misurato. Il confine si colloca tra gli utenti funzionali e tale software. b) Per definizione, esiste un confine tra ogni coppia identificata di strati, dove il software di uno strato è l'utente funzionale del software dell'altro, e quest'ultimo è l'oggetto della misurazione. Analogamente, esiste un confine tra due qualsiasi componenti alla pari del medesimo strato; in tal caso, ogni componente può essere un utente funzionale della componente sua pari.
R-05	Livello di granularità dei processi funzionali	a) La misurazione della dimensione funzionale dovrebbe essere effettuata al livello di granularità dei processi funzionali. b) Laddove sia richiesta la misurazione della dimensione funzionale di alcuni FUR che non sono ancora evoluti al livello al quale tutti i processi funzionali siano stati identificati e tutti i dettagli sui loro movimenti di dati definiti, si dovrebbero effettuare delle misurazioni delle funzionalità fin lì definite, e si dovrebbero poi riscalarle al livello di granularità dei processi funzionali. (V. il documento "Advanced & Related Topics" per metodi di "dimensionamento approssimato", cioè di stima di una dimensione funzionale nelle prime fasi del processo di determinazione dei FUR.)
R-06	Processo funzionale	a) Un processo funzionale deve essere derivato da almeno un Requisito Utente Funzionale identificabile nell'ambito prestabilito. b) Un (tipo di) processo funzionale deve essere eseguito quando si verifica un (tipo di) evento d'innescio identificabile. c) Uno specifico (tipo di) evento può innescare uno o più (tipi di) processi

ID	NOME	DESCRIZIONE REGOLE
		funzionali che si svolgono in parallelo. Uno specifico (tipo di) processo funzionale può essere innescato da più di un (tipo di) evento. d) Un processo funzionale deve comprendere almeno due movimenti di dati, un Entry più o un Exit o un Write. e) Un processo funzionale deve appartenere interamente al software di uno ed un solo strato. f) Nel contesto del software real-time, un processo funzionale deve considerarsi terminato quando raggiunge uno stato di attesa auto-indotto (cioè il processo funzionale ha svolto tutto quanto gli è richiesto di fare in risposta all'evento d'innescamento e attende finché non riceve il successivo Entry di innescamento). g) Si deve identificare un solo (tipo di) processo funzionale anche se i suoi FUR consentono che esso possa svolgersi con differenti sottoinsiemi del suo numero massimo di attributi dei dati di input, e anche se tali variazioni e/o differenti valori dei dati di input possono dar luogo a differenti percorsi di elaborazione nel corso del processo funzionale. h) Si dovrebbero distinguere (tipi di) eventi distinti e quindi (tipi di) processi funzionali distinti nei seguenti casi: - Quando delle decisioni hanno come risultato eventi distinti che sono separati nel tempo (per es. inserire i dati di un ordine oggi e confermare successivamente l'accettazione dell'ordine, con la necessità di una decisione separata, dovrebbe essere considerato indice di due processi funzionali distinti). - Quando le responsabilità delle attività sono distinte (per es. in un sistema del personale, in cui la responsabilità della gestione dei dati anagrafici di base è distinta da quella della gestione dei dati degli stipendi, indice di due processi funzionali distinti; oppure per un pacchetto software implementato in cui esiste una funzionalità, accessibile all'amministratore di sistema, per la gestione dei parametri del pacchetto, che è separata dalle funzionalità accessibili all'utente funzionale „normale“.)
R-07	Entry (E)	a) Il gruppo di dati di un Entry d'innescamento può consistere di solo un attributo che informa semplicemente il software che "un evento Y è avvenuto". Molto spesso, specialmente nel software applicativo aziendale, il gruppo dati dell'Entry d'innescamento ha vari attributi che informano il software che "un evento Y è avvenuto e questi sono i dati riguardanti questo particolare evento". b) Segnali periodici di clock che sono eventi d'innescamento devono essere sempre esterni al software oggetto di misurazione. Quindi per es. un evento periodico di clock che avviene ogni 3 secondi deve essere associato a un Entry che muove un gruppo dati con un solo attributo. Si noti che non fa alcuna differenza se l'evento d'innescamento è generato periodicamente dall'hardware o da un'altra porzione di software esterna al confine del software oggetto di misurazione. c) A meno che non sia necessario uno specifico processo funzionale, ottenere il valore di tempo dal clock di sistema non deve essere considerato scatenare un Entry. d) Se un'occorrenza di uno specifico evento innescamento l'Entry di un gruppo di dati contenente fino a "n" attributi di un particolare oggetto d'interesse e i FUR consentono che altre occorrenze dello stesso evento possano innescare un Entry di un gruppo di dati che ha valori per solo un sottoinsieme degli "n" attributi dell'oggetto d'interesse, allora si deve identificare un solo Entry, comprendente tutti gli "n" attributi dei dati.
R-08	Exit (X)	a) Tutti i messaggi generati e emessi dal software privi di dati utente (per es. messaggi d'errore) devono essere considerati come valori di un solo attributo di un unico oggetto d'interesse (che potrebbe essere chiamato "indice d'errore"). Quindi, si deve identificare un unico Exit per rappresentare

ID	NOME	DESCRIZIONE REGOLE
		tutte queste occorrenze di messaggi all'interno di ogni processo funzionale in cui siano richiesti dai FUR. b) Se un Exit di un processo funzionale muove un gruppo di dati contenente fino a "n" attributi di un particolare oggetto d'interesse e i FUR consentono il processo funzionale un'occorrenza di Exit che muove un gruppo di dati che ha valori per solo un sottoinsieme degli "n" attributi dell'oggetto d'interesse, allora si deve identificare un solo Exit, comprendente tutti gli "n" attributi dei dati.
R-09	Unicità dei movimenti di dati e possibili eccezioni	a) A meno che i Requisiti Funzionali Utente non specifichino altrimenti, tutti gli attributi dei dati descriventi un qualsiasi oggetto d'interesse richiesto in input in un processo funzionale, e tutte le manipolazioni associate, devono essere identificati e contati come un unico (tipo di) Entry. (NOTA. Un processo funzionale può, naturalmente, dover gestire molteplici (tipi di) Entry, ciascuno dei quali muove un gruppo di dati descrivente un diverso (tipo di) oggetto d'interesse.) La stessa regola equivalente si applica a qualsiasi movimento di dati di Read, Write o Exit in un qualsiasi processo funzionale. b) Si possono identificare e contare più di un (tipo di) movimento di dati di Entry, ciascuno dei quali muove un gruppo di dati descrivente lo stesso (tipo di) oggetto d'interesse in un certo processo funzionale, se vi è un Requisito Utente Funzionale per questi molteplici Entry. Analogamente, si possono identificare e contare più di un (tipo di) Entry, che muovono lo stesso (tipo di) gruppo di dati nello stesso processo funzionale, ma ciascuno con differenti (tipi di) manipolazioni associate di dati, se vi è un Requisito Utente Funzionale per questi molteplici Entry. Tali FUR possono presentarsi quando, in un unico processo funzionale, i molteplici Entry sono originati da utenti funzionali differenti, che inseriscono gruppi di dati differenti (ciascuno dei quali descrivente lo stesso oggetto d'interesse). La stessa regola equivalente si applica a qualsiasi movimento di dati di Read, Write o Exit in un qualsiasi processo funzionale. c) Occorrenze ripetute di un tipo di movimento di dati (per es. muovere lo stesso gruppo di dati con la stessa manipolazione dei dati) non devono essere identificate e contate più di una volta in un qualsiasi processo funzionale. d) Se le occorrenze ripetute di un tipo di movimento di dati in un dato processo funzionale differiscono nel trattamento dei dati associato, in quanto differenti valori degli attributi dei dati del gruppo di dati mosso danno luogo a percorsi di elaborazione differenti, non si deve identificare e contare il tipo di movimento di dati più di una volta in quel processo.
R-10	Quando un processo funzionale richiede dati da un utente funzionale	a) Un processo funzionale deve ottenere un gruppo di dati tramite un movimento di dati di Entry da parte di un utente funzionale, quando non deve specificare all'utente funzionale quali dati inviare, come in uno qualsiasi dei seguenti tre casi: • Quando un utente funzionale invia un Entry d'ingresso che avvia il processo funzionale. • Quando un processo funzionale, essendo iniziato, è in attesa, aspettando il successivo Entry da parte dell'utente funzionale (tipicamente, nel software applicativo aziendale, da parte di un utente funzionale umano) • Quando un processo funzionale, essendo iniziato, richiede all'utente funzionale "inviarmi i tuoi dati ora, se ne hai" e l'utente funzionale invia i dati (tipicamente noto come "polling" nel software real-time). Nell'ultimo caso, per convenzione, non si deve identificare alcun Exit da parte del processo funzionale per ottenere i dati richiesti. Il processo funzionale deve semplicemente inviare un messaggio di sollecito ("prompt

ID	NOME	DESCRIZIONE REGOLE
		message”) all’utente funzionale per inserire i propri dati e la funzionalità di tale messaggio di sollecito è considerata parte dell’Entry. L’utente funzionale sa cosa deve inviare e il processo funzionale sa cosa deve attendere. È necessario solo un Entry in questo caso. V. anche principio c) per l’Entry, sez. 4.1.2. b) Laddove un processo funzionale necessiti di ottenere i servizi di un utente funzionale (per es. per ottenere dei dati) e occorre specificare all’utente funzionale cosa inviare (tipicamente dove l’utente funzionale è un’altra porzione di software, esterna all’ambito del software oggetto di misurazione), si deve identificare una coppia di movimenti di dati Exit/Entry. L’Exit contiene la richiesta dei dati specifici; l’Entry contiene i dati di risposta.
R-11	Comandi di controllo nel dominio applicativi aziendali	Nel dominio degli applicativi aziendali, i “comandi di controllo” devono essere ignorati poiché non comportano alcun movimento di dati riguardante un oggetto d’interesse.
R-12	Aggregare i risultati della misurazione	a) Per qualsiasi processo funzionale, le dimensioni funzionali dei singoli movimenti di dati si devono aggregare in un unico valore di dimensione funzionale in unità di CFP sommandole aritmeticamente tra loro. $\text{Dim. (processo funzionale)}_i = \sum \text{dim. (Entry)}_i + \sum \text{dim. (Exit)}_i + \sum \text{dim. (Read)}_i + \sum \text{dim. (Write)}_i$ b) Per qualsiasi processo funzionale, la dimensione funzionale delle modifiche ai suoi Requisiti Utente Funzionali si deve aggregare a partire dalle dimensioni dei movimenti di dati che sono stati aggiunti, modificati o cancellati nel processo funzionale, per fornire una dimensione della modifica in unità di CFP, secondo la seguente formula. $\text{Dim. (Modif. (proc. funzionale))}_i = \sum \text{dim. (movimenti di dati aggiunti)}_i + \sum \text{dim. (movimenti di dati modificati)}_i + \sum \text{dim. (movimenti di dati cancellati)}_i$ Per ulteriori dettagli sull’aggregazione delle dimensioni funzionali, v. sez. 4.3.2. Per la misurazione della dimensione del software modificato, v. sez. 4.4. c) La dimensione di una porzione di software in un dato ambito si deve ottenere aggregando le dimensioni di tutti processi funzionali appartenenti a quella porzione, nel rispetto delle regole e) e f) seguenti. d) La dimensione di una qualsiasi modifica di una porzione di software in un dato ambito si deve ottenere aggregando le dimensioni di tutte le modifiche di tutti i processi funzionali appartenenti a quella porzione, nel rispetto delle regole e) e f) seguenti. e) Le dimensioni di porzioni di software o di modifiche di porzioni di software in un qualsiasi strato possono essere sommate tra loro solo se sono tutte misurate al medesimo livello di granularità dei processi funzionali dei rispettivi FUR. f) Le dimensioni di porzioni di software e/o di modifiche alle dimensioni di porzioni di software in un qualsiasi strato, o in differenti strati, devono essere sommate tra loro solo se ha senso farlo, alla luce dello scopo della misurazione. g) La dimensione di una porzione di software non può ottenersi sommando le dimensioni delle sue componenti (a prescindere dalla particolare scomposizione della porzione) <i>a meno che</i> non si eliminino i contributi alla dimensione dei movimenti di dati inter-componenti. h) Se il metodo COSMIC viene esteso localmente (per es. per misurare aspetti dimensionali non coperti dal metodo standard), la dimensione misurata tramite l’estensione locale deve allora essere riportata separatamente, come descritto in sez. 5.1 e NON può essere sommata alla dimensione ottenuta tramite il metodo standard, misurata in CFP (v. oltre, sez. 4.5).
R-13	Modifica di un	a) Se un movimento di dati deve essere modificato a causa di una variazione della manipolazione dei dati associate con il movimento dei dati e/o a causa

ID	NOME	DESCRIZIONE REGOLE
	movimento di dati	di una variazione nel numero o nel tipo degli attributi del gruppo di dati mosso, si misura un CFP modificato, indipendentemente dal numero reale di modifiche nell'unico movimento di dati. b) Se deve essere modificato un gruppo di dati, i movimenti di dati che muovono il gruppo di dati modificato le cui funzionalità non sono impattate dalla modifica del gruppo di dati non sono identificati come movimenti di dati modificati. NOTA 1. Una modifica di un valore di un'occorrenza di un attributo, come la modifica di un codice individuale di un attributo i cui valori costituiscono un sistema di codifica, non è una modifica del tipo dell'attributo. NOTA 2. Una modifica di un qualsiasi dato che compare su schermate di input o output che non sia collegato a un oggetto d'interesse di un utente funzionale non è identificata come un CFP modificato (v. sez. 3.3.4 per esempi di tali dati).
R-14	Etichettatura di una misurazione COSMIC	Un risultato di misurazione COSMIC è riportato come "x CFP (v.y) ", dove: • "x" rappresenta il valore numerico della dimensione funzionale, • "v.y" rappresenta l'identificativo della versione standard del metodo COSMIC usata per ottenere il valore numerico della dimensione funzionale „x“. ESEMPIO. Un risultato ottenuto con le regole del presente Manuale di Misurazione è indicato come "x CFP (v3.0)". NOTA. Se si è fatto uso di un metodo di approssimazione locale per ottenere la misurazione, ma, a parte ciò, la misurazione è stata effettuata tramite le convenzioni di una versione standard del metodo COSMIC, si usi la precedente convenzione di etichettatura, ma si annoti altrove l'utilizzo del metodo di approssimazione – v. sez. 5.2.
R-15	Etichettatura di estensioni locali COSMIC	Un risultato di misurazione COSMIC ottenuto mediante estensioni locali deve essere riportato come: "x CFP (v. y) + z Local FP", dove: • "x" rappresenta il valore numerico ottenuto aggregando tutti i singoli risultati di misurazione conformi al metodo COSMIC standard, versione v.y, • "v.y" rappresenta l'identificativo della versione standard del metodo COSMIC usata per ottenere il valore numerico della dimensione funzionale „x“, • "z" rappresenta il valore numerico ottenuto aggregando tutti i singoli risultati di misurazione ottenuti mediante estensioni locali del metodo COSMIC.
R-16	Presentazione delle misurazioni COSMIC	In aggiunta alle misurazioni effettive, registrate come indicato in sez. 5.1, per ogni misurazione si dovrebbero registrare i seguenti attributi. a) Identificativo della componente software misurata (nome, ID di versione o ID di configurazione). b) Le fonti informative usate per identificare i FUR usati per la misurazione. c) Il dominio del software. d) Lo scopo della misurazione. e) Una descrizione dell'ambito della misurazione e la sua relazione con l'ambito complessivo di un gruppo relazionato di misurazioni, qualora ve ne siano (usare le categorie di ambito generiche di sez. 2.2). f) Gli utenti funzionali del software. g) Il livello di granularità dei FUR e il livello di scomposizione del software. h) Il momento nel ciclo di vita del progetto in cui è stata effettuata la misurazione (specialmente se la misurazione è una stima basata su FUR incompleti, o è stata effettuata sulla base di funzionalità effettivamente rilasciate). i) Il margine di errore, ricercato o valutato, della misurazione. j) Indicazioni se si è applicato il metodo di misurazione COSMIC standard e/o un'approssimazione locale del metodo standard e/o estensioni locali (v. sez. 4.5). Si usino le convenzioni di etichettatura di sez. 5.1 o sez. 5.2. k) Un'indicazione se la misurazione è relativa a funzionalità sviluppate o consegnate (le funzionalità "sviluppate" sono ottenute creando nuovo software; le funzionalità "consegnate" includono le funzionalità "sviluppate" e anche le funzionalità ottenute con altri mezzi rispetto alla creazione di nuovo software, cioè incluse tutte le forme di riuso di software esistente, l'uso di

ID	NOME	DESCRIZIONE REGOLE
		parametri esistenti per aggiungere o modificare funzionalità, ecc.). l) Un'indicazione se la misurazione è di funzionalità fornite ex novo o è il risultato di un'attività "evolutiva" (per es. la somma delle funzionalità aggiunte, modificate e cancellate – v. sez. 4.4). m) Una descrizione dell'architettura degli strati in cui è svolta la misurazione, se applicabile. n) Il numero di componenti principali, se applicabile, le cui dimensioni sono state sommate tra loro per fornire la dimensione totale registrata. o) Per ogni ambito compreso nell'ambito complessivo della misurazione, una matrice di misurazione, come specificato in App. A. p) Il nome del misuratore e tutte le eventuali qualifiche di certificazione COSMIC.

APPENDICE D – STORIA DEI RILASCI DEL METODO COSMIC

Questa appendice contiene un riepilogo delle principali modifiche apportate nel derivare la versione 3.0 del metodo di misurazione della dimensione funzionale COSMIC dalla precedente versione 2.2. La versione 2.2 del metodo era interamente descritta nel Manuale di Misurazione (Measurement Manual, MM) v2.2, ma la documentazione della versione 3.0 è ora distribuita su quattro documenti, uno solo dei quali è intitolato „Manuale di Misurazione”.

Scopo di questa appendice è permettere al lettore familiare con il MM v2.2 di tenere traccia delle modifiche apportate e ove necessario di comprenderne le motivazioni. (Per i cambiamenti apportati nel derivare la versione 2.2 a partire dalla precedente versione 2.1, si veda il MM v2.2, App. E.)

Nel seguito si fa riferimento a due “Method Update Bulletins” (MUB). Un MUB è pubblicato dal COSMIC per proporre miglioramenti del metodo tra due rilasci principali della definizione dello stesso. I due MUB sono:

- MUB 1 “Proposed Improvements to the definition and characteristics of a software „Layer””, pubblicato nel Maggio 2003.
- MUB 2 “Proposed Improvement to the definition of an „object of interest””, pubblicato nel Marzo 2005.

Nel processo di aggiornamento del metodo dalla v2.2 alla v3.0, si è speso tempo per razionalizzare la guida tra “principi” e “regole” e per separare tutti gli esempi dai principi e dalle regole. Modifiche di questo tipo e molti altri miglioramenti di natura editoriale non sono riportati nel seguito.

Rif. v2.2	Rif. v3.0	Modifica
Ristrutturazione del Metodo COSMIC		
2.2, 2.7, 3.1, 3.2	Sez. 1.5, Cap. 2	Distinta una fase di “Strategia della Misurazione” come la prima fase di quello che ora è un metodo composto di tre fasi. La fase di Strategia della Misurazione ora include la trattazione di “strati”, “confini” e “utenti (funzionali)”, considerati in precedenza parte della fase di Mappatura nel MM v2.2.
Ristrutturazione del Manuale di Misurazione		
		Nel produrre la v3.0 del Metodo COSMIC, il MM v2.2 è stato suddiviso in quattro documenti per facilità d'uso: • “Documentation Overview and Glossary of Terms” (nuovo) • “Method Overview” (Cap. 1 e 2 del MM v2.2) • “Manuale di Misurazione” (Cap. 3, 4 e 5 e appendici del MM v2.2) • “Advanced and Related Topics” (Cap. 6 e 7 del MM v2.2) Riferimenti all’esperienza originale agli aspetti ed articoli di ricerca sono stati rimossi dal MM. Essi sono ora disponibili all’indirizzo www.cosmicon.com. Gli unici riferimenti rimanenti in questi quattro documenti sono riportati in note a fondo pagina.

Cap. 4	Cap. 4	Il capitolo sulla Fase di Misurazione è stato ristrutturato in maniera considerevole per fornire un'esposizione più logica.
App. D	--	L'appendice su "Ulteriori riflessioni sugli strati software" è stata rimossa, essendo incompatibile con il MUB 1 e aggiungendo poco valore informativo. Si vedano anche le note seguenti sulle modifiche derivanti dal MUB 1.
Modifiche di nomi e terminologia		
Generale		Semplificato il nome del metodo "COSMIC-FFP" in metodo "COSMIC".
5.1	4.2	Modificato il nome dell'unità di misura, "COSMIC functional size unit" ("Cfsu") in "COSMIC Function Point" ("CFP"). V. la Prefazione del presente MM v3.0 per una spiegazione di tale modifica.
4.1	4.1.7	Rinominata la regola di "De-duplicazione dei Movimenti di Dati" nella regola di "Unicità dei movimenti di dati".
Concetti nuovi, modificati ed eliminati		
2.7	2.3	Rimossi i concetti di "astrazione", "punto di vista", "Punto di Vista della Misurazione", "Punto di Vista della Misurazione dell'Utente Finale" e "Punto di Vista della Misurazione dello Sviluppatore". Essi sono rimpiazzati dal concetto più generale che la dimensione funzionale di una porzione di software oggetto di misurazione dipenda dalle funzionalità rese disponibili all'"utente/i funzionali" del software. Questi ultimi dovrebbero essere identificabili nei Requisiti Utente Funzionali del software oggetto di misurazione. Definire l'"utente/i funzionali" è quindi un pre-requisito per definire quale dimensione debba essere misurata o per interpretare una misurazione di dimensione esistente. V. la Prefazione del presente MM v3.0 per una spiegazione più dettagliata di tale modifica.
3.2	2.3	Sostituito il concetto di "utente" (così come definito nello standard ISO/IEC 14143 Parte 1) con il concetto di "utente funzionale", che è un concetto più delimitato. V. la Prefazione del presente MM v3.0 per una spiegazione più dettagliata di tale modifica. Introdotti in sez. 2.3.2 due esempi di dove la dimensione funzionale varia a seconda del tipo di utente funzionale identificato nei FUR, per un telefono cellulare e per un pacchetto di software applicativo aziendale.
--	2.2.2	Introdotta il "livello di scomposizione" del software oggetto di misurazione nella discussione dell'Ambito della Misurazione.
--	2.4	Introdotta il "livello di granularità" dei Requisiti Utente Funzionali del software oggetto di misurazione nella discussione della Strategia della Misurazione. Fornito un esempio esauriente in merito. V. la Prefazione del presente MM v3.0 per una spiegazione più dettagliata di tale modifica.
--	2.4.3	Definito un "livello di granularità dei processi funzionali", corredato da regole corrispondenti e una raccomandazione.
3.4	4.2	Rimosso il concetto di "Persistenza di un Gruppo di Dati", che includeva tre tipi (livelli) di persistenza, ed esattamente "temporanea", "breve" e "indefinita", poiché si è dimostrato non necessario. Introdotta il concetto di "memoria permanente".
4.1	--	Il concetto di "de-duplicazione" è stato rimosso poiché si è dimostrato non necessario.
Definizioni, principi e regole migliorate e perfezionate		
2.3	2.2	Modificata la definizione di "Requisiti Utente Funzionali" per conformità all'edizione 2007 dello standard ISO/IEC 14143 Parte 1.
2.4.1	1.3	Il "Modello del Contesto del Software" è stato esteso e perfezionato in termini dei relativi principi.
2.4.2	1.4	Il "Modello Generale del Software" è stato esteso e perfezionato in termini dei relativi principi.
2.4, 3.1	2.2.3, 2.2.4,	Aggiornata la definizione di "strato" in considerazione del MUB 1. Sostituite le Fig. 2.4.1.1, 2.4.1.2 e 2.4.1.3 del MM v2.2, usate per illustrare

	3.1	l'interazione degli utenti del software a "strati", con le Fig. 2.2.3.1 e 2.2.3.2 della v3.0, illustranti tipiche architetture fisiche a strati, e con le Fig. 3.1.1 e 3.1.2, illustranti l'interazione logica degli utenti funzionali con il software a strati. Obiettivo di tale modifica è distinguere in modo più chiaro la vista <i>fisica</i> di tipiche architetture software a strati dalla vista <i>logica</i> di un utente funzionale che interagisce con una porzione di software oggetto di misurazione secondo il modello COSMIC.
--	3.2.4	Definito il concetto di "componente alla pari" e stabiliti i suoi principi in considerazione del MUB 1.
2.5	4.2, 4.3	Riscritte le "caratteristiche" del processo di misurazione descritto nella v2.2 come insieme di principi e regole nella v3.0.
2.6	2.4	Parte della trattazione sul "dimensionamento anticipato nella vita di un progetto: scalabilità delle misurazioni" è stata spostata nel MM v3.0, sez. 2.4, alla voce "Livello di Granularità" e ulteriormente elaborata nel documento "Advanced and Related Topics".
2.7	2.2	Aggiunta una nota alla definizione di "ambito" per distinguere l'"ambito complessivo" dell'esecuzione di una misurazione (che può includere diverse porzioni separate di software la cui dimensione debba essere misurata) dall'"ambito" di una singola misurazione di dimensione.
3.2	4.1.8	Spostati e riscritti tre casi illustranti l'interazione degli utenti attraverso un confine con il software in strati differenti e da differenti "punti di vista della misurazione", alla luce dell'eliminazione dei "punti di vista della misurazione". V. nel seguito sez. 4.1.8 (del MM v3.0).
--	3.1	Aggiunto un principio riguardante l'applicazione del "Modello Generale del Software" al software oggetto di misurazione.
3.3	3.2.1	Perfezionata la definizione di "processo funzionale" tenendo conto dell'introduzione del concetto di "utente funzionale" che sostituisce il termine "attore" in tale definizione.
3.3	3.2.1	Perfezionata la definizione di "evento d'innescò".
--	3.2.1	Chiarita la relazione tra evento d'innescò, utente funzionale, Entry d'innescò e processo funzionale in Fig. 3.1.1.
3.1	3.2.2	Combinati i principi e le regole per un "processo funzionale" in un insieme di regole aggiornato.
--	3.2.3, 3.2.4, 3.2.5	Introdotti numerosi esempi di processi funzionali e di come distinguerli tra loro.
--	3.3.1	Introdotta la definizione di "oggetto d'interesse", in linea con il MUB 2.
3.4	3.3.3	Gli esempi di identificazione degli oggetti di interesse e dei gruppi di dati sono stati separati dalle regole sui gruppi di dati. Rimosso il materiale domini specifici sulle convenzioni dell'analisi E/R (Entity-Relationship) e spostato nel documento "Guideline for Sizing Business Application Software", v1.0.
--	3.3.4	Fornite indicazioni su "dati o gruppi di dati che non sono candidati per essere riconosciuti come movimenti di dati".
--	3.3.5	Fornite indicazioni su quando, tipicamente nel software real-time, potrebbe non essere indispensabile distinguere tra un "utente funzionale" e un "oggetto d'interesse" di cui vengano mossi dei dati.
3.5	3.4	Ridotta e semplificata la discussione degli "attributi dei dati", poiché la considerazione degli attributi dei dati non costituisce una parte obbligatoria del metodo.
4.1	4.1.1	Razionalizzata la definizione di "movimento di dati".
4.1	4.1.7	Chiarite significativamente le regole sulla "de-duplicazione dei movimenti di dati" (ora rinominate come regole sull'"unicità dei movimenti di dati e possibili eccezioni") con l'aggiunta di diversi esempi.
4.1	--	Rimosse regole specifiche per dominio sui movimenti di dati per Read e Write nei processi funzionali di tipo "Update" e spostate nel documento "Guideline for Sizing Business Application Software", v1.0.

4.1	4.1.8	Le regole sulla “corrispondenza dei movimenti di dati attraverso i confini” (applicate nel MM v2.2 al “Punto di Vista della Misurazione dello Sviluppatore”, ora eliminato) sono state rimosse, ma i relativi concetti sono stati combinati con i casi precedentemente illustrati nel MM v2.2, sez. 3.2, producendo una nuova sezione su “quando un processo funzionale muove dati da o verso la memoria permanente”.
4.1	4.1.6	Definito il concetto di “manipolazione dei dati” e aggiunto un principio sulla “manipolazione dei dati associata ai movimenti di dati”. Estese le linee guida sulla manipolazione dei dati associata con i diversi tipi di movimento di dati.
4.1.1	4.1.2	Razionalizzati i principi e le regole dell’“Entry”. Aggiunto un nuovo principio relativo alle funzionalità di “richiesta di inserimento”.
4.1.2	4.1.3	Razionalizzati i principi e le regole dell’“Exit”, includendo la rimozione del riferimento al “Punto di Vista della Misurazione dell’Utente Finale”.
4.1.3	4.1.4	Razionalizzati i principi del “Read”. Aggiunto un nuovo principio relativo alle funzionalità di „richiesta di lettura”.
4.1.4	4.1.5	Razionalizzati i principi del “Write”.
--	4.1.9	Aggiunte nuove regole su “quando un processo funzionale richiede dati da un utente funzionale”.
--	4.1.10	Introdotta una definizione e una regola sul concetto di “comando di controllo”, che è valido solo nel dominio del software applicativo aziendale.
4.1.5	4.5	Estesa la trattazione delle “estensioni locali del metodo”.
4.3	4.3	Trasformati i principi di “Aggregazione dei Risultati della Misurazione” in “regole” ed estesi in modo da tener conto delle regole per ottenere la dimensione di una porzione di software sommando le dimensioni delle sue componenti. Il MM v2.2 faceva riferimento a tali regole solo nel contesto del “Punto di Vista della Misurazione dello Sviluppatore”. Tale restrizione è stata rimossa.
--	4.4	Introdotta una nuova sezione sulla “misurazione della dimensione delle modifiche del software” e aggiunte nuove regole.
5.1	5.1	Modificate le regole per etichettare i risultati della misurazione, per riconoscere la modifica dell’unità di misura da “Cfsu” a “CFP”.
5.2	5.2	Estese le regole sulla “Presentazione delle Misurazioni”, per elencare un maggior numero di elementi.
App. B	App. B	Aggiornata ai Principi della v3.0.
App. C	App. C	Aggiornata alle Regole della v3.0.

APPENDICE E – PROCEDURA PER RICHIESTE DI MODIFICA E COMMENTI

Il COSMIC Measurement Practices Committee (MPC) è lieto di ricevere feedback, commenti e, se necessario, richieste di modifica (Change Request) riguardanti il Manuale di Misurazione COSMIC. Questa appendice illustra le modalità per comunicare con il COSMIC MPC.

Tutte le comunicazioni dirette al COSMIC MPC vanno inviate per e-mail al seguente indirizzo:

mpc-chair@cosmicon.com

Feedback e commenti generici informali

Commenti informali e feedback sul Manuale di Misurazione, come per es. difficoltà nella comprensione o nell'applicazione del metodo COSMIC, suggerimenti di miglioramento generale, ecc. dovrebbero essere inviati per e-mail all'indirizzo sopra riportato.

Si prenderà atto dei messaggi e sarà data di norma notifica entro due settimane dalla ricezione. L'MPC non può garantire di mettere necessariamente in pratica tali commenti generici.

Richieste di Modifica formali

Qualora il lettore del Manuale di Misurazione ritenga che vi sia un errore nel testo o la necessità di un chiarimento, o che una porzione del testo richieda di essere migliorata, è possibile sottoporre una Richiesta di Modifica (Change Request, CR) formale.

Si prenderà atto delle CR formali e sarà data notifica entro due settimane dalla ricezione. A ogni CR sarà quindi assegnato un numero progressivo e essa sarà fatta circolare tra i membri del COSMIC MPC, un gruppo di esperti a livello mondiale del metodo COSMIC. Il normale ciclo di revisione richiede un minimo di un mese e può richiedere più tempo se la CR si rivela di difficile risoluzione.

L'esito dell'esame della CR può essere "accettata", "rifiutata" o "in attesa di ulteriore discussione" (nell'ultimo caso per es. se esiste una qualche relazione con un'altra CR) e sarà comunicato al mittente non appena possibile.

Una CR formale sarà accettata solo se corredata di tutte le seguenti informazioni.

- Nome, qualifica e organizzazione di appartenenza di chi sottopone la CR.
- Riferimenti di contatto di chi sottopone la CR.
- Data di invio.
- Scopo generale della CR (per es. "occorre migliorare il testo ...").
- Testo esistente che richiede modifica, sostituzione o cancellazione (o chiaro riferimento ad esso).
- Testo aggiuntivo o sostitutivo proposto.
- Spiegazione esauriente del motivo per cui è necessaria la modifica.

Un modulo per sottoporre una CR è disponibile presso il sito web www.cosmicon.com.

La decisione del COSMIC MPC sull'esito dell'esame di una CR e, se accettata, sulla versione del Manuale di Misurazione dalla quale si applicherà la CR è definitiva.

Quesiti sull'applicazione del metodo COSMIC

Il COSMIC MPC si rammarica di non poter rispondere a quesiti relativi all'utilizzo o all'applicazione del metodo COSMIC. Esistono organizzazioni commerciali che possono fornire formazione e consulenza o strumenti di supporto sul metodo. Si prega di consultare il sito web www.cosmicon.com per ulteriori dettagli.